Linear algebra

이창민

어영정교수님 수업자료를 일부 수정하였습니다

Vectors and Matrices

한개의 sample에 여러 값이 있는 경우

학생 1의 TOEFL 점수

- Reading 30
- Listening 10
- Speaking 15
- Writing 25

학생 2의 TOEFL 점수

- Reading 10
- Listening 30
- Speaking 25
- Writing 10

student1_reading = 30

…

student2_writing = 10

한개의 sample에 여러 값이 있는 경우 → list로 표현하기?

학생 1의 TOEFL 점수

- Reading 30
- Listening 10
- Speaking 15
- Writing 25

학생 2의 TOEFL 점수

- Reading 10
- Listening 30
- Speaking 25
- Writing 10

student1 = [30, 10, 15, 25]

student2 = [10, 30, 25, 10]

Vector

An array of numbers

vector는 lowercase, bold로 표시
element들은 진하지 않게, 아래첨자로 index 표시

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

vectors are points in some finite-dimensional space

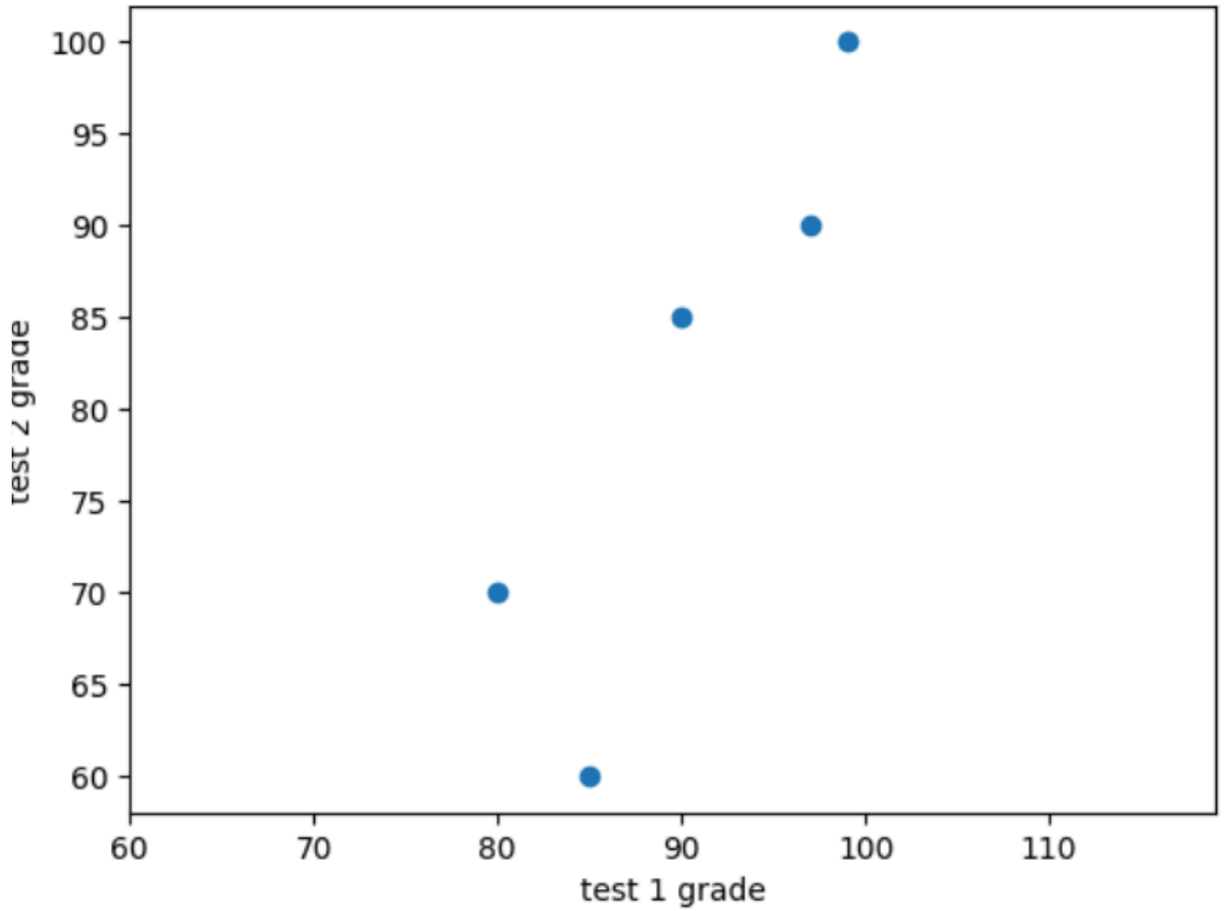
```
test_1_grades = [ 99, 90, 85, 97, 80]
test_2_grades = [100, 85, 60, 90, 70]

plt.scatter(test_1_grades, test_2_grades)
plt.title("Axes Aren't Comparable")
plt.xlabel("test 1 grade")
plt.ylabel("test 2 grade")
plt.show()
```

student1 = [99, 100]

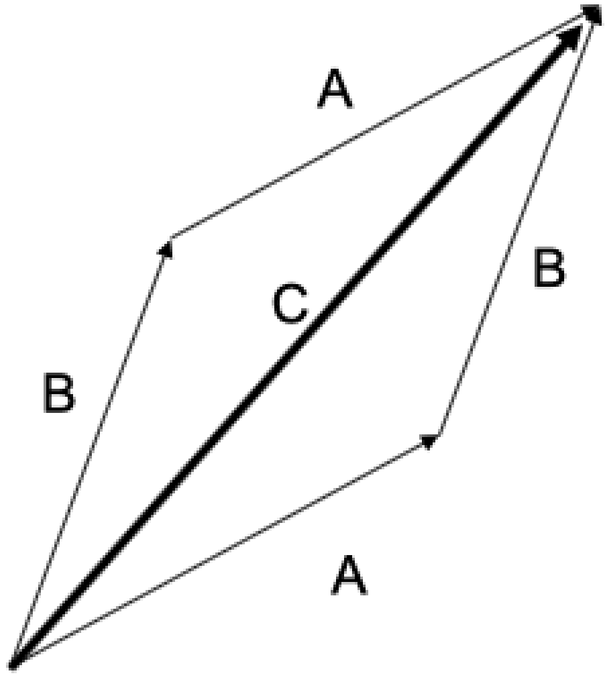
…

student5 = [80, 70]



vector의 덧셈

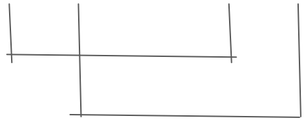
P1 + P2 = (x1, y1) + (x2, y2) = ((x1+x2), (y1+y2))



A+B = C
(use the head-to-tail method to combine vectors)

vector의 덧셈

(1, 2) + (2, 1) = ((1 + 2), (2 + 1)) = (3, 3)



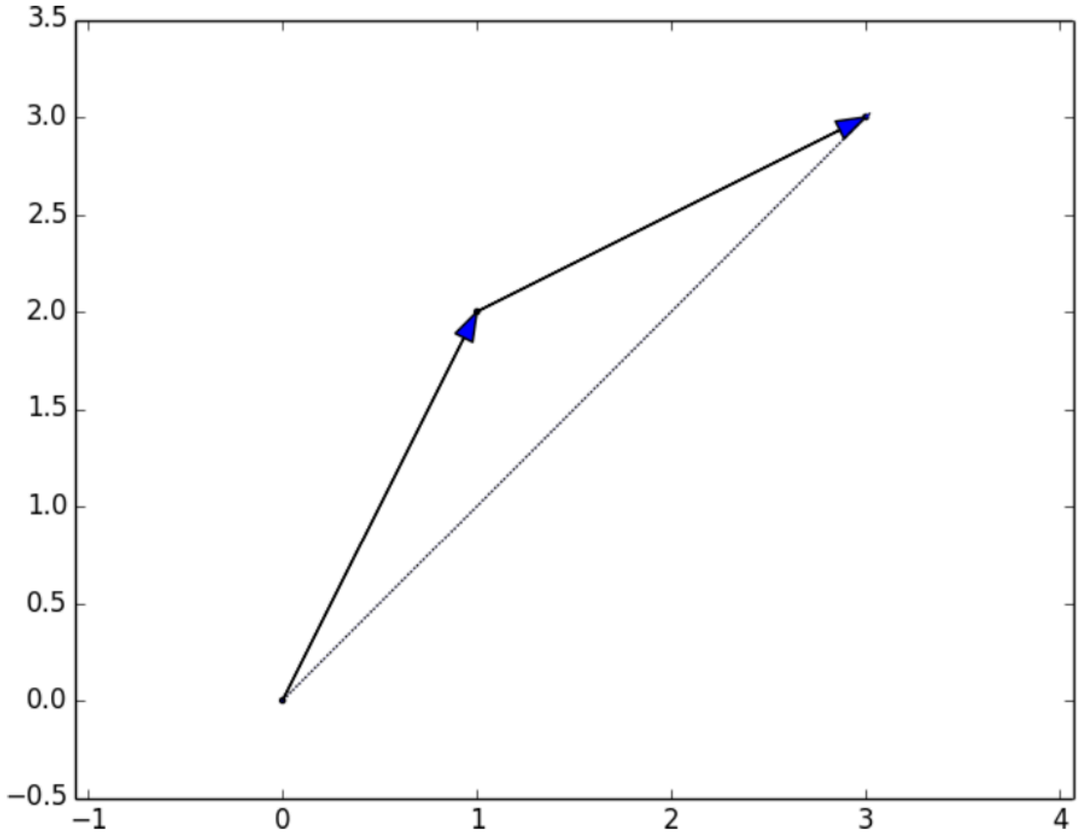


Figure 4-1. Adding two vectors

```python
def add(v: Vector, w: Vector) -> Vector:
    """Adds corresponding elements"""
    assert len(v) == len(w), "vectors must be the same length"

    return [v_i + w_i for v_i, w_i in zip(v, w)]

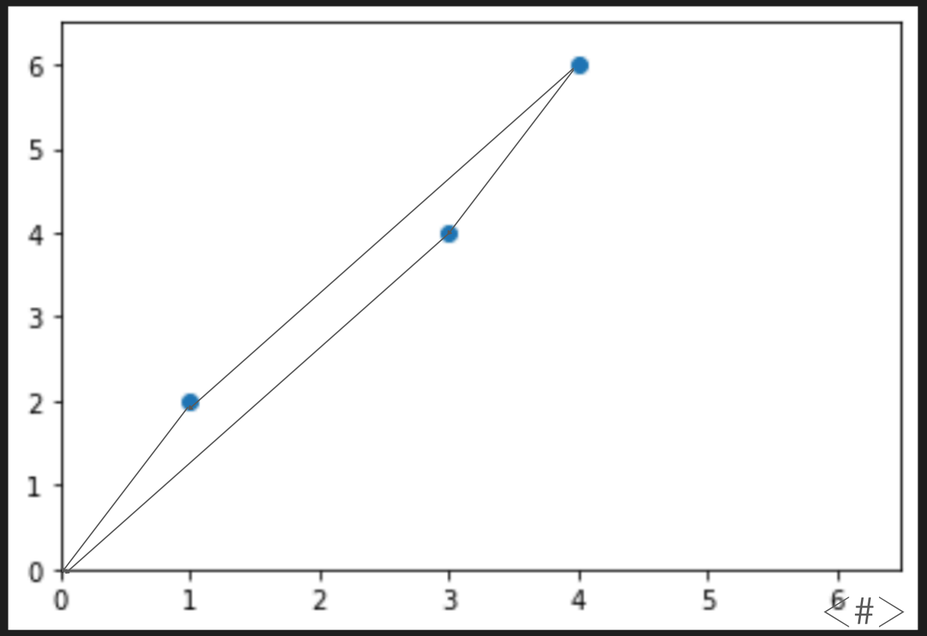
assert add([1, 2, 3], [4, 5, 6]) == [5, 7, 9]
```

vector의 덧셈 in numpy

```
vector1 = np.array([1, 2])
vector2 = np.array([3, 4])
vector3 = vector1 + vector2
print(vector3)
stack = np.stack([vector1, vector2, vector3])
print(stack)
plt.scatter(stack[:, 0], stack[:, 1])
plt.xlim(0, 6.5)
plt.ylim(0, 6.5)
plt.show()
```

[17] ✓ 0.1s

```
[4 6]
[[1 2]
 [3 4]
 [4 6]]
```



vector * scalar

$$a\mathbf{v} = a(x_1, x_2) = (ax_1, ax_2)$$

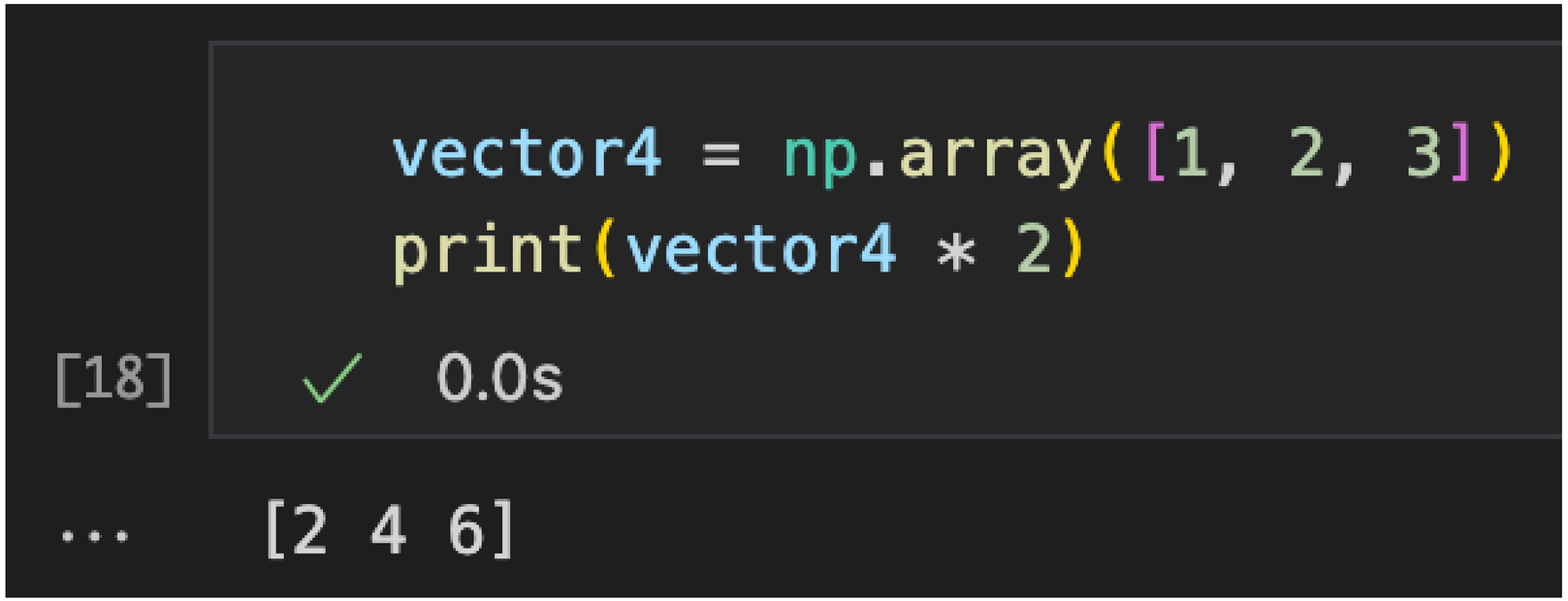


vector * scalar

```python
def scalar_multiply(c: float, v: Vector) -> Vector:
    """Multiplies every element by c"""
    return [c * v_i for v_i in v]

assert scalar_multiply(2, [1, 2, 3]) == [2, 4, 6]
```

in numpy



mean vector

```python
def vector_mean(vectors: List[Vector]) -> Vector:
    """Computes the element-wise average"""
    n = len(vectors)
    return scalar_multiply(1/n, vector_sum(vectors))

assert vector_mean([[1, 2], [3, 4], [5, 6]]) == [3, 4]
```

in numpy

```
    vector1 = np.array([1, 2])
    vector2 = np.array([3, 4])
    vector3 = np.array([5, 6])
    print(sum([vector1, vector2, vector3]) / 3)
[19]  ✓  0.0s

...  [3. 4.]
```

easier mean vector in numpy

```
    vector1 = np.array([1, 2])
    vector2 = np.array([3, 4])
    vector3 = np.array([5, 6])
    print(sum([vector1, vector2, vector3]) / 3)
```

[19] ✓ 0.0s

```
[3. 4.]
```

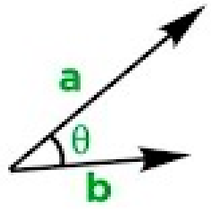
```
    vectors = np.stack([vector1, vector2, vector3])
    print(vectors)
    print(vectors.mean(axis=0))
```

[20] ✓ 0.0s

```
[[1 2]
 [3 4]
 [5 6]]
[3. 4.]
```

compute mean over 0-th axis

dot product



$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$$

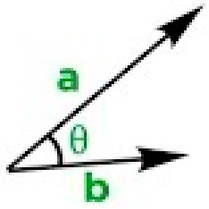
$$\begin{bmatrix} a & b \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \end{bmatrix}$$

```
def dot(v: Vector, w: Vector) -> float:
    """Computes v_1 * w_1 + ... + v_n * w_n"""
    assert len(v) == len(w), "vectors must be same length"

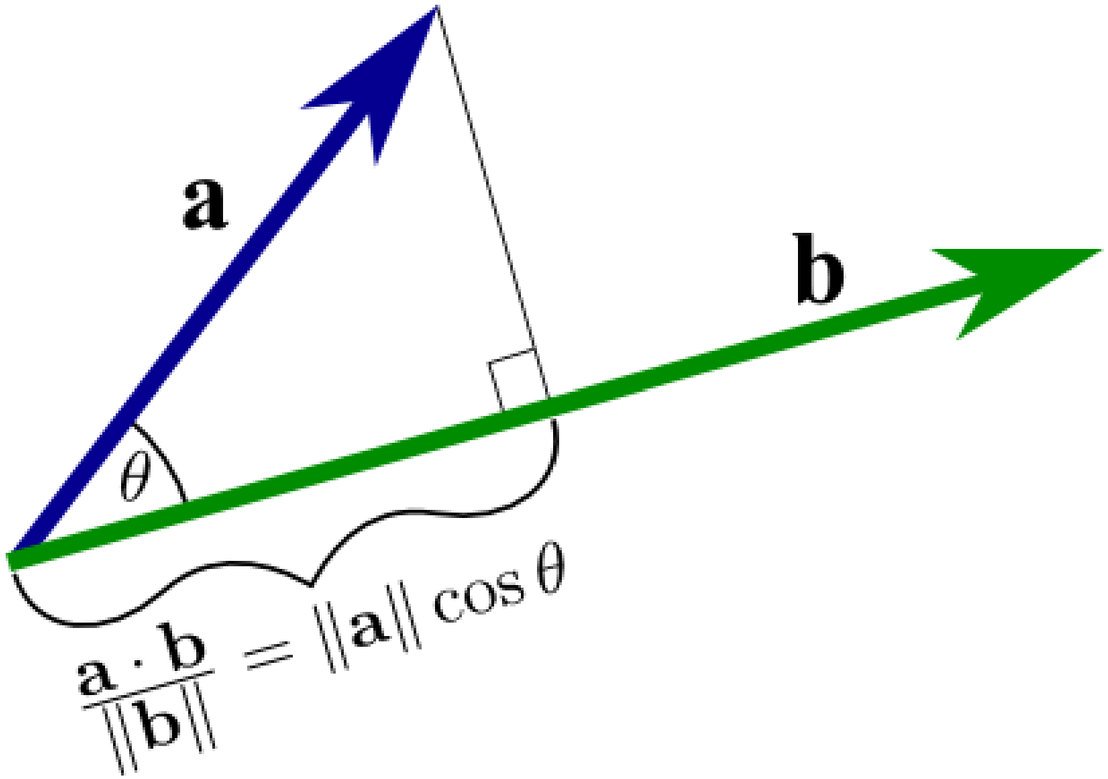
    return sum(v_i * w_i for v_i, w_i in zip(v, w))

assert dot([1, 2, 3], [4, 5, 6]) == 32  # 1 * 4 + 2 * 5 + 3 * 6
```

dot product

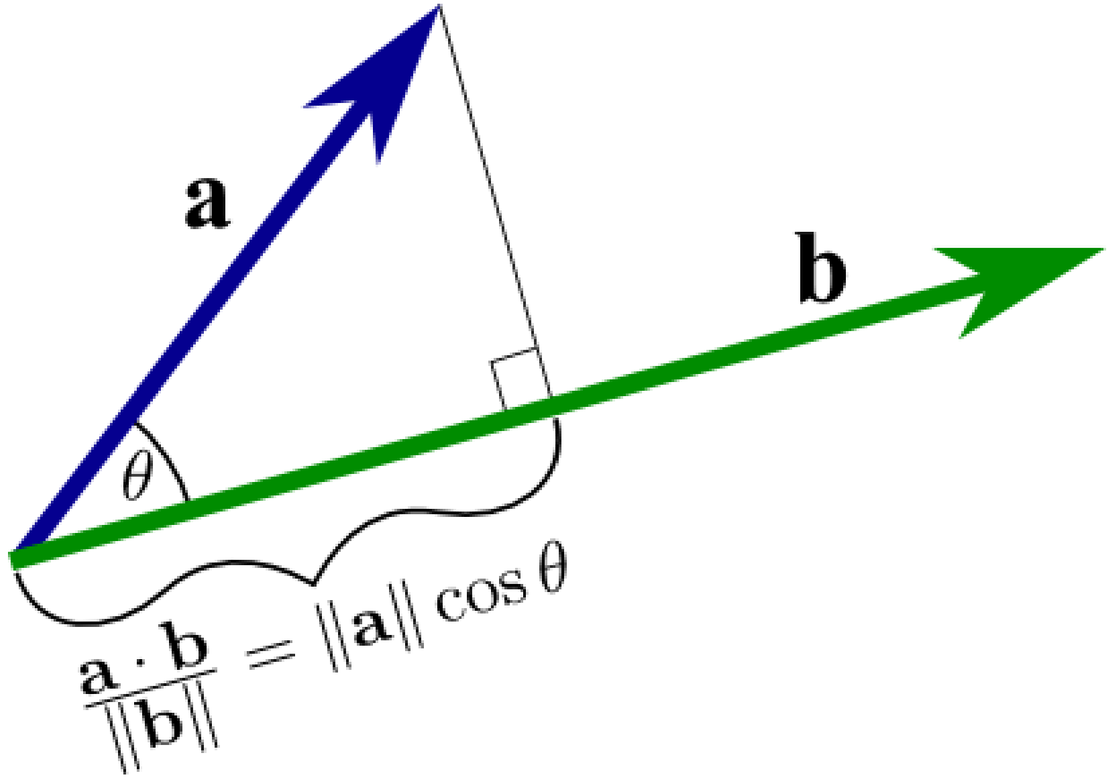


$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$$



dot product with a unit vector

unit vector = a vector of length (magnitude) 1
The dot product measures how far the vector extends along the unit vector



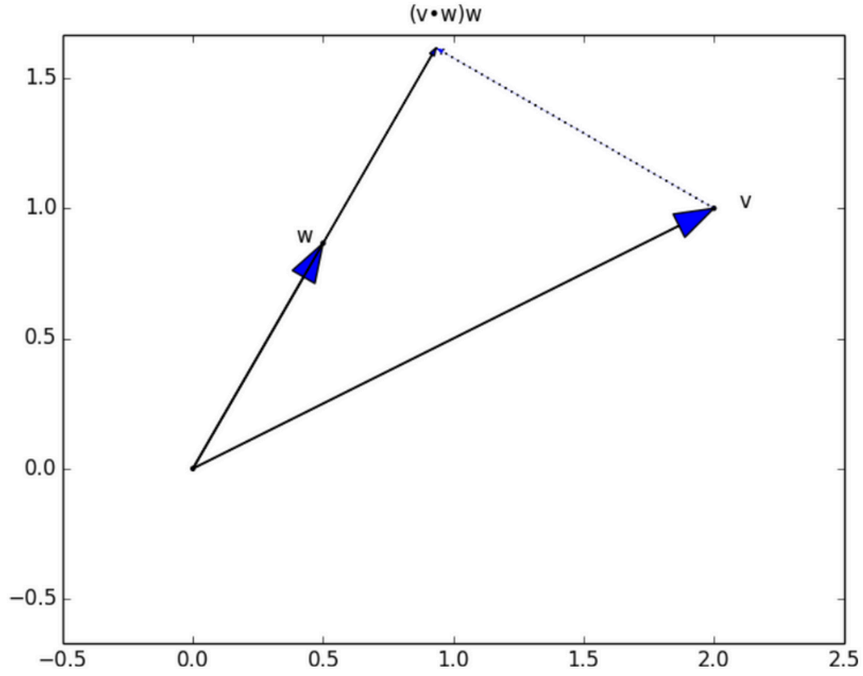


Figure 4-2. The dot product as vector projection

dot product in numpy

```
vector4 = np.array([1, 2, 3])
vector5 = np.array([4, 5, 6])
print(vector4.dot(vector5))
print(np.dot(vector4, vector5))
```

[21] ✓ 0.1s

```
32
32
```

a vector’s sum of squares

```
def sum_of_squares(vec):
    return sum([e * e for e in vec])

vector4 = np.array([1, 2, 3])
print(sum_of_squares(vector4))
print(np.dot(vector4, vector4))
```

[22] ✓ 0.0s

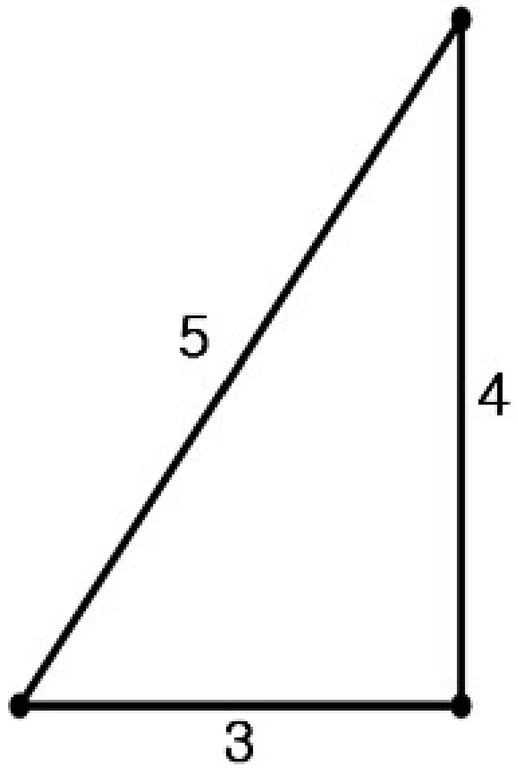
```
14
14
```

a vector’s magnitude (length)

```
vector6 = np.array([3, 4])
print(np.sqrt(np.dot(vector6, vector6)))
print(np.linalg.norm(vector6))
```

[23] ✓ 0.0s

```
5.0
5.0
```



Matrix

A matrix is a two-dimensional collection of numbers.
If is a matrix, then A[i][j] is the element in the i-th row and the j-th column.

```
A = [[1, 2, 3],  # A has 2 rows and 3 columns
     [4, 5, 6]]

B = [[1, 2],     # B has 3 rows and 2 columns
     [3, 4],
     [5, 6]]
```

Matrix

A matrix is a two-dimensional collection of vectors.
If is a matrix, then A[i][j] is the element in the i-th row and the j-th column.

```
A = [[1, 2, 3],  # A has 2 rows and 3 columns
     [4, 5, 6]]
# A is a 2x3 matrix
# A has 3 column vectors

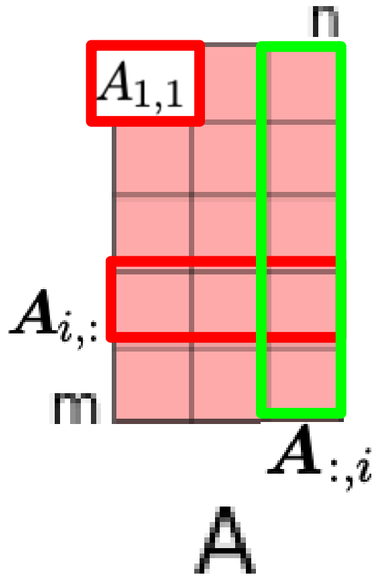
B = [[1, 2],     # B has 3 rows and 2 columns
     [3, 4],
     [5, 6]]
# B is a 3x2 matrix
# B has 2 column vectors
```

Matrix의 표기법

A matrix: Bold uppercase $\boldsymbol{A}$

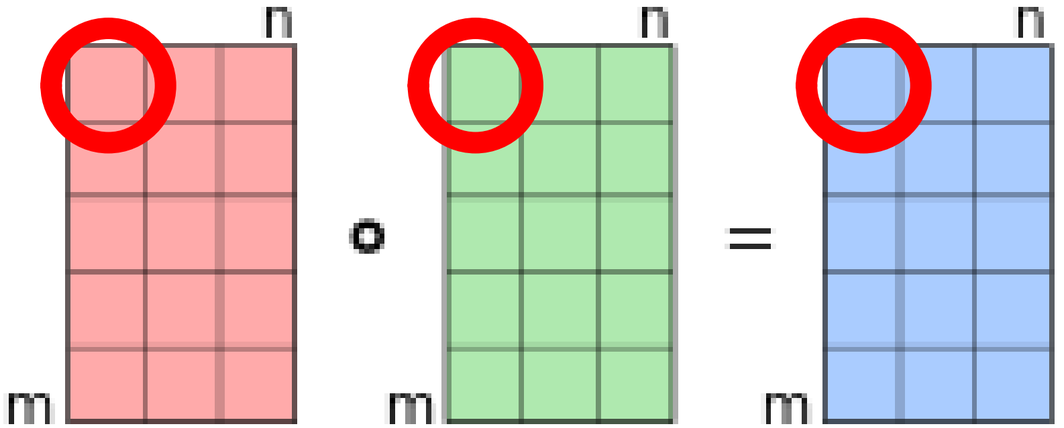
Its element: non-bold capital with indices $A_{1,1}$
colon identifies all elements in its dimension

- i-th row of **A**: $\boldsymbol{A}_{i,:}$
- i-th column of **A**: $\boldsymbol{A}_{:,i}$



Matrix 덧셈

상응하는 element끼리 더한다.



$$\boldsymbol{C} = \boldsymbol{A} + \boldsymbol{B}$$

$$C_{i,j} = A_{i,j} + B_{i,j}$$

Matrix와 scalar의 덧셈/곱셈

scalar를 모든 element에 더한다/곱한다.

$$\boldsymbol{D} = a \cdot \boldsymbol{B} + c$$

$$D_{i,j} = a \cdot B_{i,j} + c$$

$$x\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} ax & bx \\ cx & dx \end{bmatrix}$$

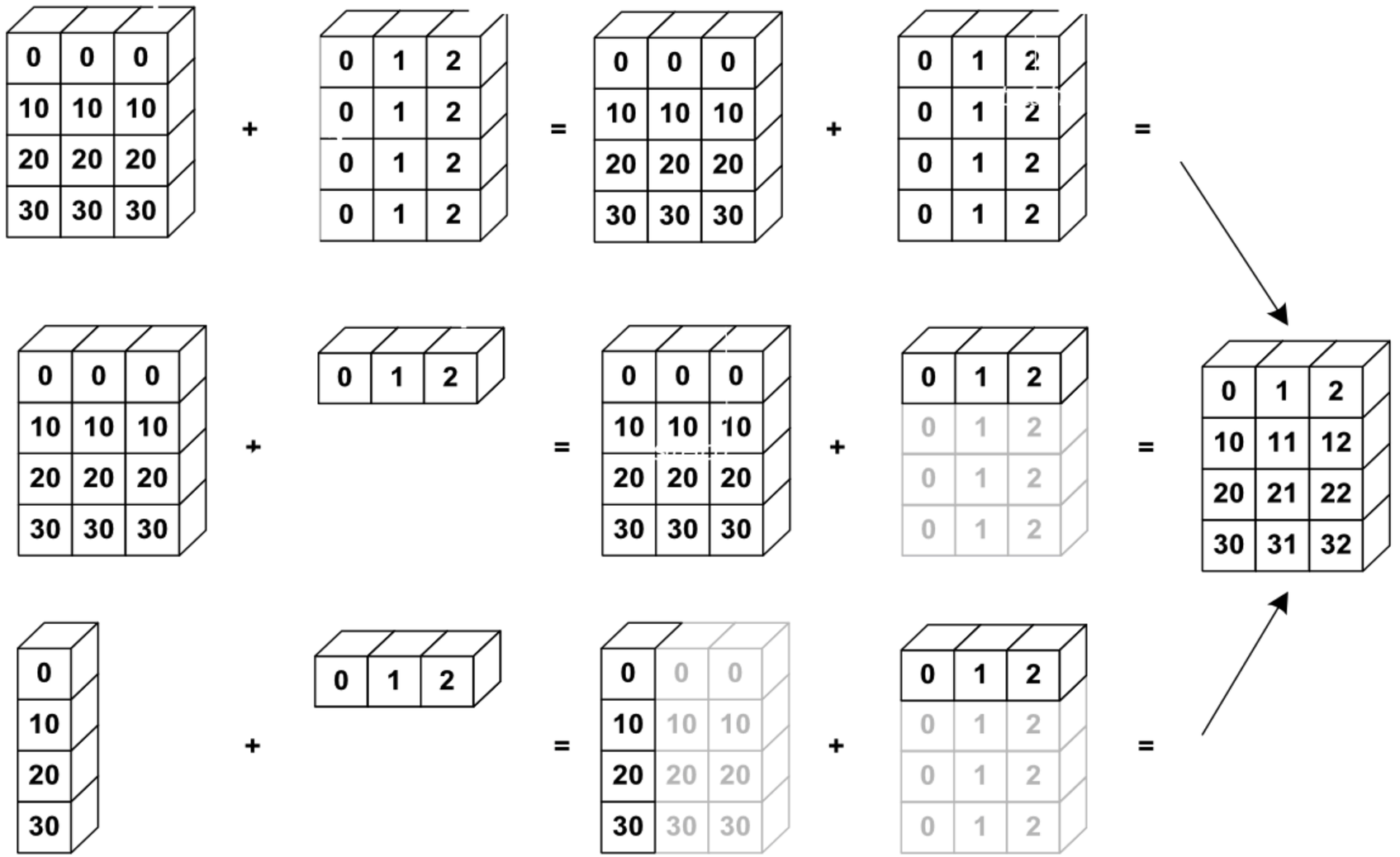
Matrix와 vector의 덧셈

implicit broadcasting

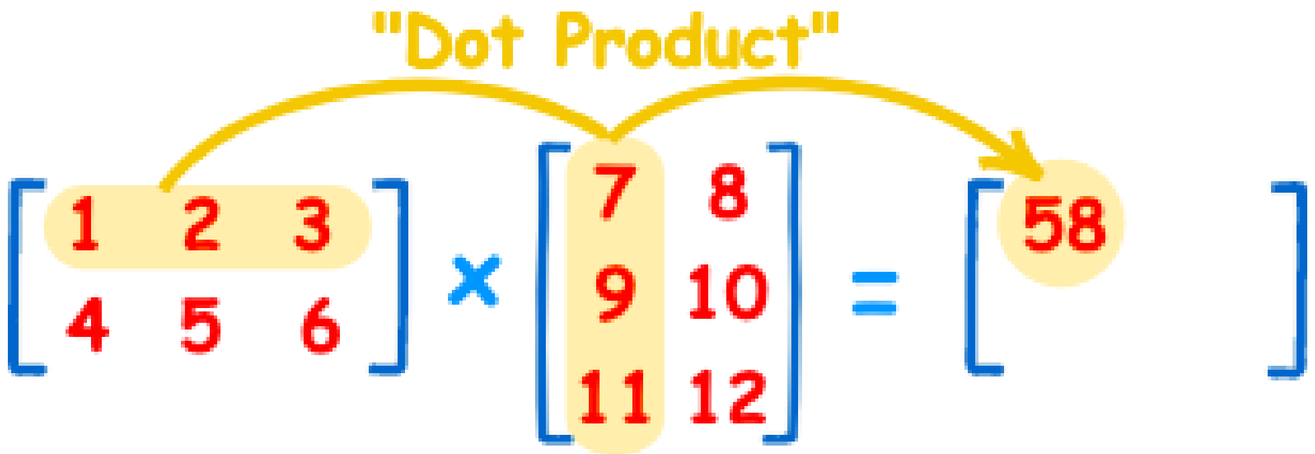
$$\begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 2 & 3 \\ 2 & 3 \\ 2 & 3 \end{bmatrix}$$

Matrix와 vector의 덧셈

implicit broadcasting



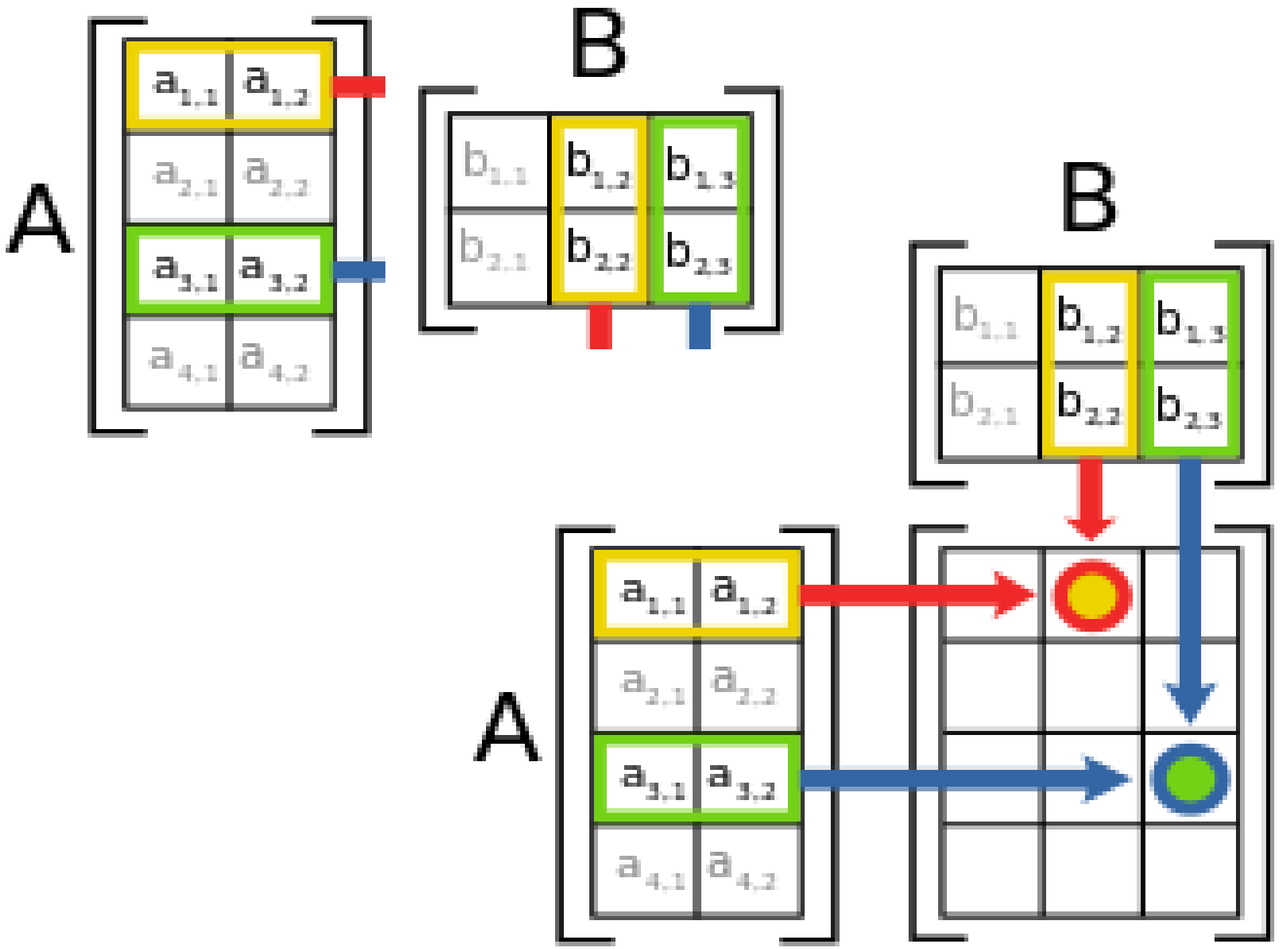
Matrix 곱셈 = vector들의 dot product의 모음



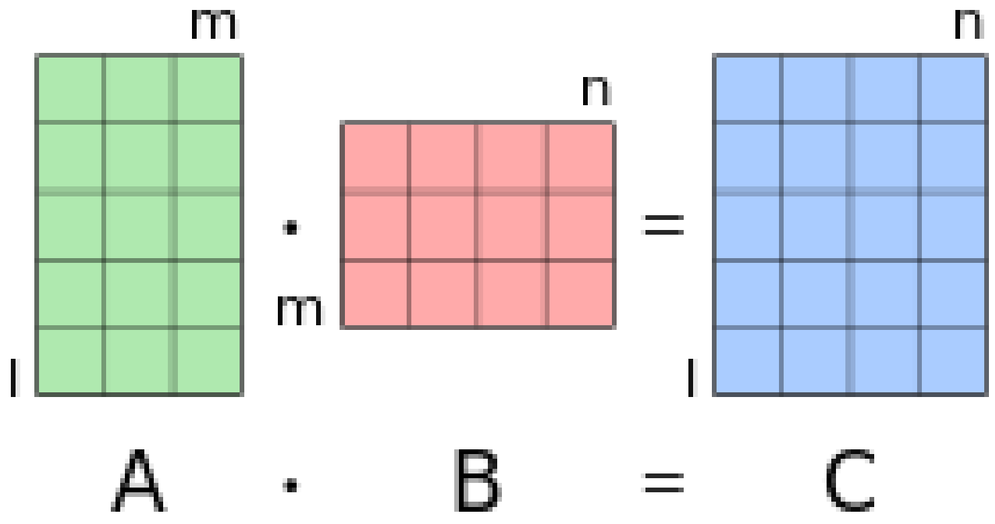
Matrix 곱셈 = vector들의 dot product의 모음

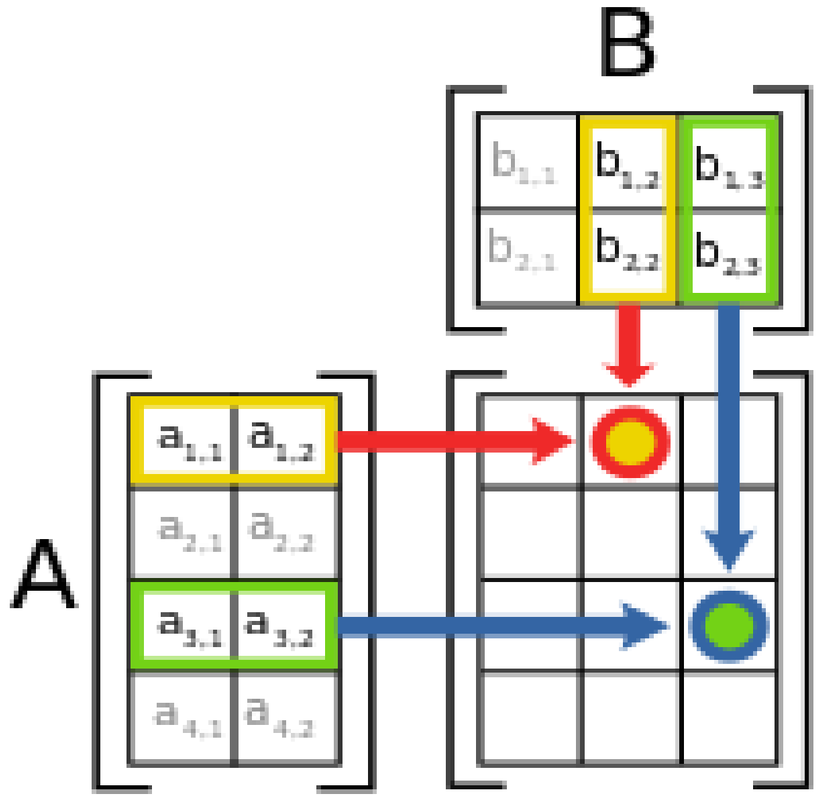
$$\boldsymbol{C} = \boldsymbol{AB}$$

$$C_{i,j} = \sum_{k} A_{i,k} B_{k,j}$$



Matrix 곱셈의 크기관계





Transpose

Transposing a 2x3 matrix to create a 3x2 matrix

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}^T = \begin{bmatrix} 6 & 1 \\ 4 & -9 \\ 24 & 8 \end{bmatrix}$$

matrix 곱셈

A * B 는 element-wise multiplication : dot product와 다르다.

vector dot product들의 합체

```
A = np.arange(6).reshape(3, 2)
B = np.arange(6).reshape(3, 2) - 3
print(A)
print(B)
```

✓ 0.0s

```
[[0 1]
 [2 3]
 [4 5]]
[[-3 -2]
 [-1  0]
 [ 1  2]]
```

```
print(A * B)
print(A.dot(B.transpose()))
```

✓ 0.0s

```
[[ 0 -2]
 [-2  0]
 [ 4 10]]
[[ -2   0   2]
 [-12  -2   8]
 [-22  -4  14]]
```

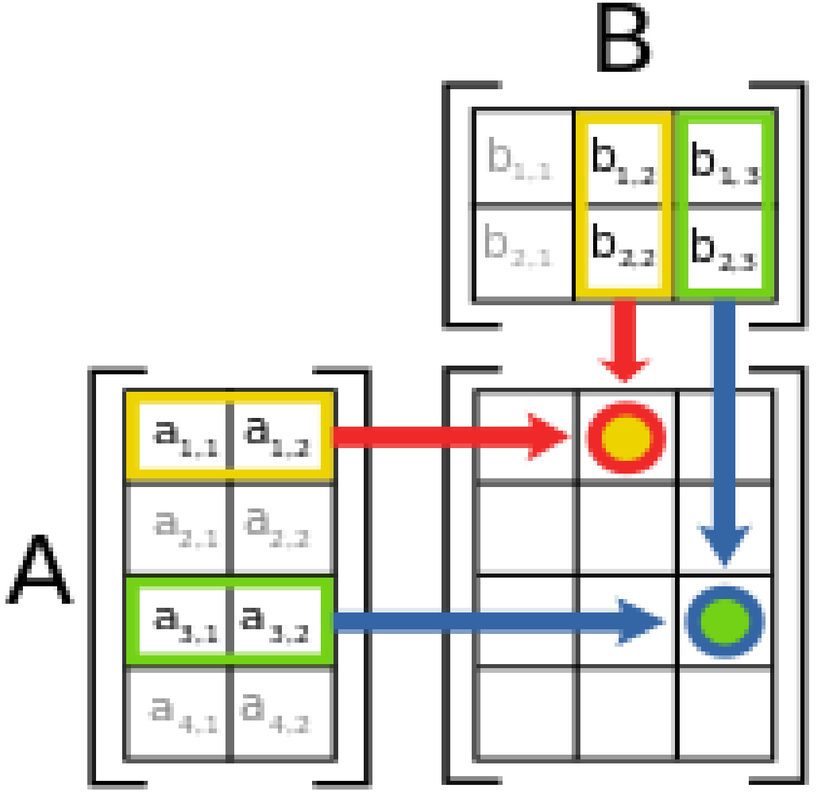
Matrix 곱셈 코드

```
[21] A = np.array(range(6)).reshape(3, 2)
     B = np.array(range(6)).reshape(3, 2) - 3
     print(A)
     print(B)
```

```
[[0 1]
 [2 3]
 [4 5]]
[[-3 -2]
 [-1  0]
 [ 1  2]]
```

```
[23] print(A.dot(B.transpose()))
```

```
[[ -2   0   2]
 [-12  -2   8]
 [-22  -4  14]]
```



Matrix 곱셈의 성질

분배법칙이 성립한다. (matrix multiplication is distributive.)

$$\boldsymbol{A}(\boldsymbol{B}+\boldsymbol{C}) = \boldsymbol{A}\boldsymbol{B} + \boldsymbol{A}\boldsymbol{C}.$$

결합법칙이 성립한다. (matrix multiplication is associative.)

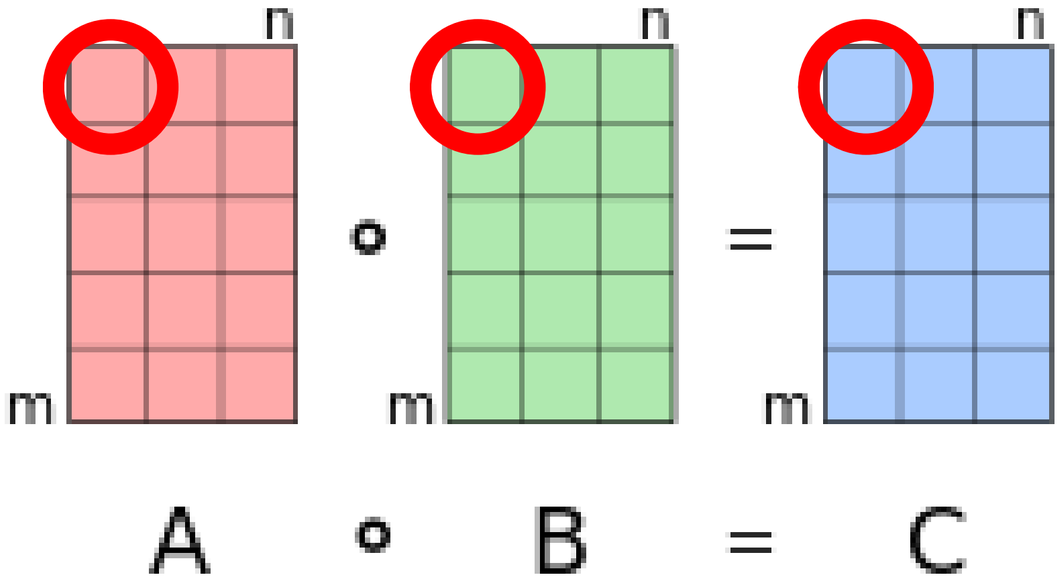
$$\boldsymbol{A}(\boldsymbol{B}\boldsymbol{C}) = (\boldsymbol{A}\boldsymbol{B})\boldsymbol{C}.$$

교환법칙은 성립하지 않는다

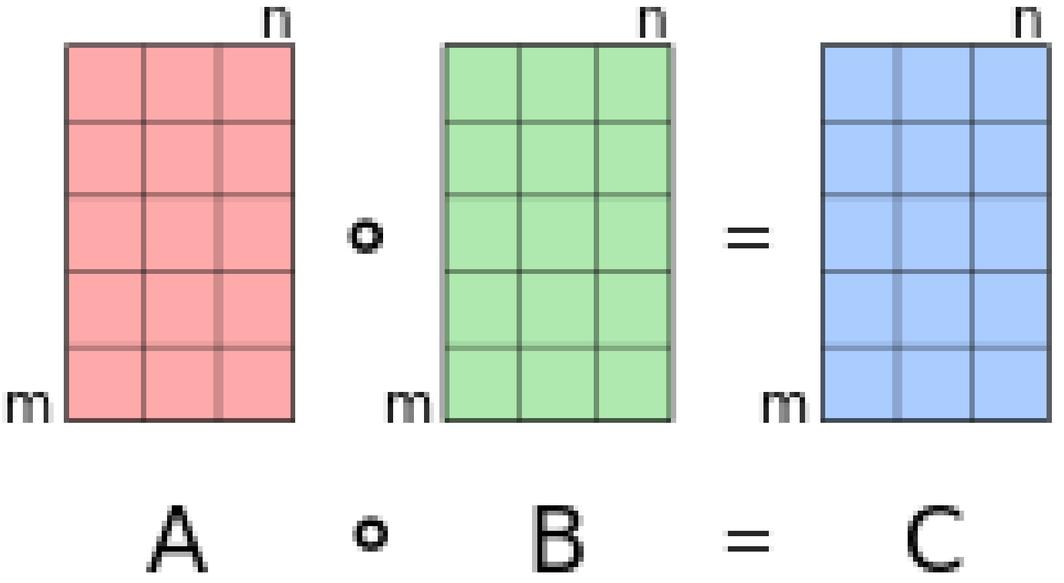
$$\boldsymbol{A}\boldsymbol{B} \stackrel{?}{=} \boldsymbol{B}\boldsymbol{A}$$

element-wise matrix 곱셈

(Hadamard product)



element-wise matrix 곱셈




```
[21] A = np.array(range(6)).reshape(3, 2)
     B = np.array(range(6)).reshape(3, 2) - 3
     print(A)
     print(B)
```

```
[[0 1]
 [2 3]
 [4 5]]
[[-3 -2]
 [-1  0]
 [ 1  2]]
```

```
[22] print(A * B)
```

```
[[ 0 -2]
 [-2  0]
 [ 4 10]]
```

Large matrices for representing data

1000명의 키, 몸무게, 나이 = a 3 x 1000 matrix

T

```
data = [[70, 170, 40],
        [65, 120, 26],
        [77, 250, 19],
        # ....
       ]
```

Matrix as a linear function

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 3 & -2 \\ 5 & -3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Matrices for a adjacency

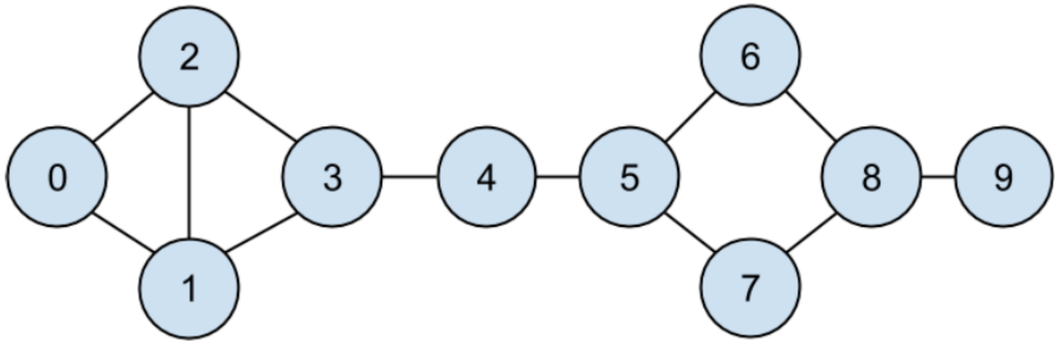


Figure 1-1. The DataSciencester network

```
friendships = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 3), (3, 4),
               (4, 5), (5, 6), (5, 7), (6, 8), (7, 8), (8, 9)]

friend_matrix = [[0, 1, 1, 0, 0, 0, 0, 0, 0, 0],  # user 0
                 [1, 0, 1, 1, 0, 0, 0, 0, 0, 0],  # user 1
                 [1, 1, 0, 1, 0, 0, 0, 0, 0, 0],  # user 2
                 [0, 1, 1, 0, 1, 0, 0, 0, 0, 0],  # user 3
                 [0, 0, 0, 1, 0, 1, 0, 0, 0, 0],  # user 4
                 [0, 0, 0, 0, 1, 0, 1, 1, 0, 0],  # user 5
                 [0, 0, 0, 0, 0, 1, 0, 0, 1, 0],  # user 6
                 [0, 0, 0, 0, 0, 1, 0, 0, 1, 0],  # user 7
                 [0, 0, 0, 0, 0, 0, 1, 1, 0, 1],  # user 8
                 [0, 0, 0, 0, 0, 0, 0, 0, 1, 0]]  # user 9
```

Vector functions

벡터함수 Vector functions

Vector를 출력하는 함수

$$f_1(x) = x^2$$

$$f_2(x) = 5x$$

$$f(x) = \begin{bmatrix} x^2 \\ 5x \end{bmatrix}$$

벡터함수 Vector functions

Vector를 출력하는 함수

$$f_1(x, y) = x^2 + y^2$$

$$f_2(x, y) = 5x + 2y$$

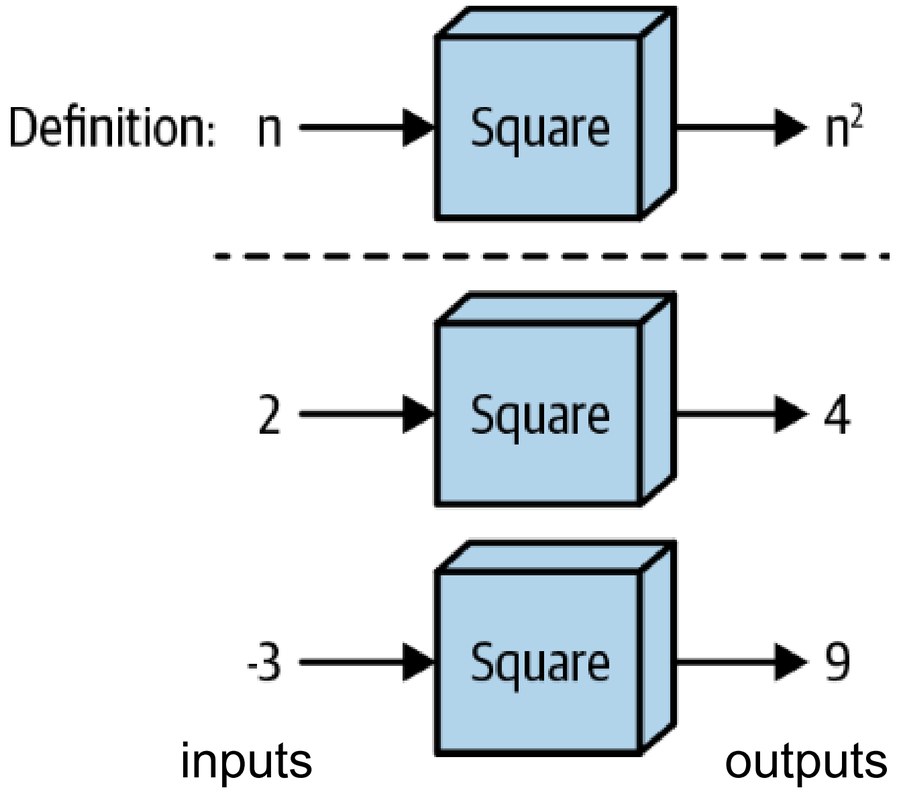
$$f(x, y) = \begin{bmatrix} x^2 + y^2 \\ 5x + 2y \end{bmatrix}$$

Linear functions

함수 Functions

Functions = mathematical entities that assign unique outputs to given inputs.

$$f_1(x) = x^2$$



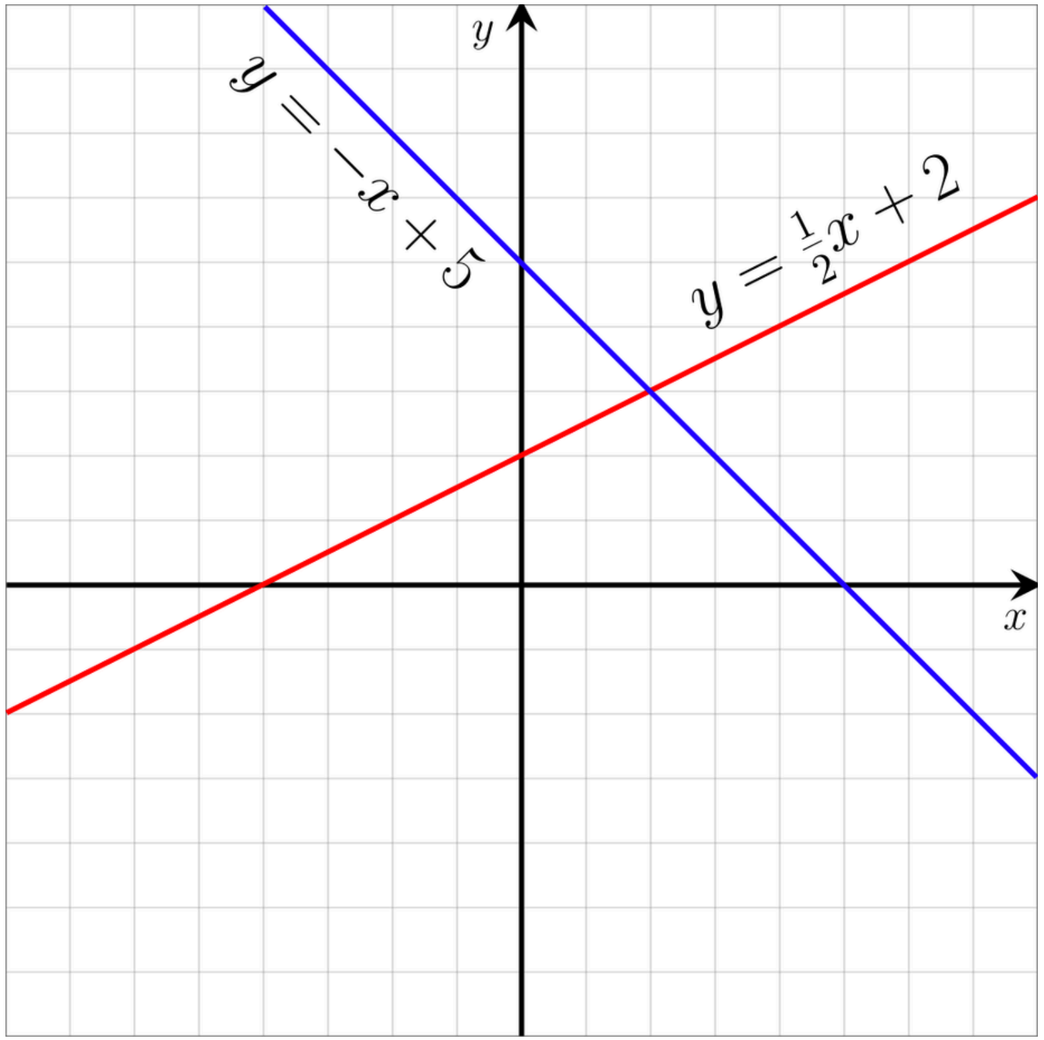
선형 함수 Linear functions

그래프가 직선인 함수

$$f(x) = ax + b$$

$$f_1(x) = -x + 5$$

$$f_2(x) = \frac{1}{2}x + 2$$

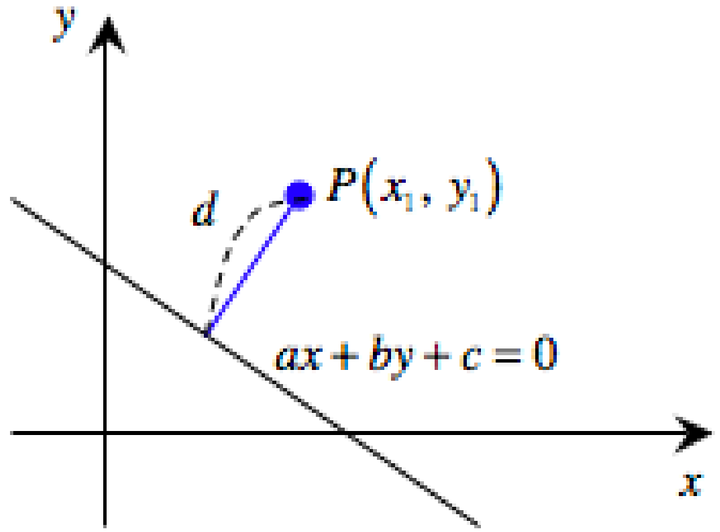


입력이 여러개인 함수

d(x1, y1) = …

점 $P(x_1, y_1)$에서
직선 $ax+by+c=0$까지의 거리 d는

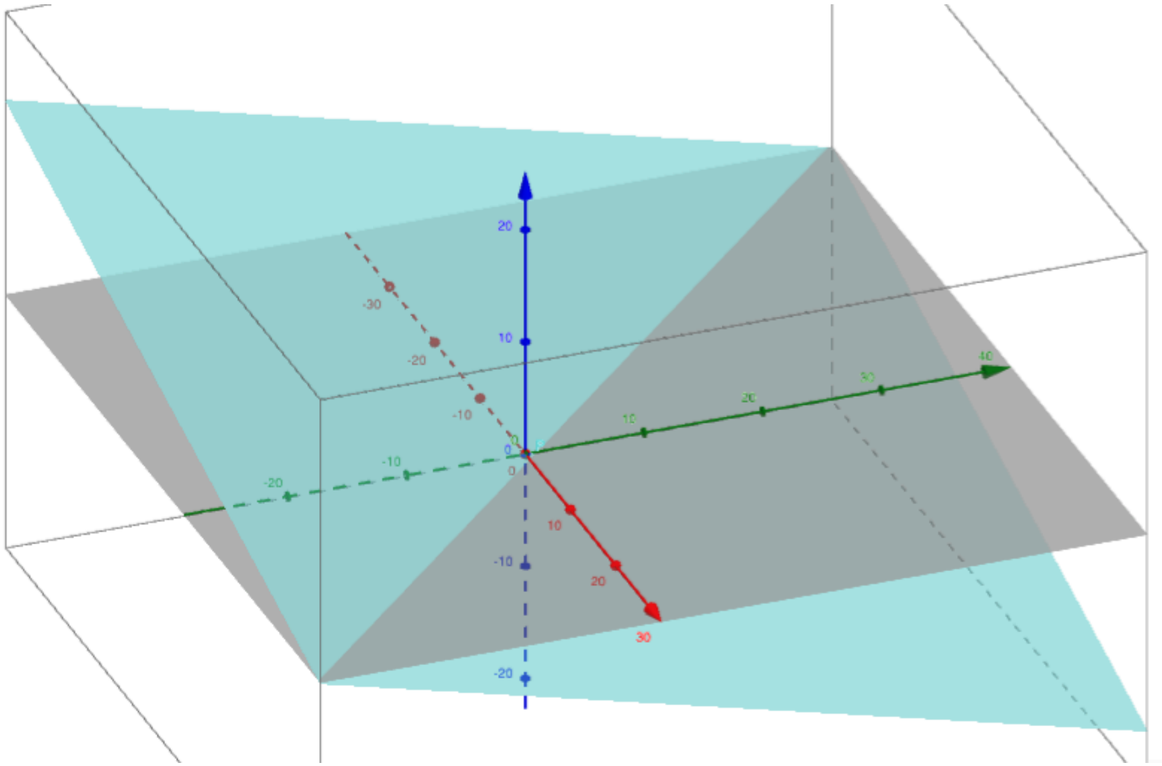
$$d=\frac{|ax_1+by_1+c|}{\sqrt{a^2+b^2}}$$



입력이 여러개인 선형 함수

$$f(x,y) = ax + by + c$$

$$f(x,y) = \frac{1}{4}x + \frac{1}{4}y - 1$$



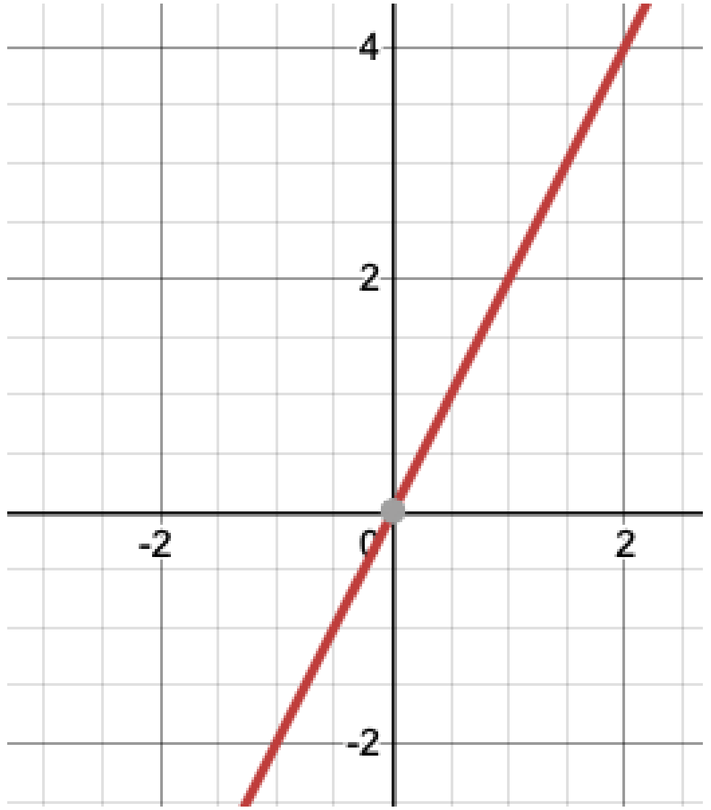
https://www.geogebra.org/m/ceqknsyf

Linear transformation

Linear transformation in 1D

a=2
y = 2x
x’ = 2x
1 → 2
2 → 4
-0.5 → -1

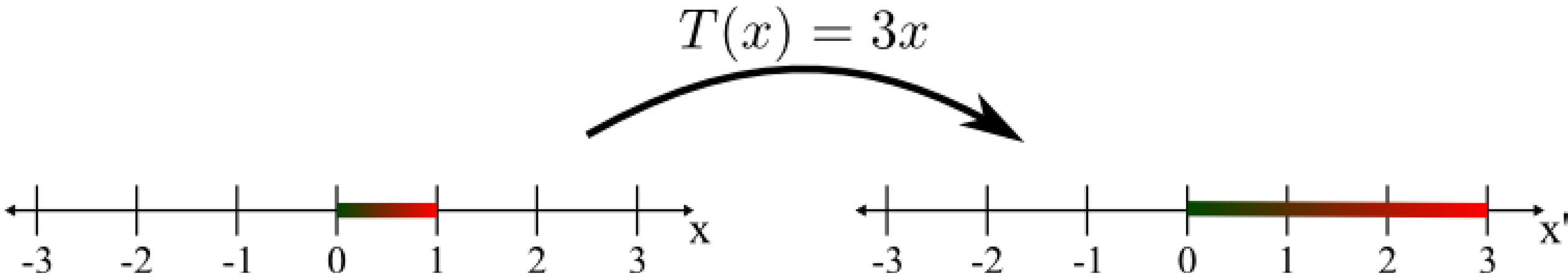
$$T(x) = ax$$



Linear transformation in 1D

a = 3
T maps the interval [0, 1] to the interval [0, 3]
a를 1x1 matrix로 생각해보세요.

$$T(x) = ax$$

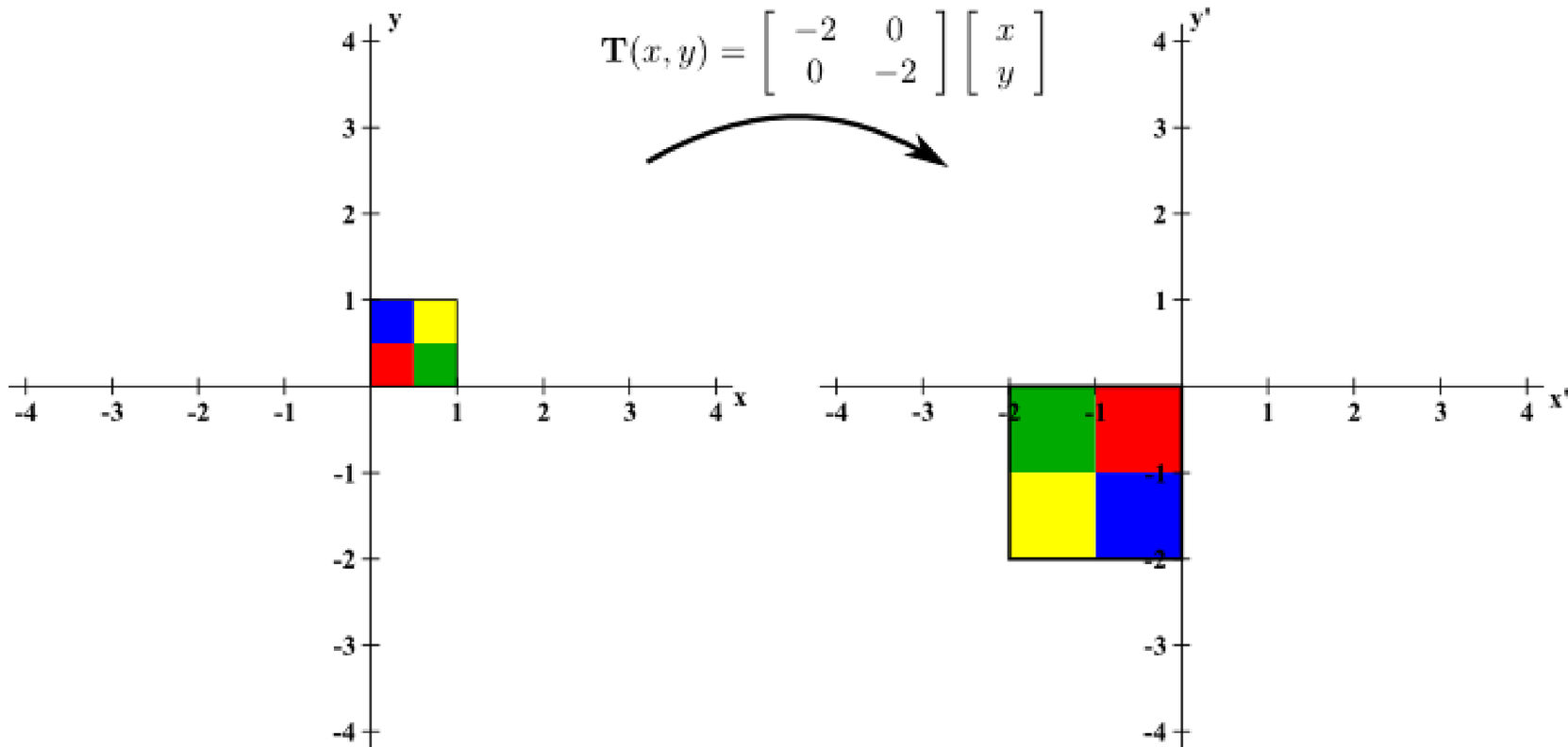


Linear transformation in 2D

a function $\mathbf{T} : \mathbf{R}^2 \rightarrow \mathbf{R}^2$

$$\mathbf{T}(x, y) = (ax + by, cx + dy) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\mathbf{T}(\mathbf{x}) = A\mathbf{x}$$

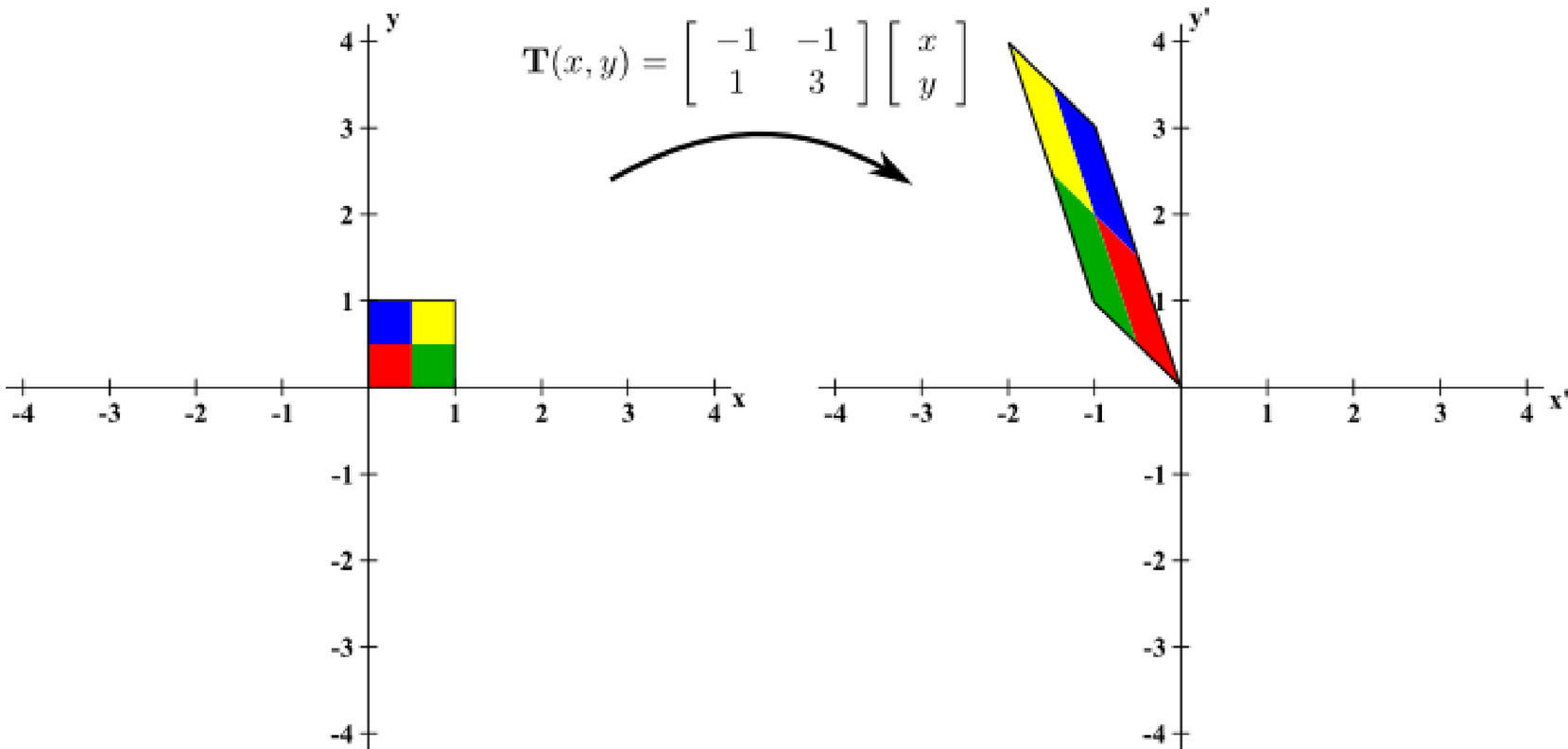


Linear transformation in 2D

a function $\mathbf{T} : \mathbf{R}^2 \rightarrow \mathbf{R}^2$

$$\mathbf{T}(x, y) = (ax + by, cx + dy) = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\mathbf{T}(\mathbf{x}) = A\mathbf{x}$$



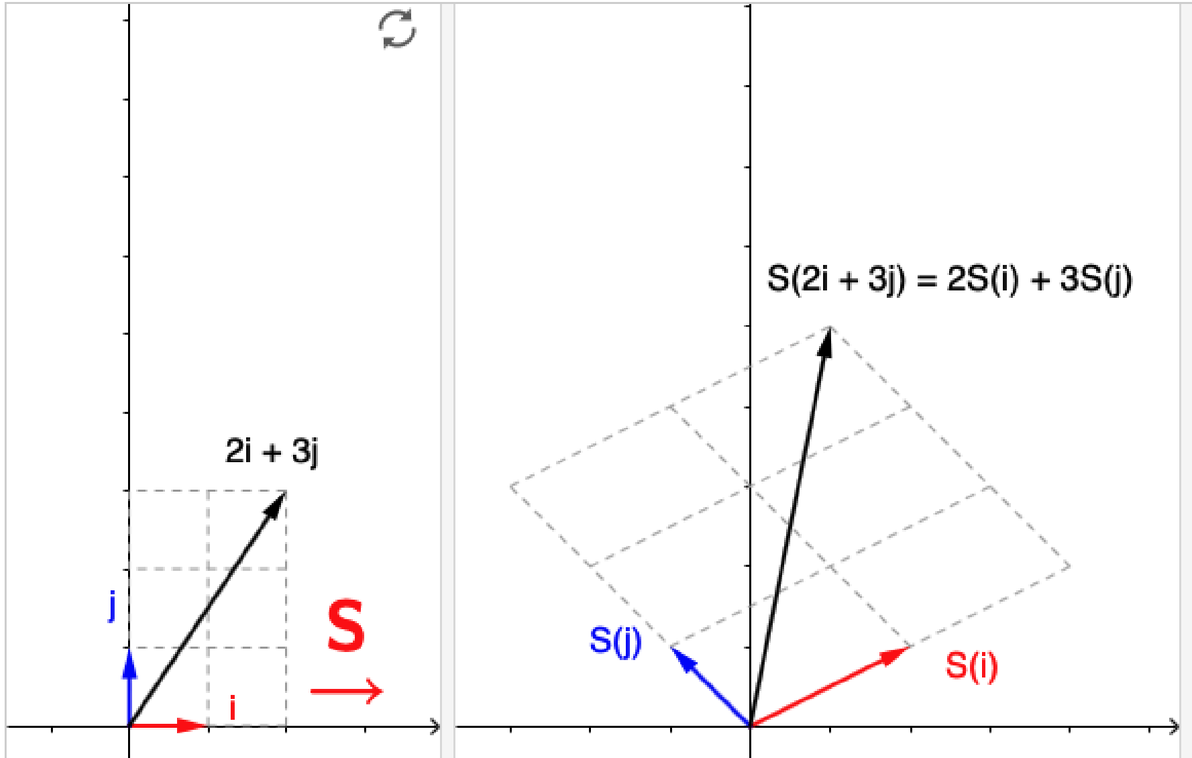
Linear transformation의 정의

Linear transformation is a mapping $f : V \to W$ between two vector spaces that

- preserves the operations of vector addition
- and preserves the operations of scalar multiplication.

$$f(\mathbf{u}+\mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$$
$$f(c\mathbf{u}) = cf(\mathbf{u})$$

play with this → https://www.geogebra.org/m/kGJZH7AU



선형 벡터 함수 Linear vector functions

Vector를 출력하는 선형 함수

$$f_1(x, y) = 2x + 3y$$

$$f_2(x, y) = 5x + 2y$$

$$f(x, y) = \begin{bmatrix} 2x + 3y \\ 5x + 2y \end{bmatrix}$$

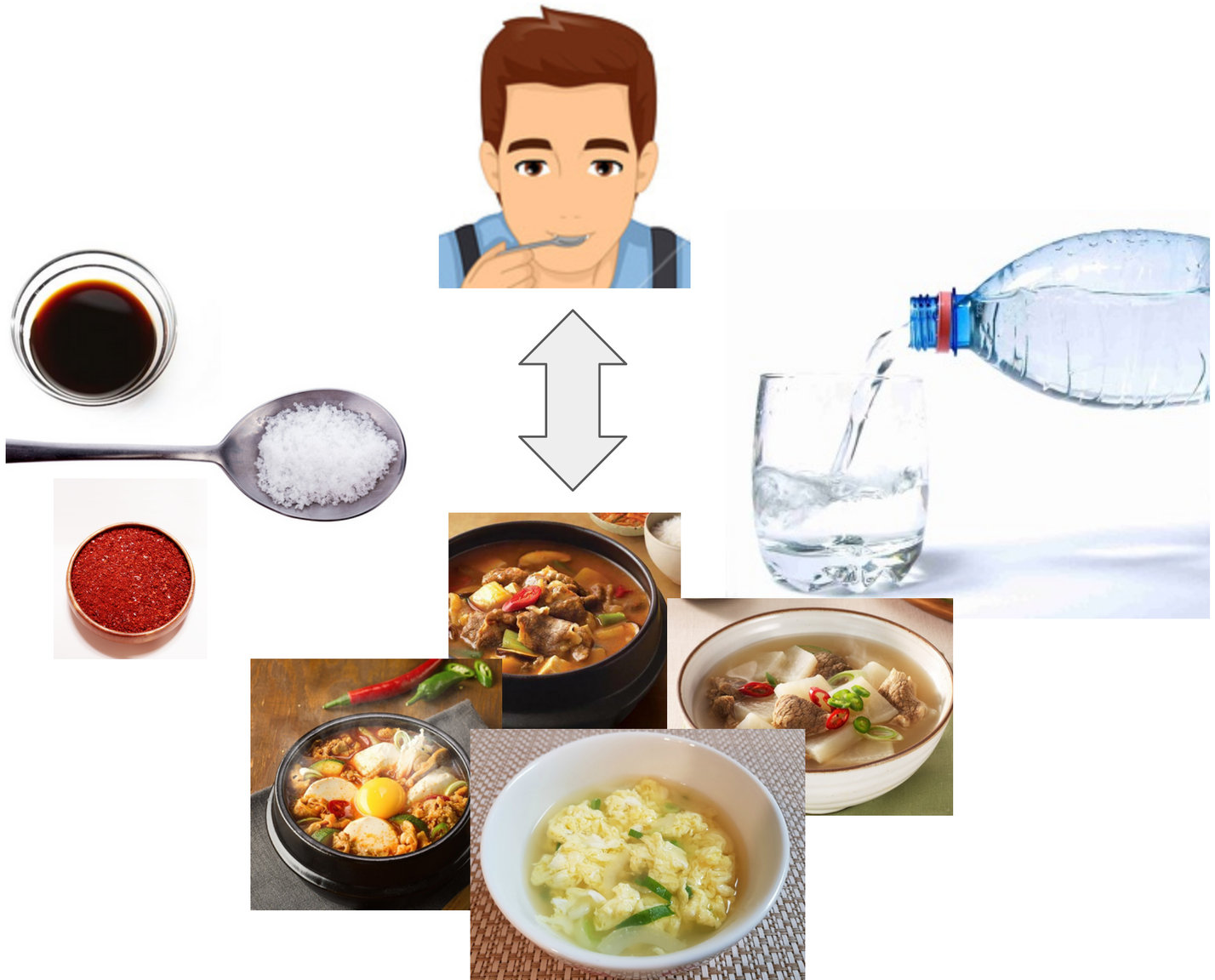
Matrix multiplication으로 표현된 Linear vector functions

=Linear transformation

$$f(x,y) = \begin{bmatrix} 2x+3y \\ 5x+2y \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

요리

다양한 양념으로 다양한 요리를 만든다.



장설파마깨후참

간장2 + 111111 = 불고기소스
고추장2 + 111111 = 제육볶음소스



요리

소금 / 간장 / 고춧가루 각각 몇숟가락?









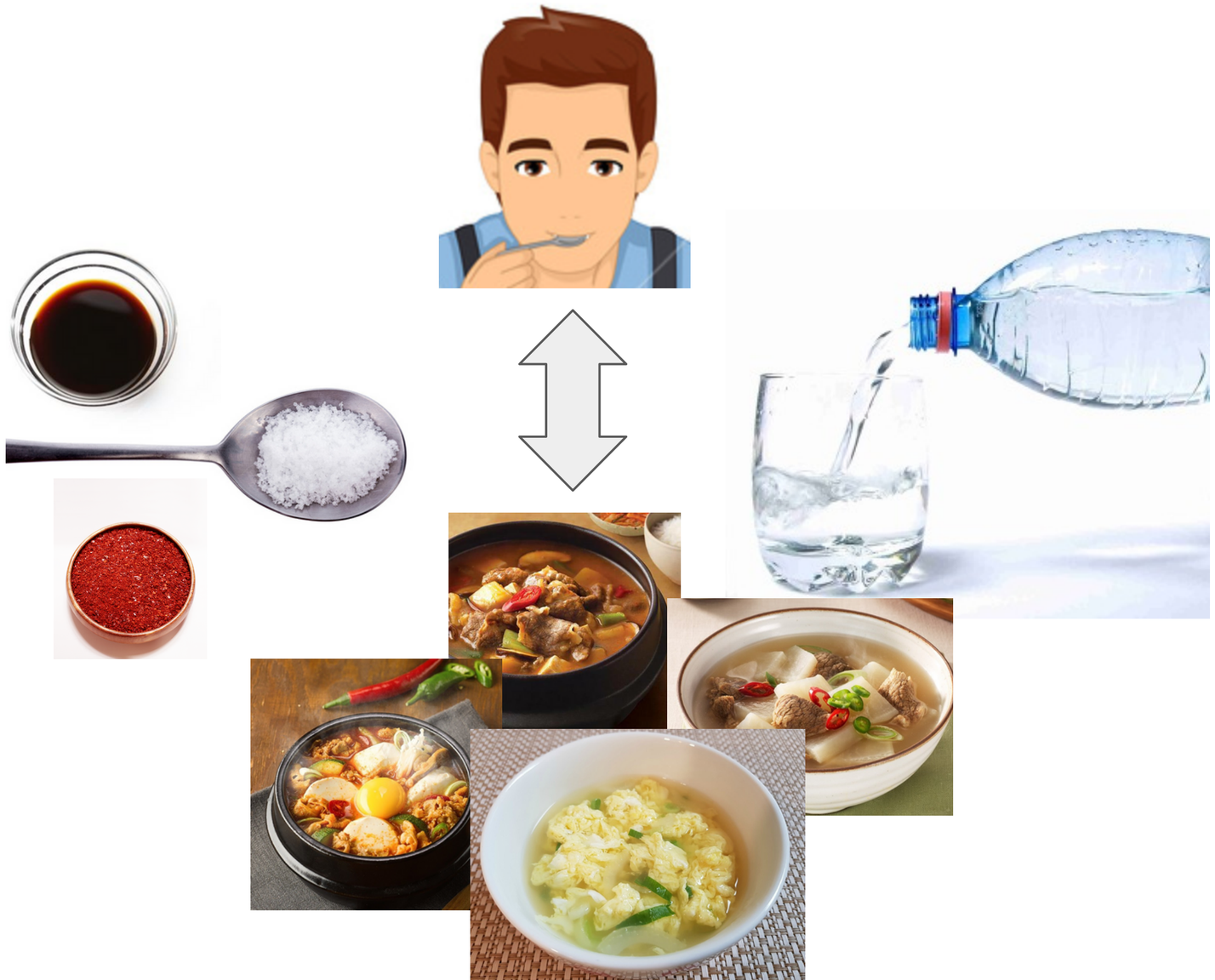
장설파마깨후참

간장2 + 111111 = 불고기소스

$$[\text{불고기소스}] = [\text{간장} \quad \text{고추장} \quad \text{설} \quad \text{파} \quad \text{마} \quad \text{깨} \quad \text{후} \quad \text{참}] \begin{bmatrix} 2 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

요리

된장찌개만 할줄알면 된다?



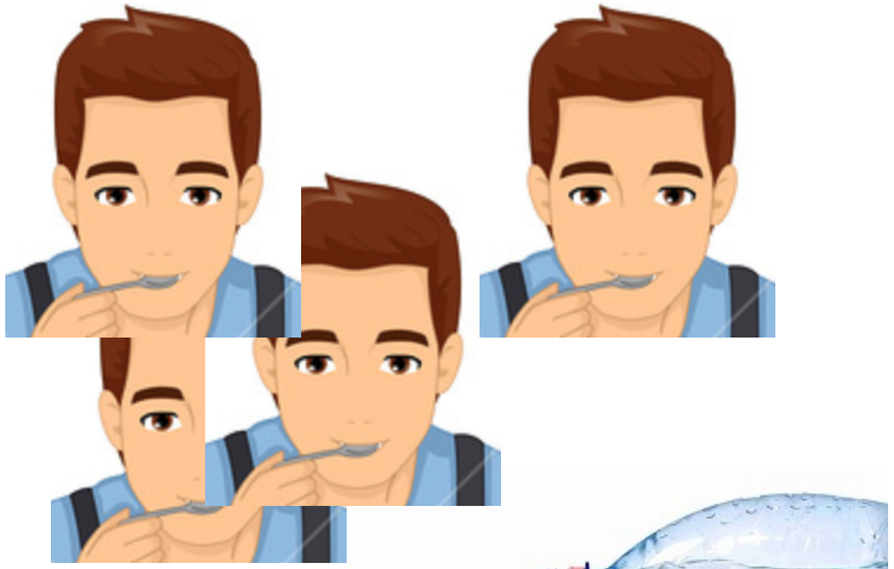
장설파마깨후참

간장2 + 111111 = 불고기소스
고추장2 + 111111 = 제육볶음소스

$$f(장설파마깨후참) = [불고기소스 \quad 제육소스] = [간장 \quad 고추장 \quad 설 \quad 파 \quad 마 \quad 깨 \quad 후 \quad 참] \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

요리

우리집에서만 맛있으면 된다?
집마다 재료/숟가락/먹는사람 다르다









요리

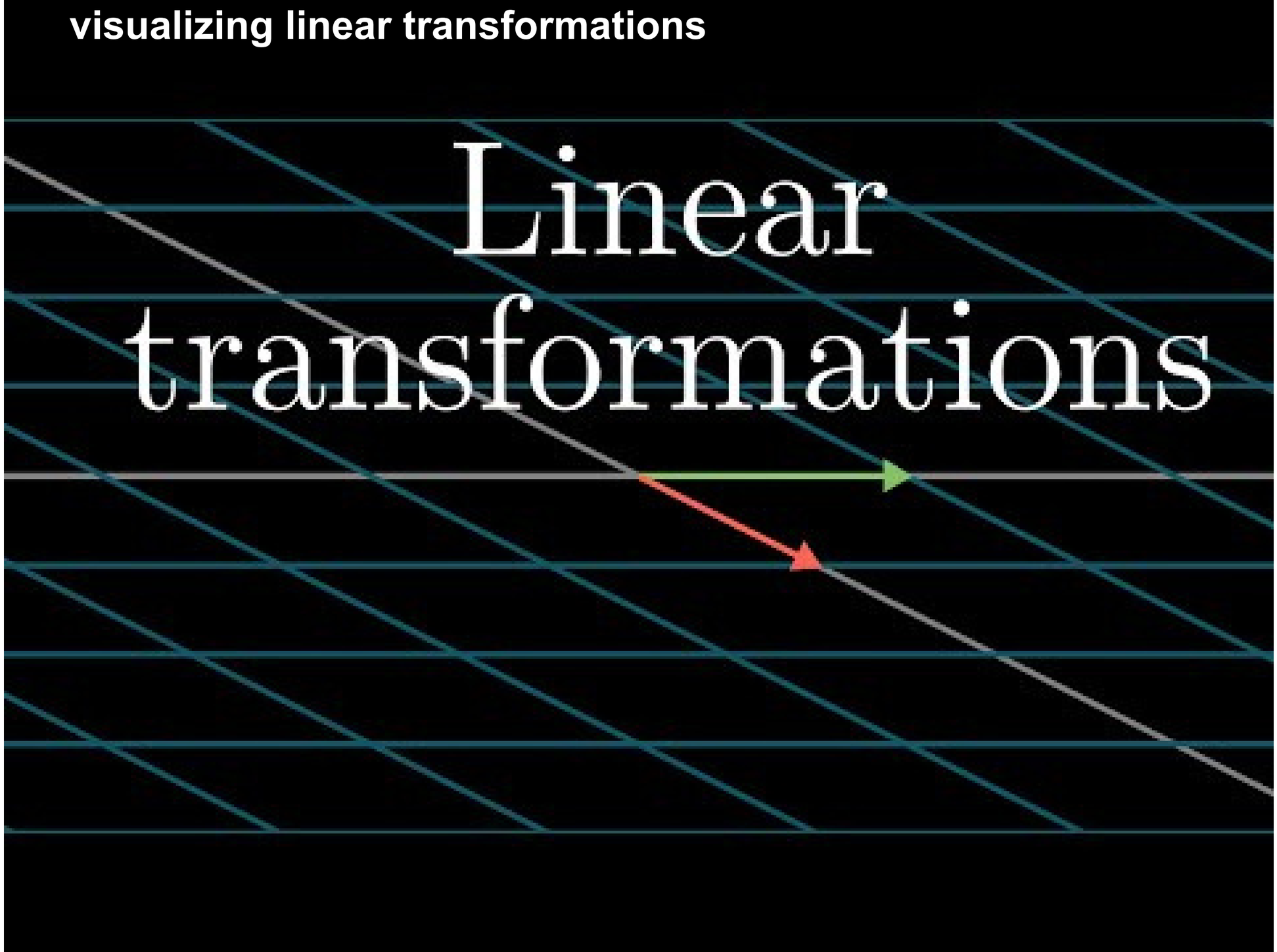
고추장이 집마다 다르다.

- 우리집 고추장은 옆집 고추장보다 2배 맵다
- 간장2 + 111111 = 불고기소스
- 고추장2 + 111111 = 제육볶음소스
- 우리집고추장2 + 111111 = 우리집제육볶음소스

$$\begin{bmatrix} 불고기소스 & 제육소스 \\ 불고기소스 & 우리집제육소스 \end{bmatrix} = \begin{bmatrix} 간장 & 고추장 & 설 & 파 & 마 & 깨 & 후 & 참 \\ 간장 & 우리집고추장 & 설 & 파 & 마 & 깨 & 후 & 참 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

visualizing linear transformations

visualizing linear transformations

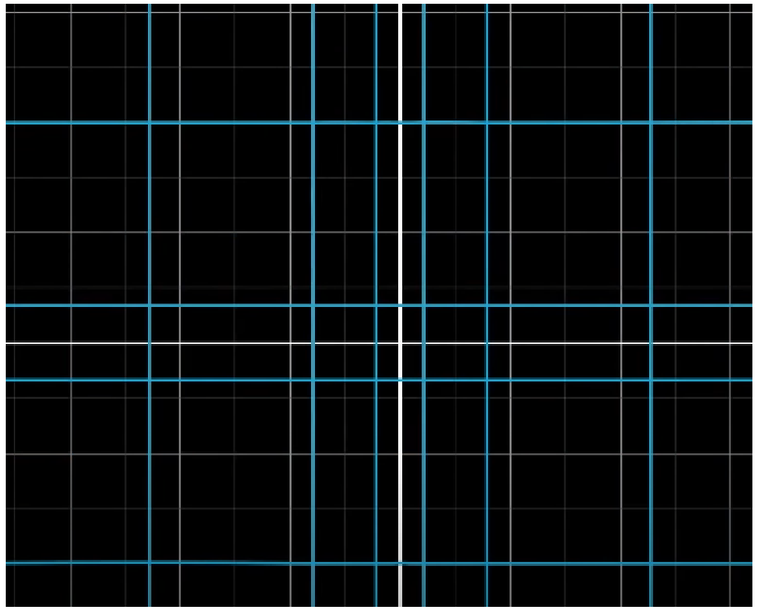


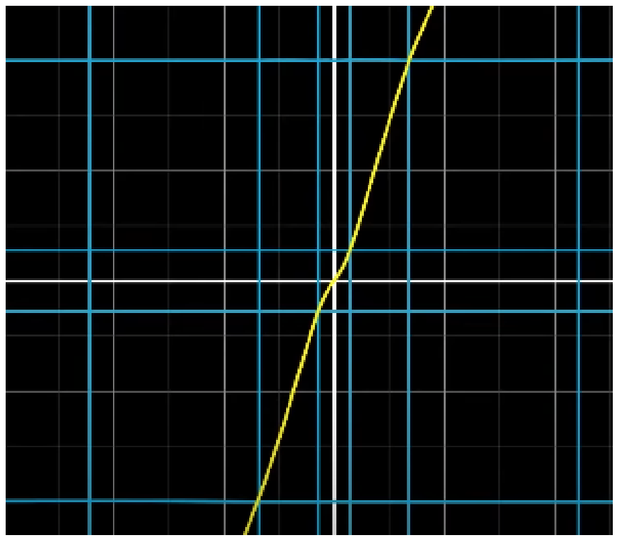
Linear transformation in 2D

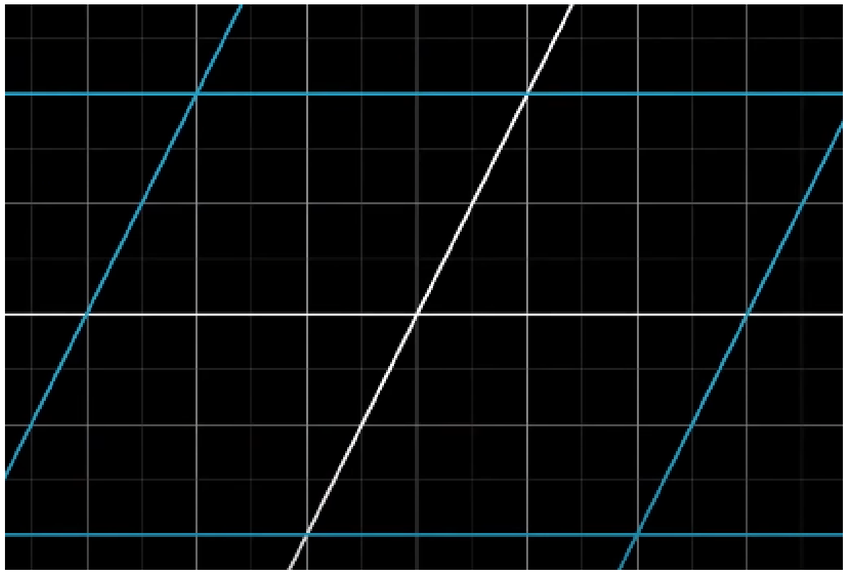
Linear transformation은 변형을 통해서

- 직선을 직선으로 유지한다.
- 원점은 원점에 유지된다.
- grid lines가 evenly spaced로 유지된다.

$$f(\mathbf{u}+\mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$$
$$f(c\mathbf{u}) = cf(\mathbf{u})$$

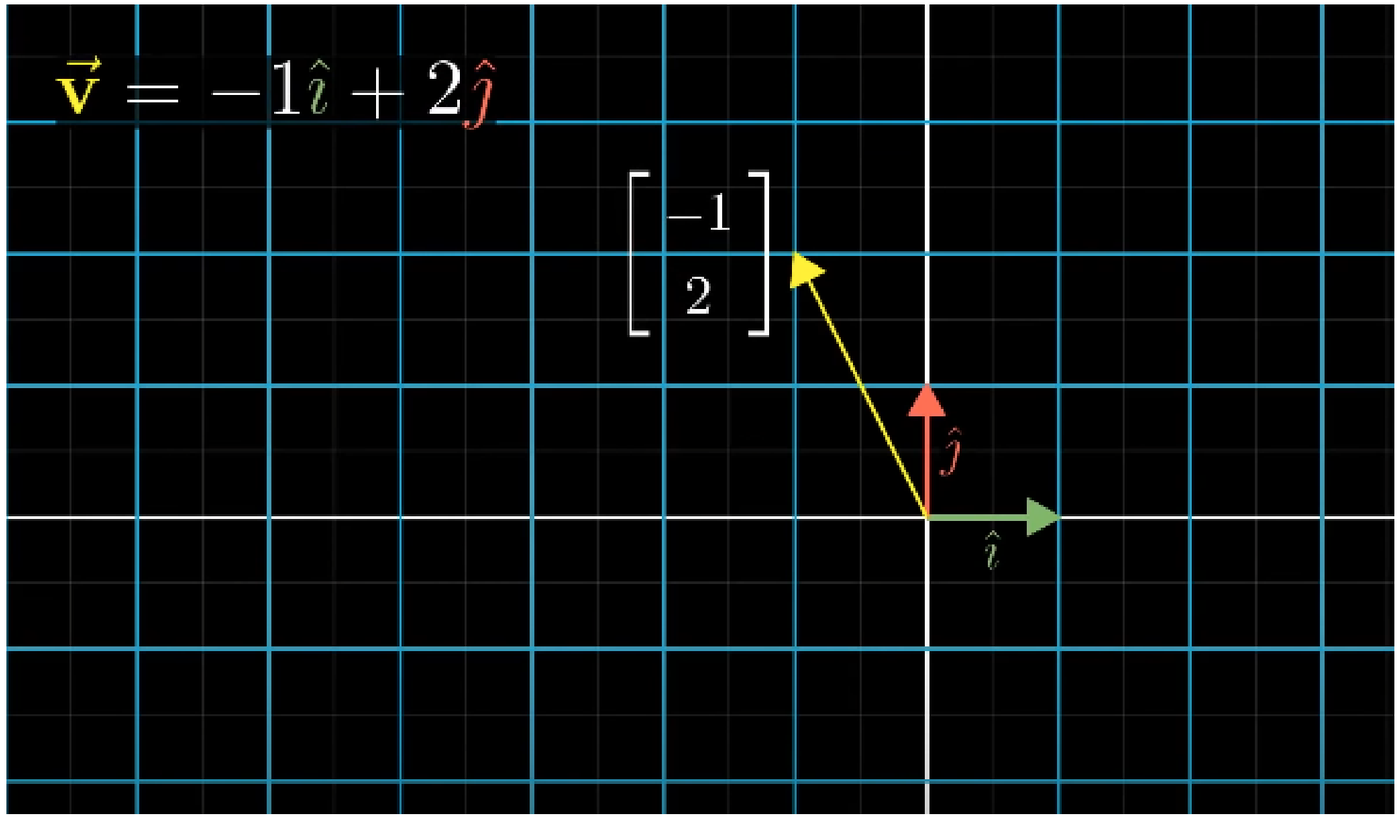






Linear transformation in 2D

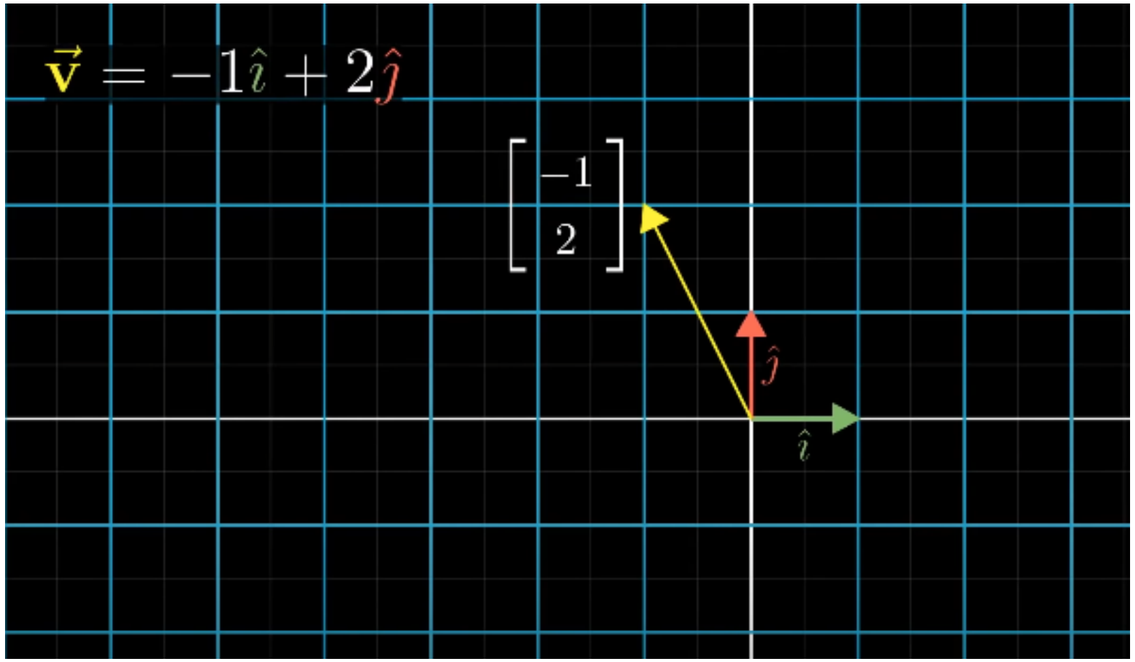
Linear transformation은 coordinate system을 통째로 변형한다.

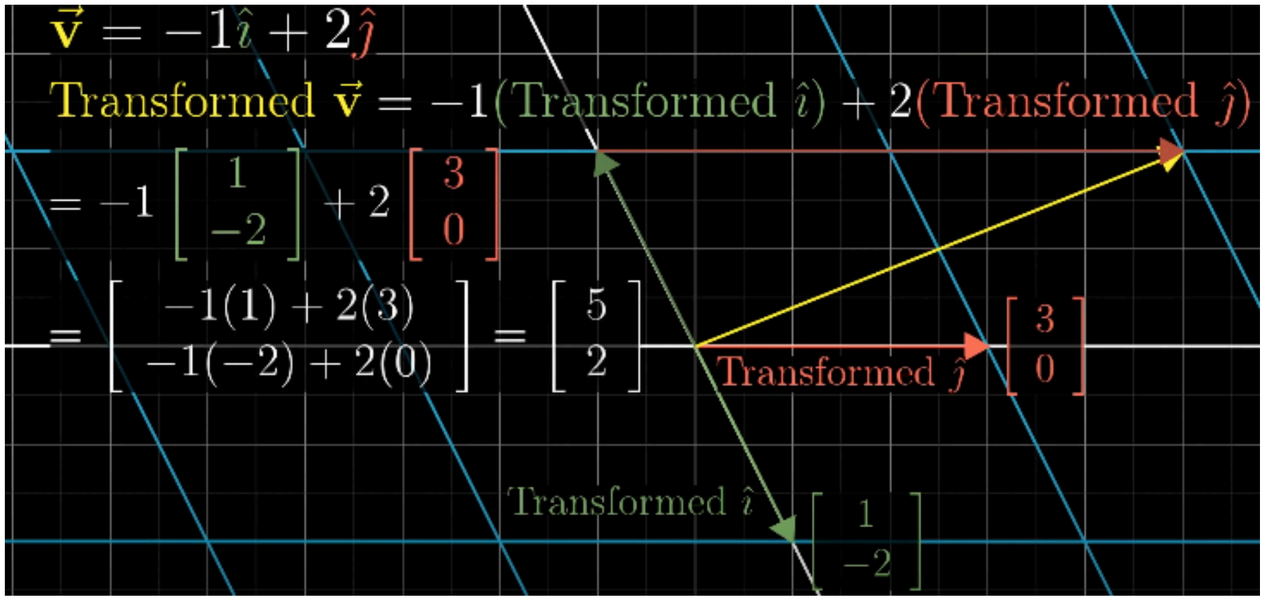


← 원래의 coordinate system

Linear transformation in 2D

Linear transformation은 coordinate system을 통째로 변형한다.





$$f(\mathbf{u} + \mathbf{v}) = f(\mathbf{u}) + f(\mathbf{v})$$
$$f(c\mathbf{u}) = cf(\mathbf{u})$$

Linear transformation in 2D

basis vector들이 어디로 가는지 알면
임의의 벡터 (x, y)가 어디로 가는지 계산할 수 있다.

$$\hat{\imath} \rightarrow \begin{bmatrix} 1 \\ -2 \end{bmatrix} \qquad \hat{\jmath} \rightarrow \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

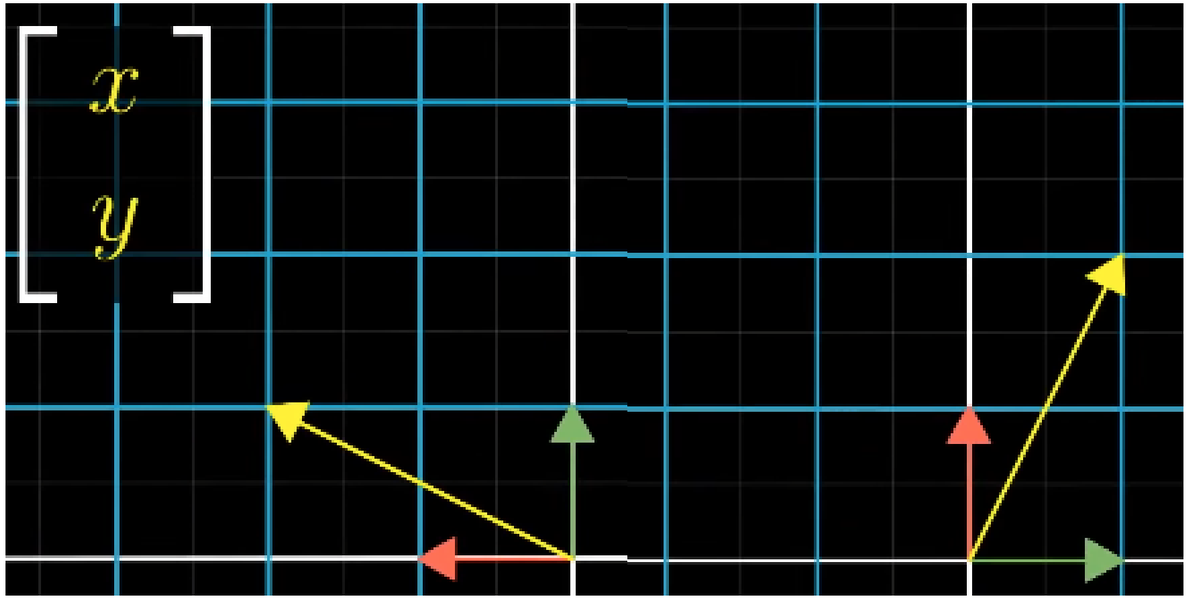
$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow x \begin{bmatrix} 1 \\ -2 \end{bmatrix} + y \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1x + 3y \\ -2x + 0y \end{bmatrix}$$

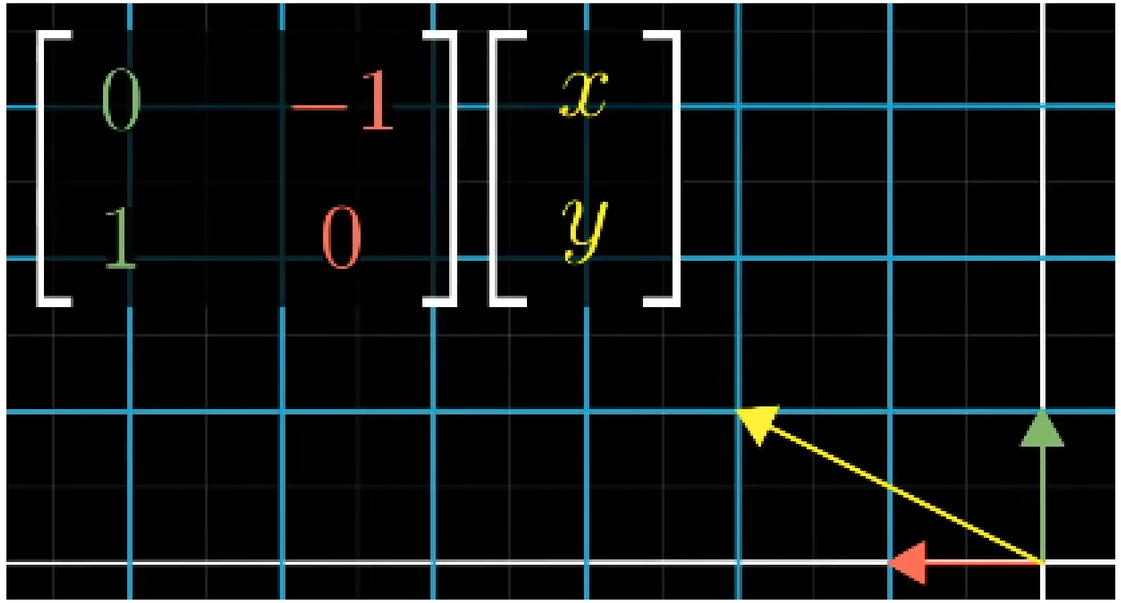
Linear transformation in 2D

Matrix multiplication으로 linear transformation 표현하기

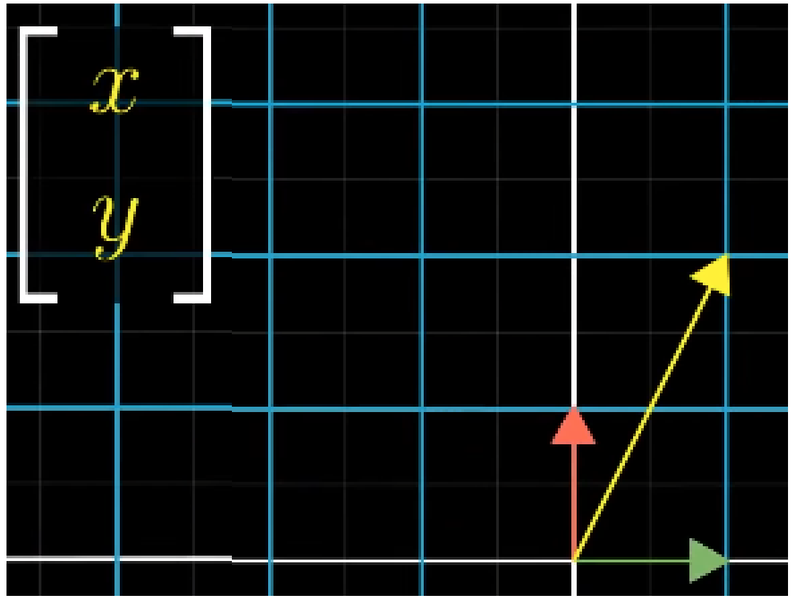
$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x \begin{bmatrix} a \\ c \end{bmatrix} + y \begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} a\,x + b\,y \\ c\,x + d\,y \end{bmatrix}$$

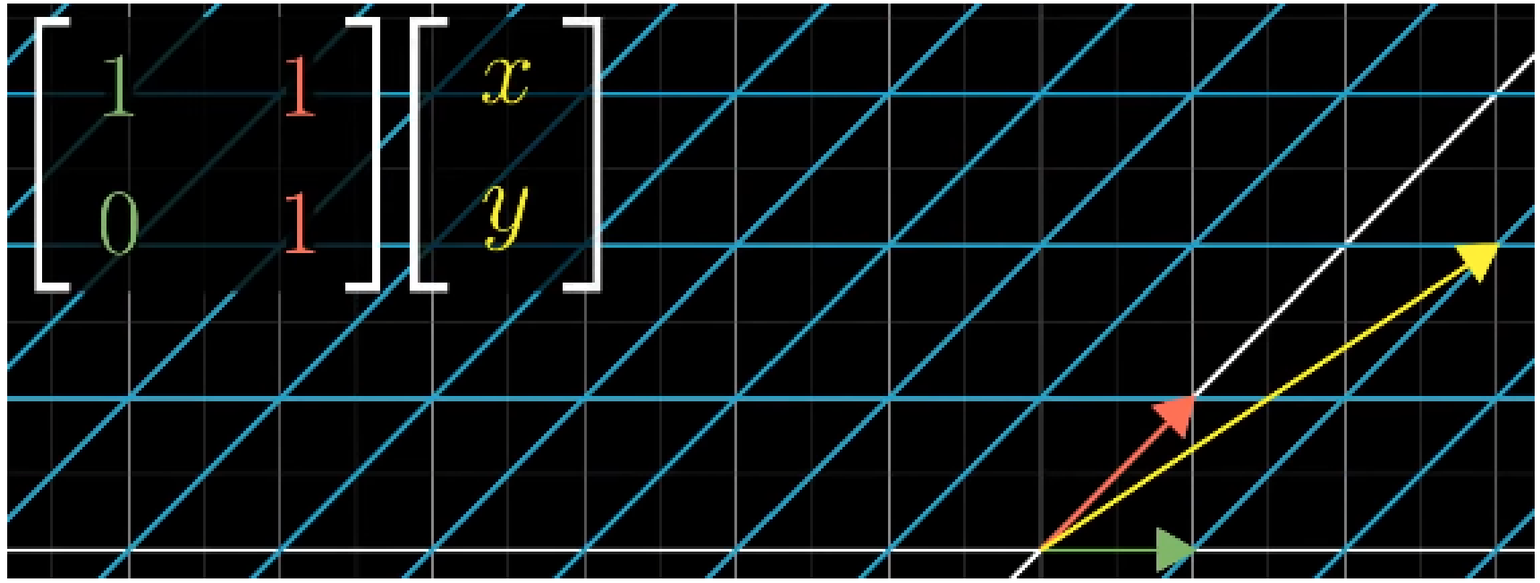
Rotation (기억해둘 linear transformation)





Shear(기억해둘 linear transformation)





실습

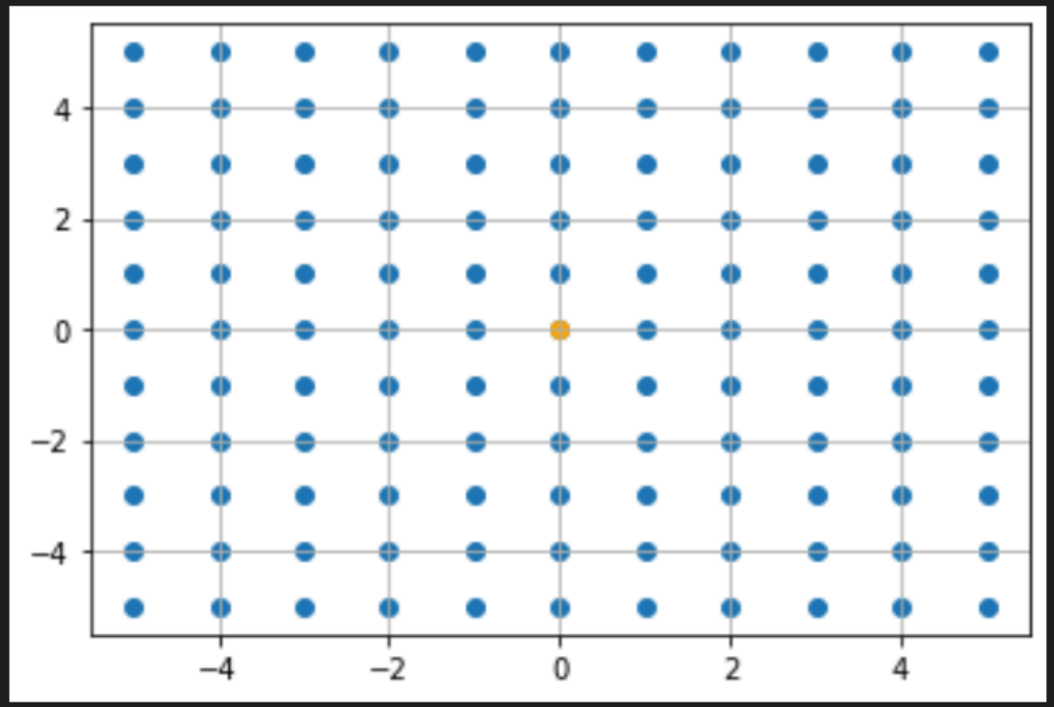
```
import numpy as np
import matplotlib.pyplot as plt

x,y = np.meshgrid(np.arange(-5,6), np.arange(-5,6))
print(x) ; print(y)
```

[8]

```
[[-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]
 [-5 -4 -3 -2 -1  0  1  2  3  4  5]]
[[-5 -5 -5 -5 -5 -5 -5 -5 -5 -5 -5]
 [-4 -4 -4 -4 -4 -4 -4 -4 -4 -4 -4]
 [-3 -3 -3 -3 -3 -3 -3 -3 -3 -3 -3]
 [-2 -2 -2 -2 -2 -2 -2 -2 -2 -2 -2]
 [-1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1]
 [ 0  0  0  0  0  0  0  0  0  0  0]
 [ 1  1  1  1  1  1  1  1  1  1  1]
 [ 2  2  2  2  2  2  2  2  2  2  2]
 [ 3  3  3  3  3  3  3  3  3  3  3]
 [ 4  4  4  4  4  4  4  4  4  4  4]
 [ 5  5  5  5  5  5  5  5  5  5  5]]
```

```
plt.scatter(x,y)
plt.scatter(0,0,color='orange')
plt.grid()
plt.show()
```

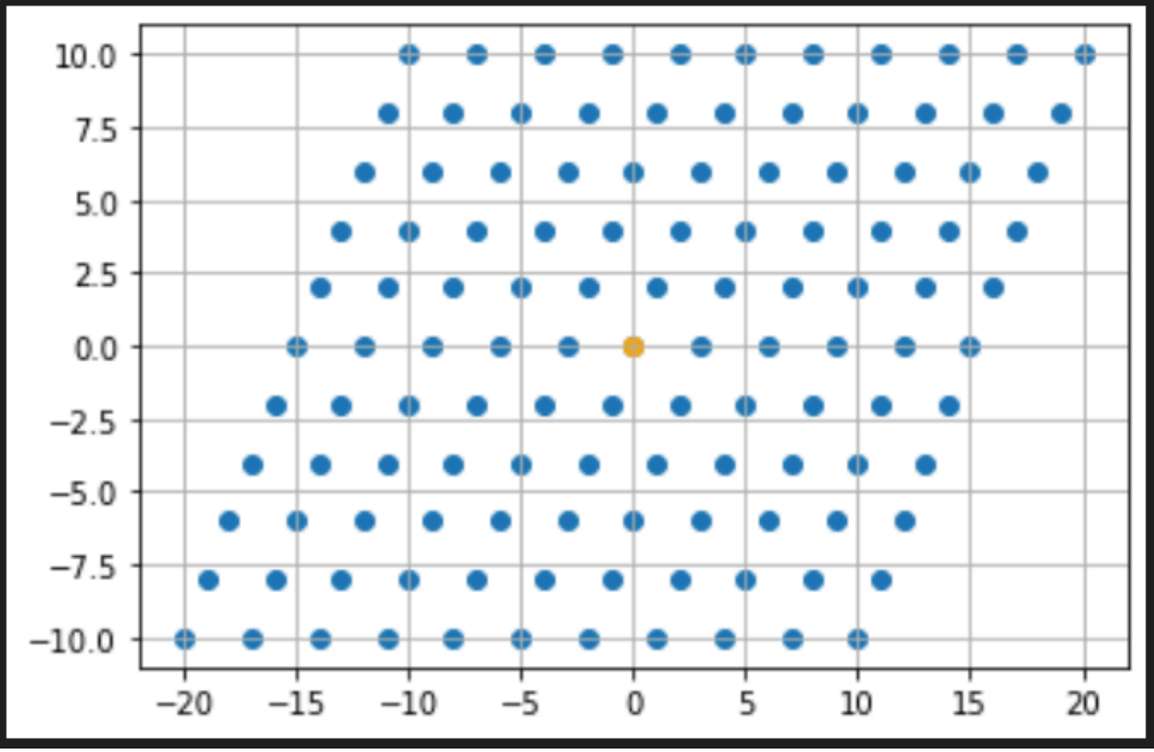


실습

```
M = np.array([[3,1],
              [0,2]])
print(M)
print(M.shape)
```

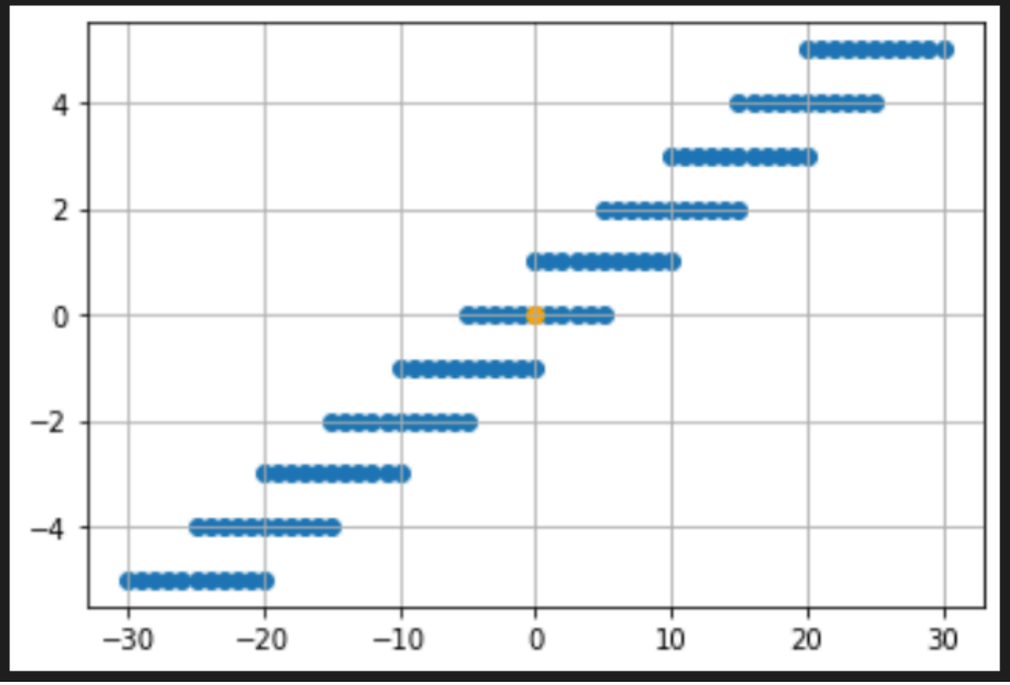
```
[[3 1]
 [0 2]]
(2, 2)
```

```
t_x, t_y = np.dot(M,np.array([x.flatten(), y.flatten()]))
plt.scatter(t_x,t_y)
plt.scatter(0,0,color='orange')
plt.grid()
plt.show()
```

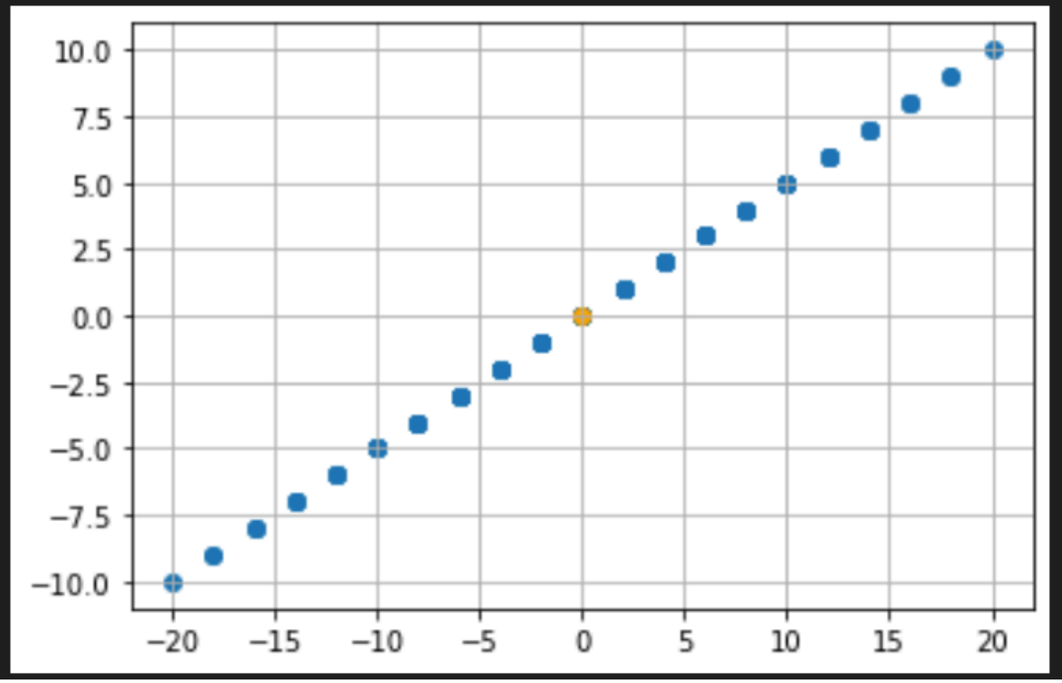


실습

```
M = np.array([[1,5],
              [0,1]])
t_x, t_y = np.dot(M,np.array([x.flatten(), y.flatten()]))
plt.scatter(t_x,t_y)
plt.scatter(0,0,color='orange')
plt.grid()
plt.show()
```




```
M = np.array([[2,-2],
              [1,-1]])
t_x, t_y = np.dot(M,np.array([x.flatten(), y.flatten()]))
plt.scatter(t_x,t_y)
plt.scatter(0,0,color='orange')
plt.grid()
plt.show()
```



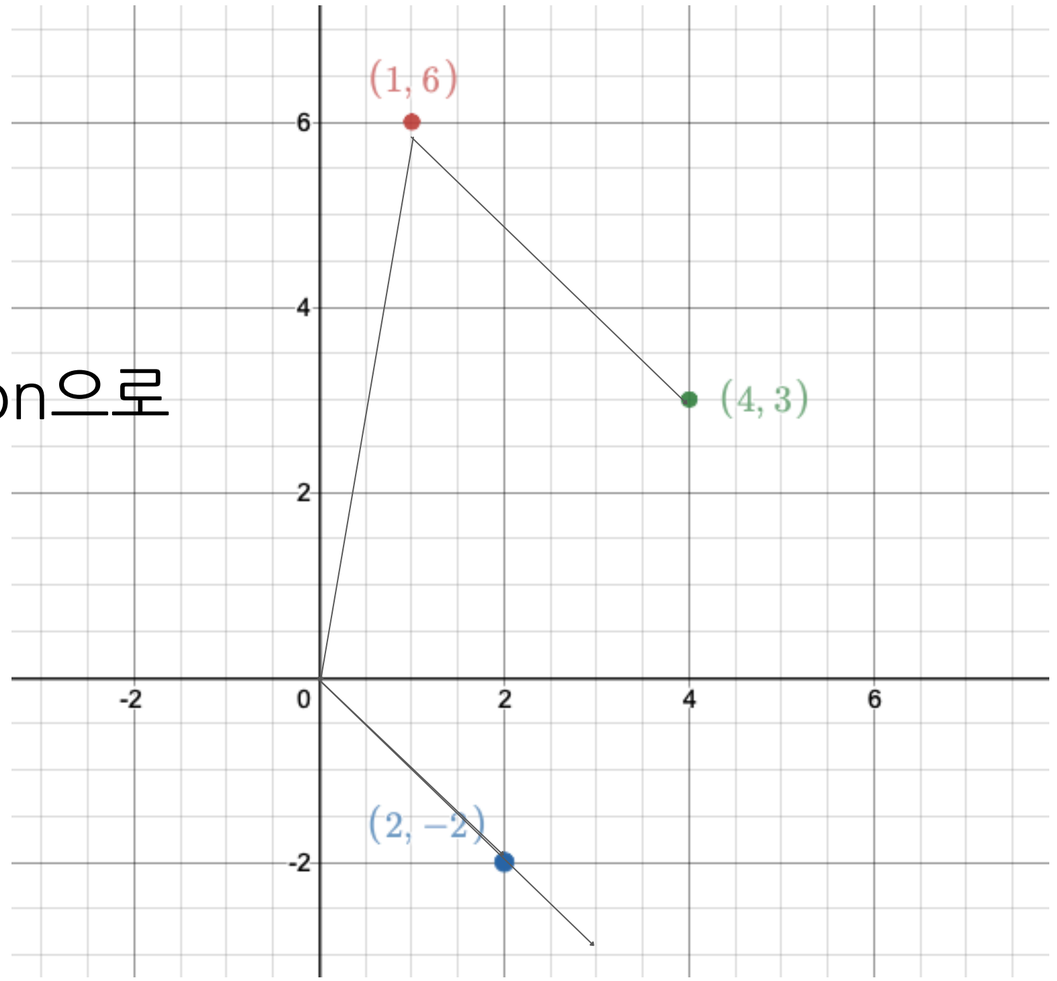
Linear dependence

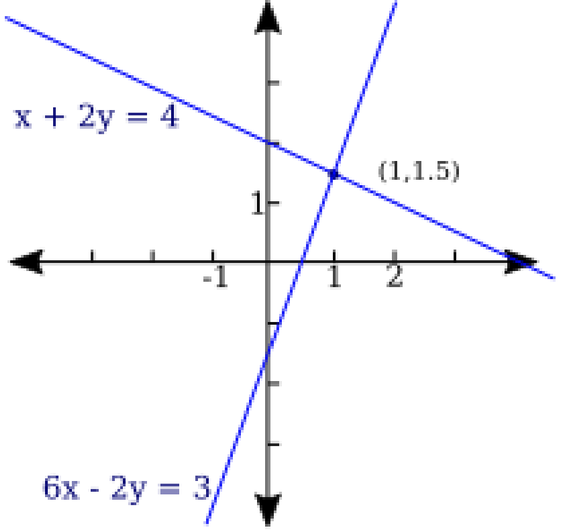
Linear system의 다른 해석

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

A의 column vector의 direction으로
얼만큼 가면 b가 나오는가?

$$\boldsymbol{Ax = b}$$





Linear combination

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix} \qquad \boldsymbol{A}\boldsymbol{x} = \sum_i x_i \boldsymbol{A}_{:,i}$$

$$\begin{bmatrix} 1 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 2 \\ -2 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

The span of a set of vectors

The set of all points obtainable by linear combination of the vectors

예) { $\begin{bmatrix}1\\6\end{bmatrix}$ $\begin{bmatrix}2\\-2\end{bmatrix}$ }의 span = $\begin{bmatrix}1\\6\end{bmatrix} x_1 + \begin{bmatrix}2\\-2\end{bmatrix} x_2$ for $x_i \in \mathbb{R}$

Column space of a matrix

= The span of its column vectors

예) $\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix}$ 의 column space = $\begin{bmatrix} 1 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 2 \\ -2 \end{bmatrix} x_2$ for $x_i \in \mathbb{R}$

해의 유무

$\boldsymbol{Ax} = \boldsymbol{b}$ 의 해가 존재하는지 == $\mathbf{b} \in$ the column space of $\boldsymbol{A}$

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 2 \\ -2 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

해의 유무

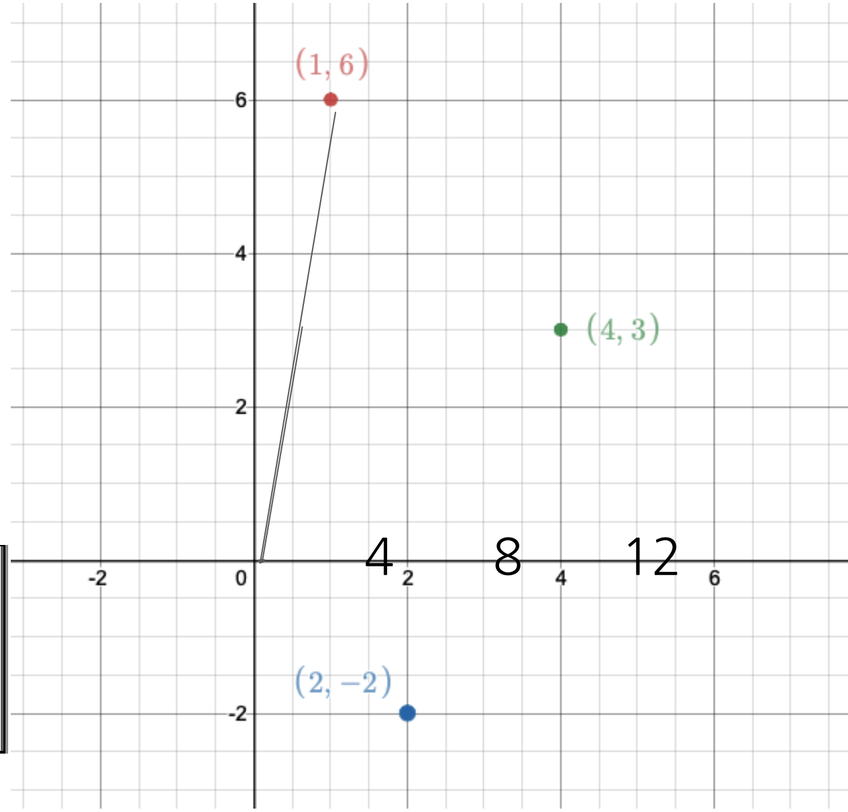
$\boldsymbol{Ax = b}$ 의 해가 존재하는지 == $\mathbf{b} \in$ the column space of $\boldsymbol{A}$

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 2 \\ -2 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 1 \\ 3 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$



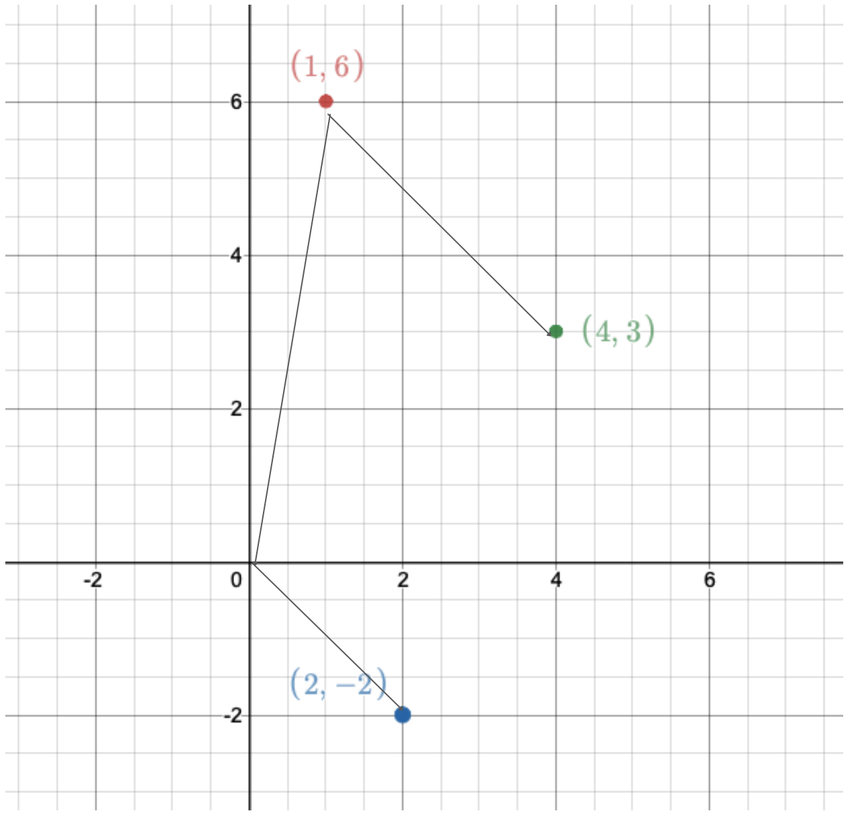
해의 유무 for any $\mathbf{b} \in \mathbb{R}^m$

$\boldsymbol{Ax} = \boldsymbol{b}$ 의 해가 존재하는지 == $\mathbf{b} \in$ the column space of $\boldsymbol{A}$

== the column space of $\boldsymbol{A}$ is all of $\mathbb{R}^m$

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 2 \\ -2 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$



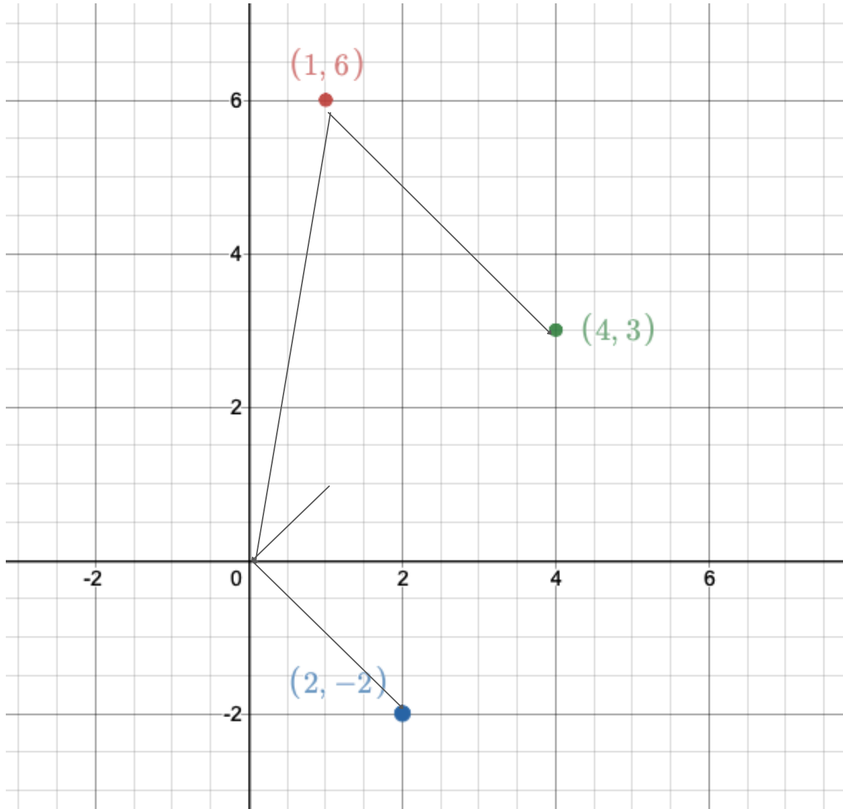
Linear dependence

A set of vectors is linearly independent
if no vector in the set is a linear combination of the other vectors.
한 벡터를 추가해도 linear combination으로 닿는 점이 추가로 생기지는 않는다.

예) 의 column vectors는 linearly dependent.

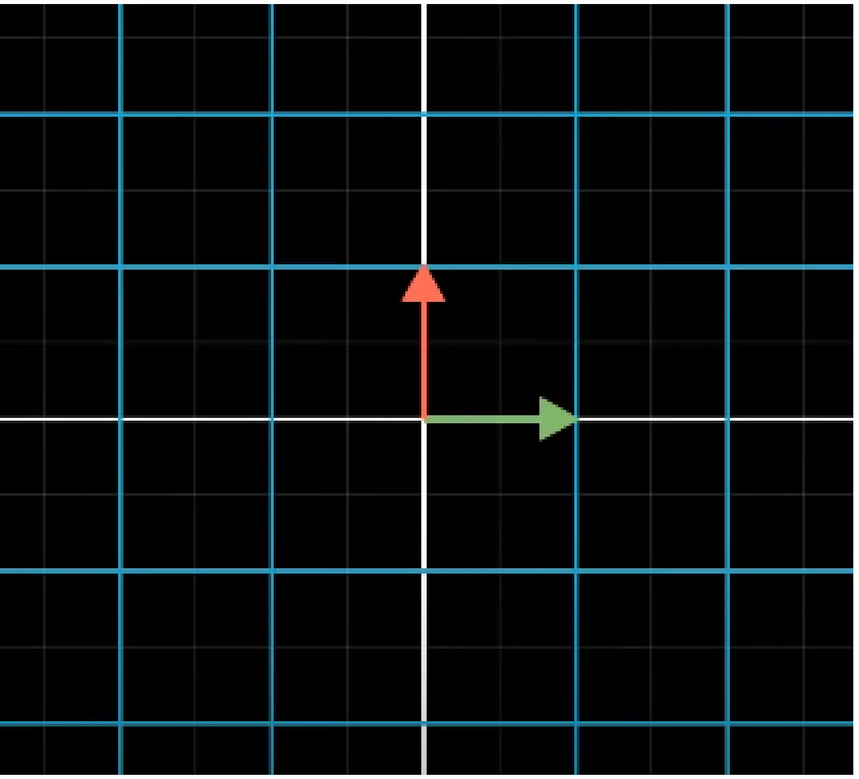
$$\begin{bmatrix} 1 & 2 & 1 \\ 6 & -2 & 1 \end{bmatrix}$$

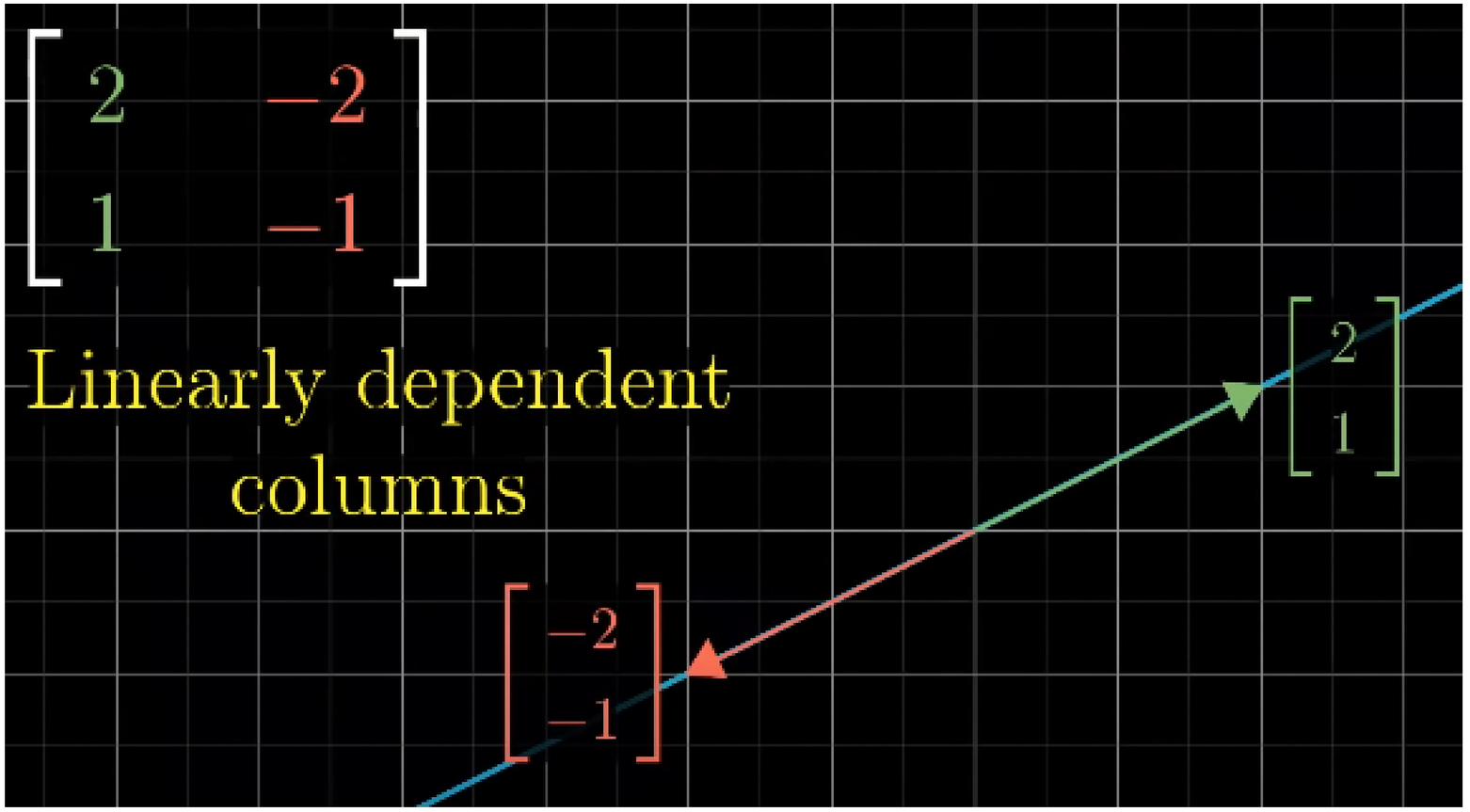
Matrix의 행보다 열이 많다
→ column vectors는 linearly dependent.



Linearly dependent columns로 된 transformation?

coordinate system을 한 span으로 납작하게 한다.

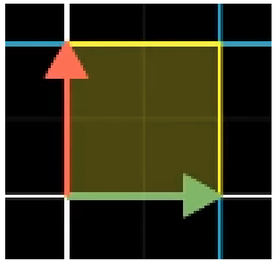


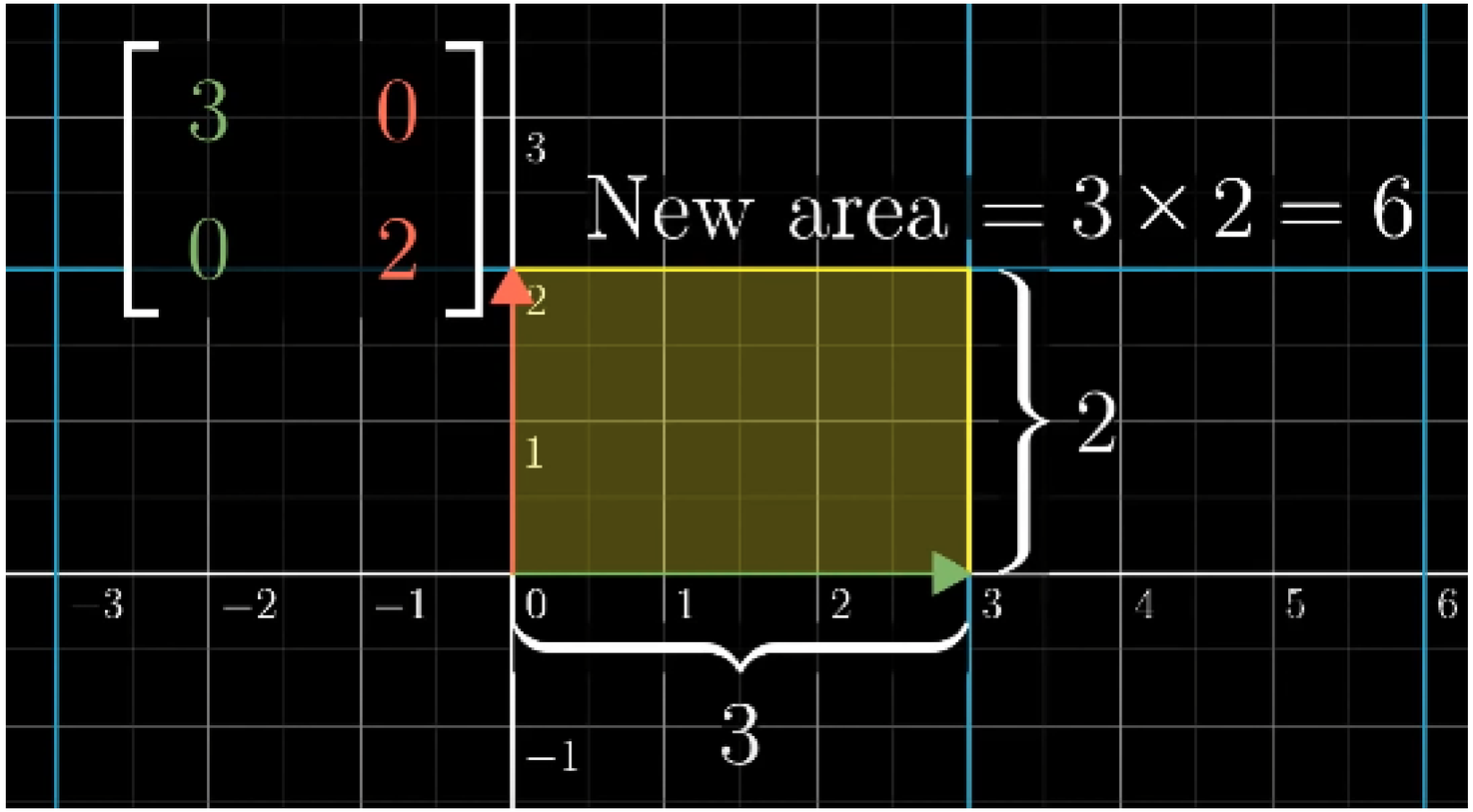


Determinant

Determinant

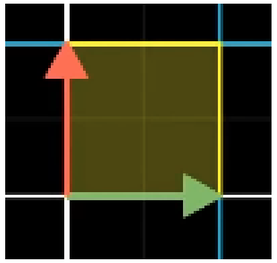
Linear transformation은 원래 coordinate system에서의 넓이를 변형한다.

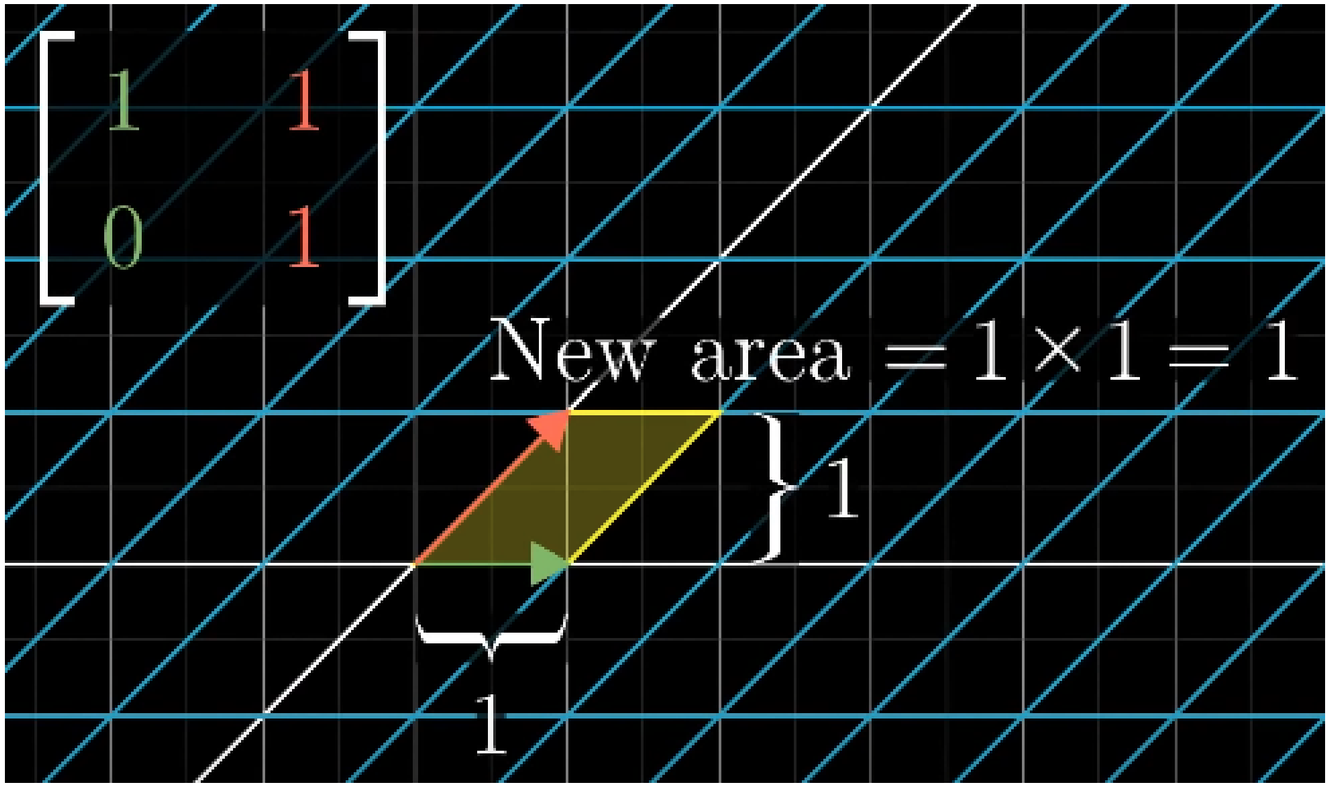




Determinant

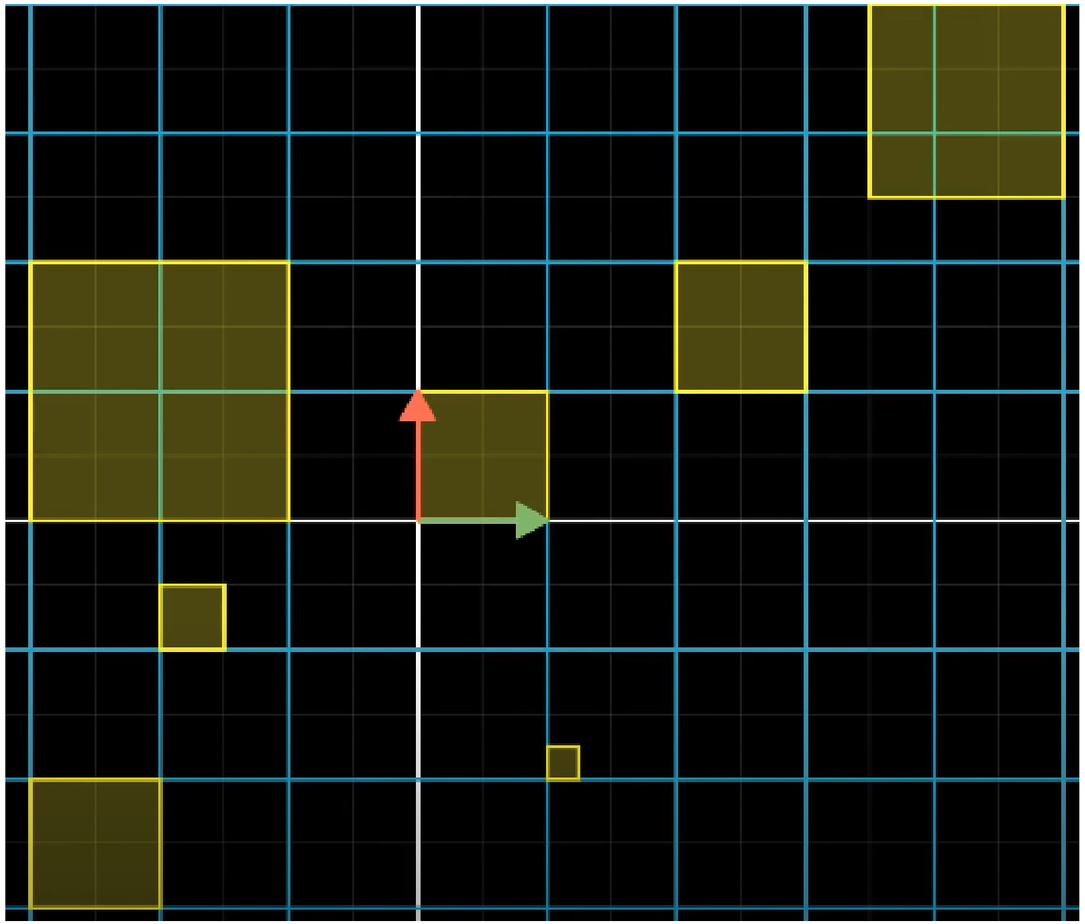
Linear transformation은 원래 coordinate system에서의 넓이를 변형한다.

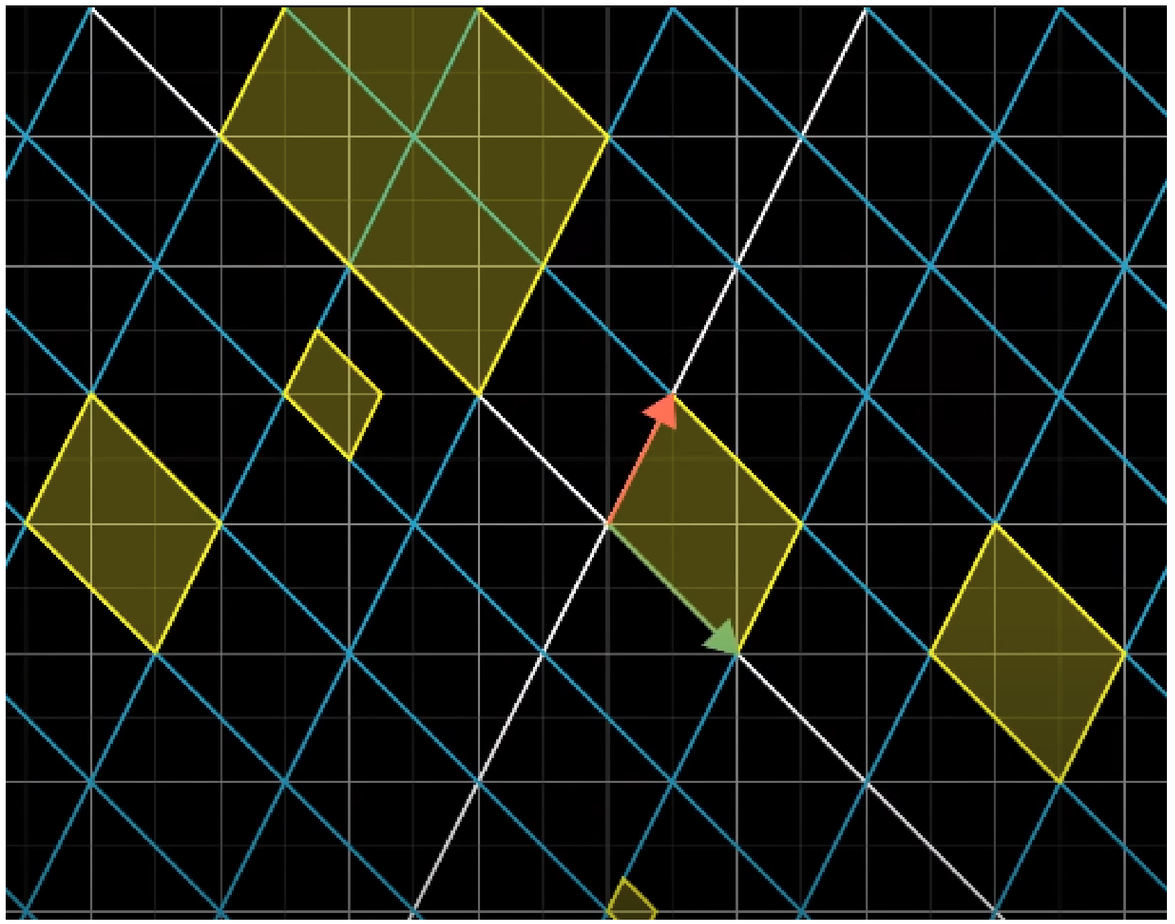




Determinant

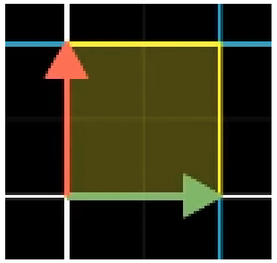
Linear transformation은 원래 coordinate system에서의 넓이를 변형한다.

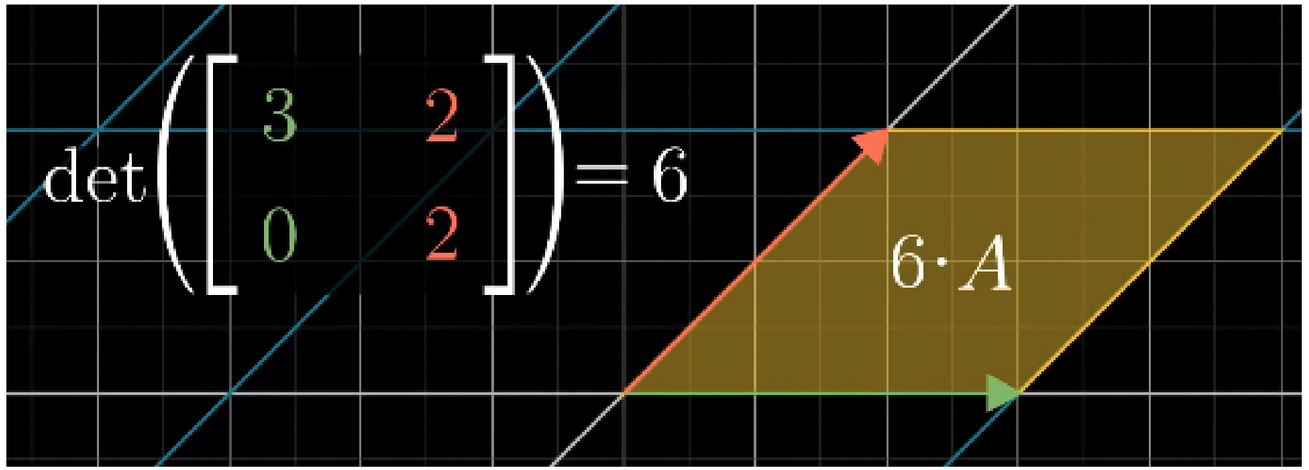


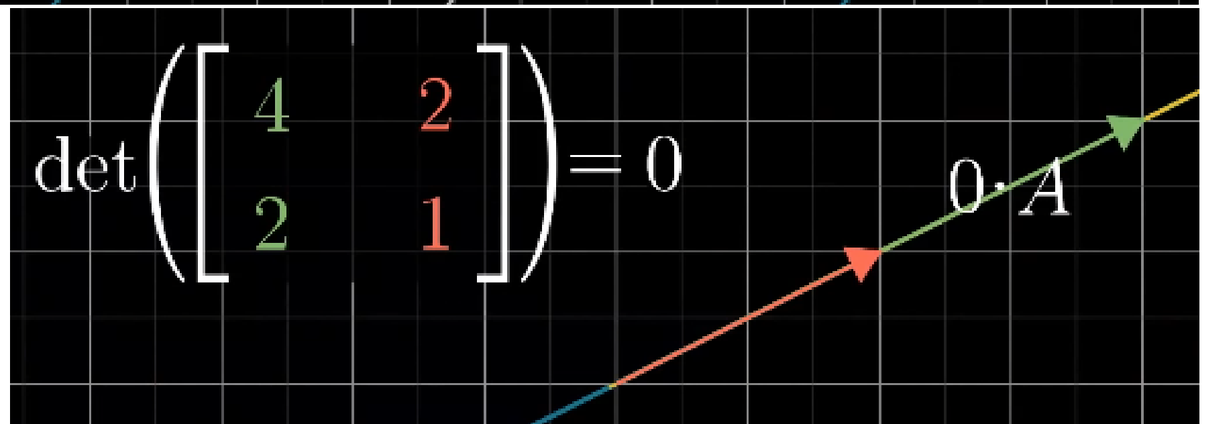


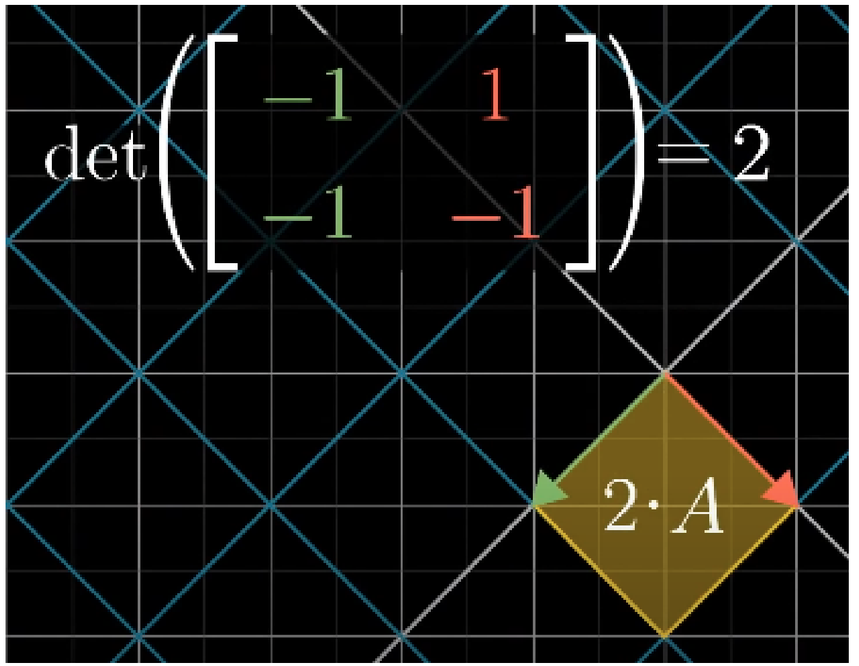
Determinant

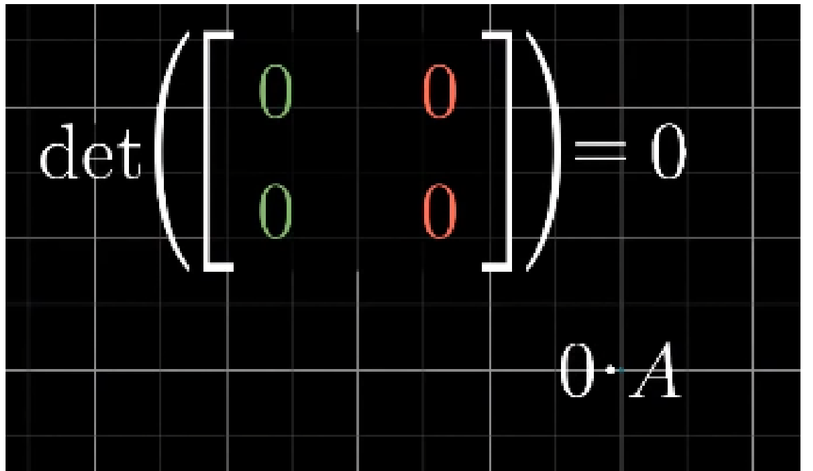
Determinant는 넓이가 몇배로 변하는지를 나타낸다.
det(A), det A, |A|





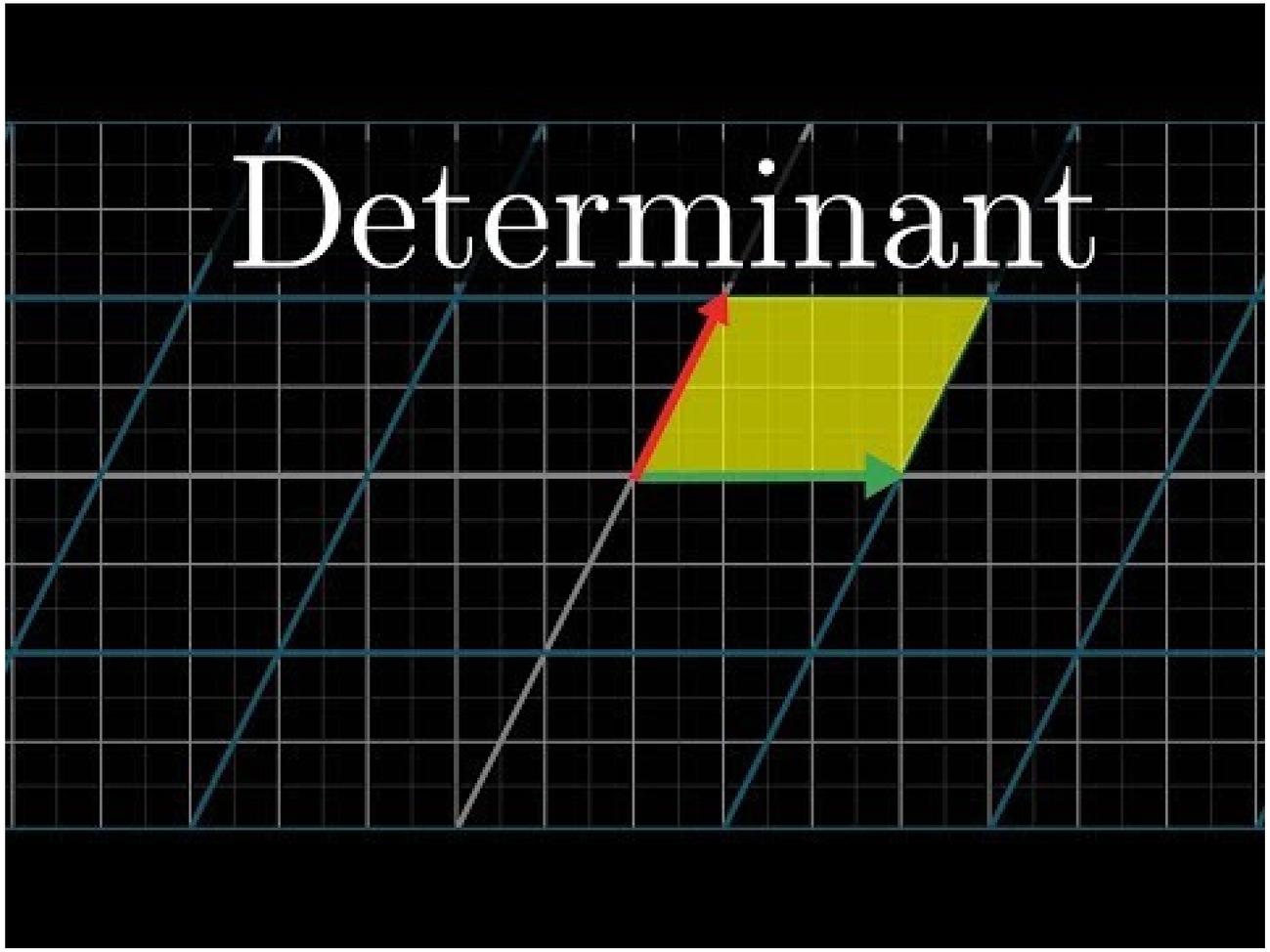




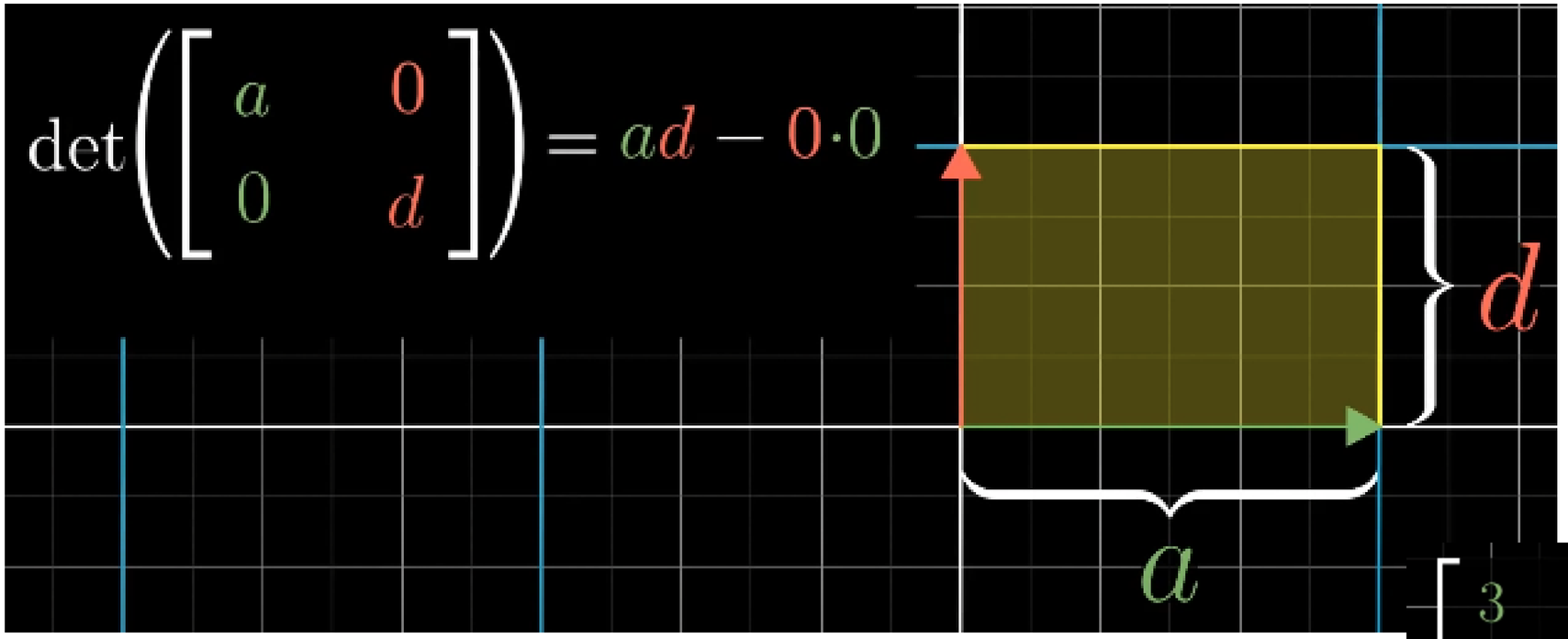


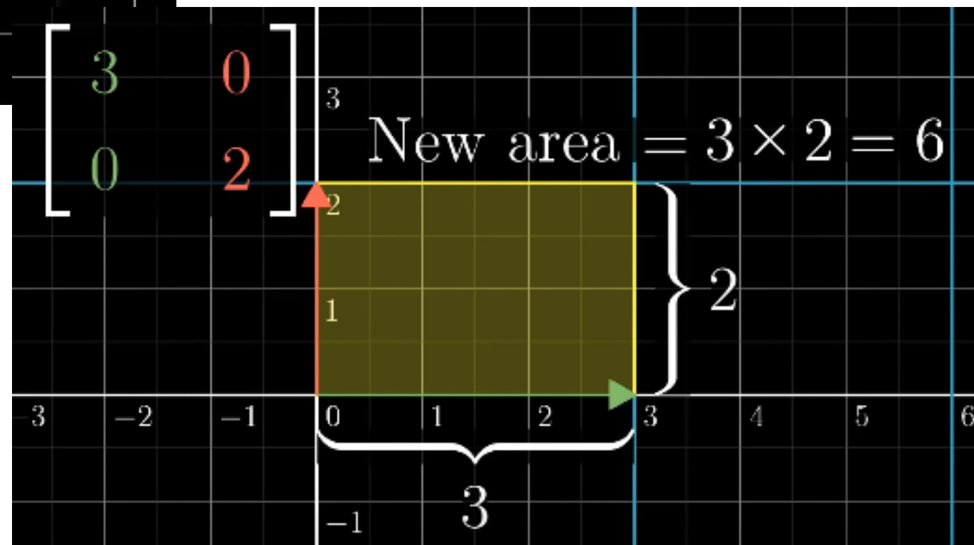
Determinant

Negative determinant는 coordinate system의 뒤집힘을 나타낸다.

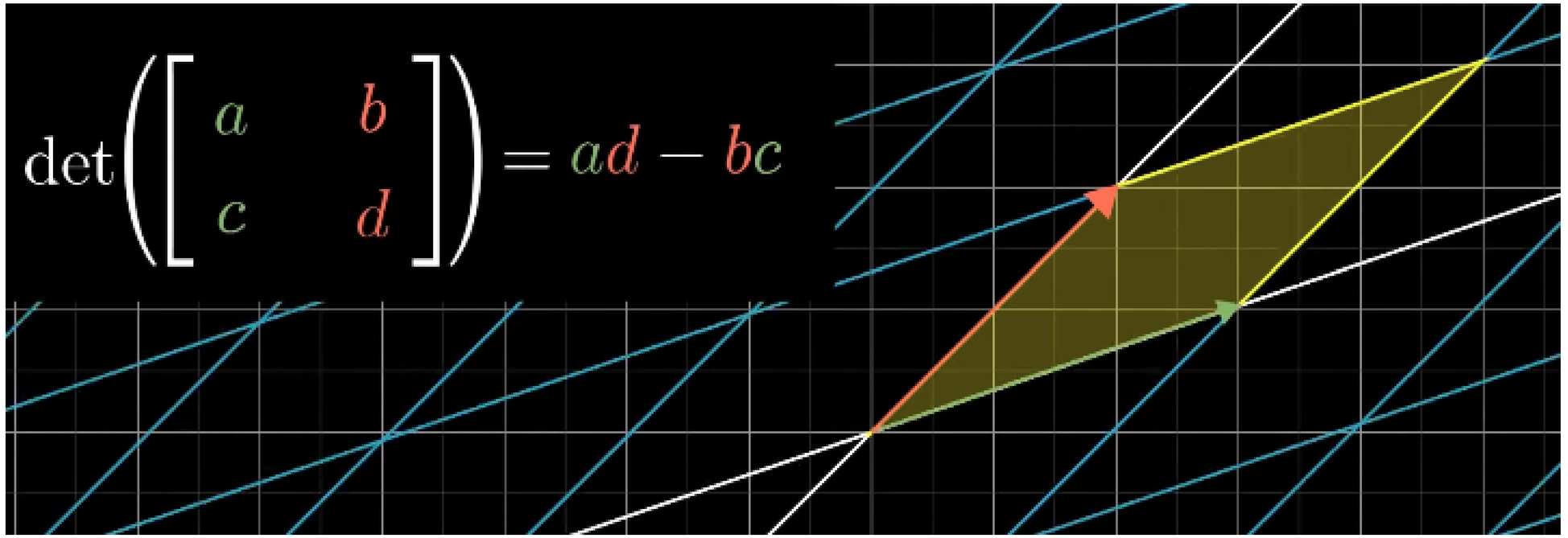


How to compute determinant

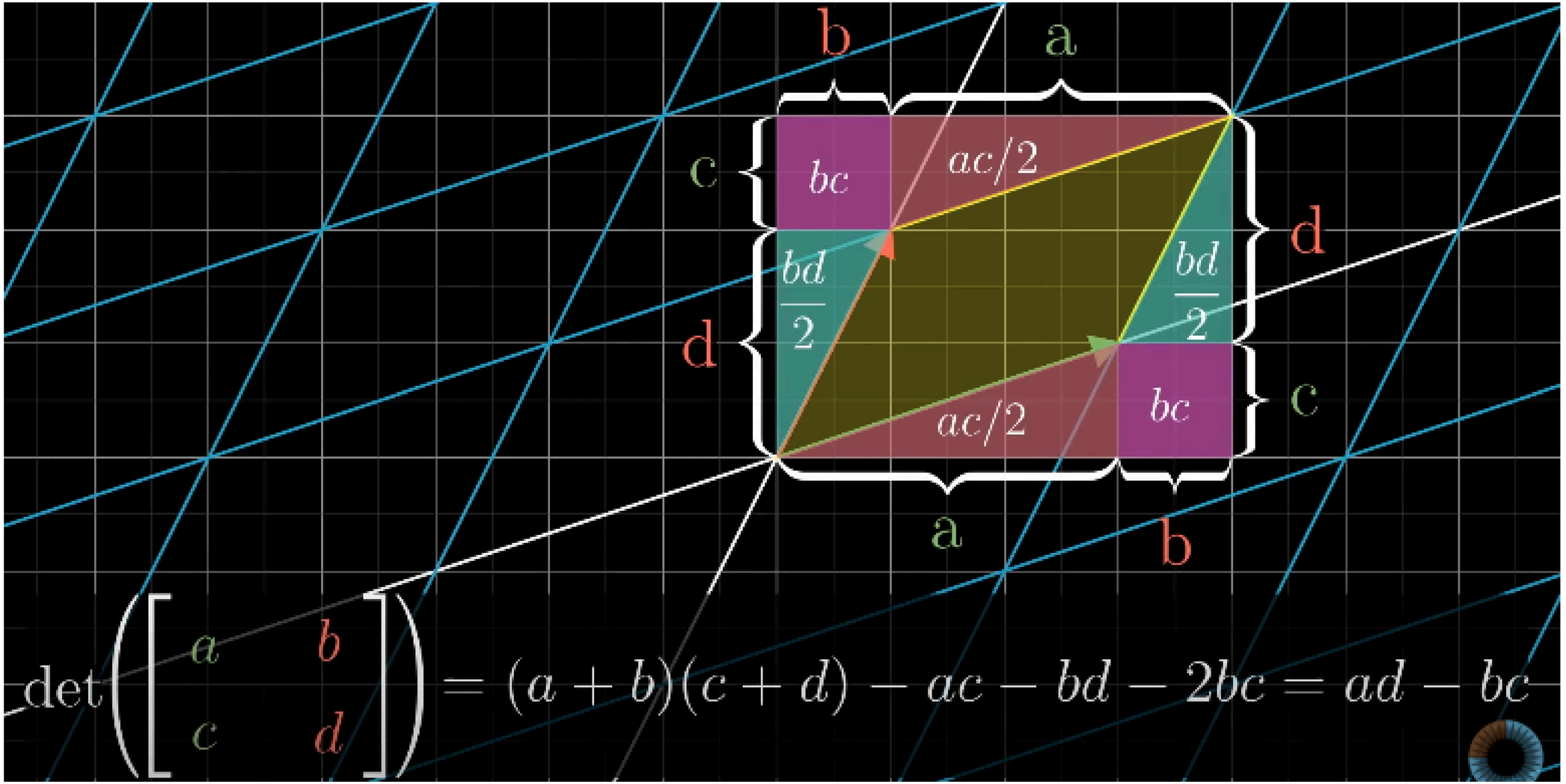




How to compute determinant



How to compute determinant



다시보기: Visualizing linear transformations

Linear transformations and matrices:
https://youtu.be/kYB8IZa5AuE
The determinant: https://youtu.be/Ip3X9LOh2dk

Rank

The (column) rank of a matrix **A**

= the dimension of the vector space generated (or spanned) by its columns.
= maximal number of linearly independent columns of **A**.

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 1 \\ 6 & -2 & 1 \end{bmatrix} \quad \begin{bmatrix} 2 & 2 \\ 6 & -3 \\ 1 & 2 \end{bmatrix}$$

The (row) rank of a matrix **A**

= the dimension of the vector space generated (or spanned) by its rows.
= maximal number of linearly independent rows of **A**.

$$\begin{bmatrix} 1 & 2 \\ 6 & -2 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 1 \\ 6 & -2 & 1 \end{bmatrix} \quad \begin{bmatrix} 2 & 2 \\ 6 & -3 \\ 1 & 2 \end{bmatrix}$$

Gaussian elimination → Rank 계산하기

Rn = n-th row

$$\begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix} \xrightarrow{2R_1+R_2\to R_2} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 3 & 5 & 0 \end{bmatrix} \xrightarrow{-3R_1+R_3\to R_3} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & -1 & -3 \end{bmatrix}$$

$$\xrightarrow{R_2+R_3\to R_3} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{-2R_2+R_1\to R_1} \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix}.$$

Gaussian elimination

$$
\begin{aligned}
R_{2'} &= 2R_1 + R_2 \\
R_{3'} &= -3R_1 + R_3 \\
R_{3''} &= R_{2'} + R_{3'} \\
&= (2R_1 + R_2) + (-3R_1 + R_3) \\
&= -R_1 + R_2 + R_3 = 0 \\
R_3 &= -R_1 + R_2
\end{aligned}
$$

$$
\begin{bmatrix} 1 & 2 & 1 \\ -2 & -3 & 1 \\ 3 & 5 & 0 \end{bmatrix}
\xrightarrow{2R_1+R_2\to R_2}
\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 3 & 5 & 0 \end{bmatrix}
\xrightarrow{-3R_1+R_3\to R_3}
\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & -1 & -3 \end{bmatrix}
$$

$$
\xrightarrow{R_2+R_3\to R_3}
\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix}
\xrightarrow{-2R_2+R_1\to R_1}
\begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{bmatrix}.
$$

Identity and Inverse matrices

Equation의 solution 구하기

$$x \times 3 = 6$$

$$x = 6/3$$

Linear system의 solution 구하기?

$$x \times 3 = 6$$

$$x = 6/3$$

$$Ax = b$$

$$x = ?$$

$$\boldsymbol{Ax} = \boldsymbol{b}$$

$$\boldsymbol{A}^{-1}\boldsymbol{Ax} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{I}_n\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

identity matrix I

어느 vector에 곱해도 그 vector 자신이 되는 matrix
I_n = n-dimensional identity matrix

$$\forall \boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{I}_n \boldsymbol{x} = \boldsymbol{x}$$

$$\boldsymbol{I}_n \in \mathbb{R}^{n \times n}$$

$$I_1 = [1],\ I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},\ I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},\ \cdots,\ I_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}.$$

```
[24] np.eye(3)

     array([[1., 0., 0.],
            [0., 1., 0.],
            [0., 0., 1.]])
```

inverse matrix

Matrix **A**의 inverse = $\boldsymbol{A}^{-1}$

자신과 곱하면 identity가 나오는 행렬

$$\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}_n$$

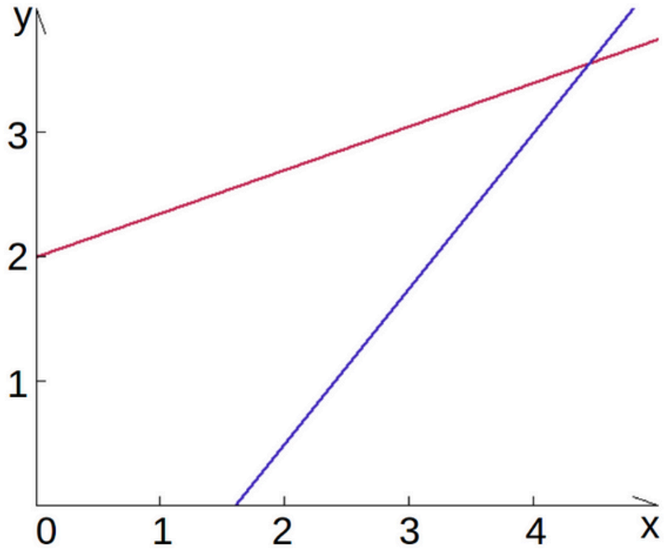
inverse matrix로 linear system의 solution 구하기

$$\begin{cases} -0.35x + y = 2 \\ 2.5x - 2y = 4 \end{cases}$$

$$\begin{bmatrix} -0.35 & 1 \\ 2.5 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

The same system in matrix form

The same system in form of a plot



$$\boldsymbol{Ax} = \boldsymbol{b}$$

$$\boldsymbol{A}^{-1}\boldsymbol{Ax} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{I}_n\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

inverse matrix

$\boldsymbol{A}^{-1}$ = np.linalg.inv(A)

```
[29] A = np.array([0.35, 1, 2.5, -2]).reshape(2, 2)
     b = np.array([2, 4]).reshape(2, 1)
     x = np.dot(np.linalg.inv(A), b)
     print(x)
```

```
[[2.5  ]
 [1.125]]
```

$$\boldsymbol{Ax} = \boldsymbol{b}$$

$$\boldsymbol{A}^{-1}\boldsymbol{Ax} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{I}_n\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

inverse matrix

Gauss-Jordan elimination

$$A = \begin{bmatrix} 1 & -2 & -4 \\ 2 & -3 & -6 \\ -3 & 6 & 15 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & -4 & 1 & 0 & 0 \\ 2 & -3 & -6 & 0 & 1 & 0 \\ -3 & 6 & 15 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & -2 & -4 & 1 & 0 & 0 \\ 0 & 1 & 2 & -2 & 1 & 0 \\ 0 & 0 & 3 & 3 & 0 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & -2 & -4 & 1 & 0 & 0 \\ 0 & 1 & 2 & -2 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & \frac{1}{3} \end{bmatrix}$$

$$\rightarrow \begin{bmatrix} 1 & 0 & 0 & -3 & 2 & 0 \\ 0 & 1 & 2 & -2 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & \frac{1}{3} \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 0 & 0 & -3 & 2 & 0 \\ 0 & 1 & 0 & -4 & 1 & -\frac{2}{3} \\ 0 & 0 & 1 & 1 & 0 & \frac{1}{3} \end{bmatrix}$$

A^{-1}

Existence of inverse

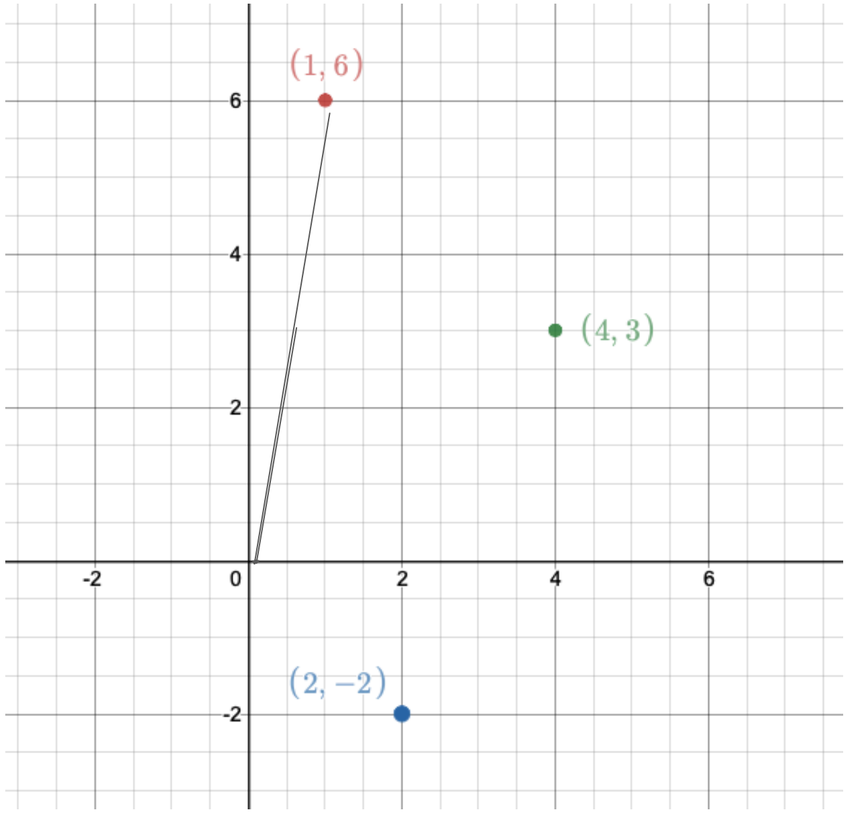
$$\boldsymbol{A}\boldsymbol{x} = \sum_i x_i \boldsymbol{A}_{:,i} = \boldsymbol{b}$$

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

$$\begin{bmatrix} 2 & 1 \\ 6 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 6 \end{bmatrix} x_1 + \begin{bmatrix} 1 \\ 3 \end{bmatrix} x_2 = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

x가 한가지로 정해질 수 없는 경우는
(=무수히 많은 x가 해가 될 수 있거나, 해가 없는 경우)
inverse가 존재하지 않는다.



Existence of inverse

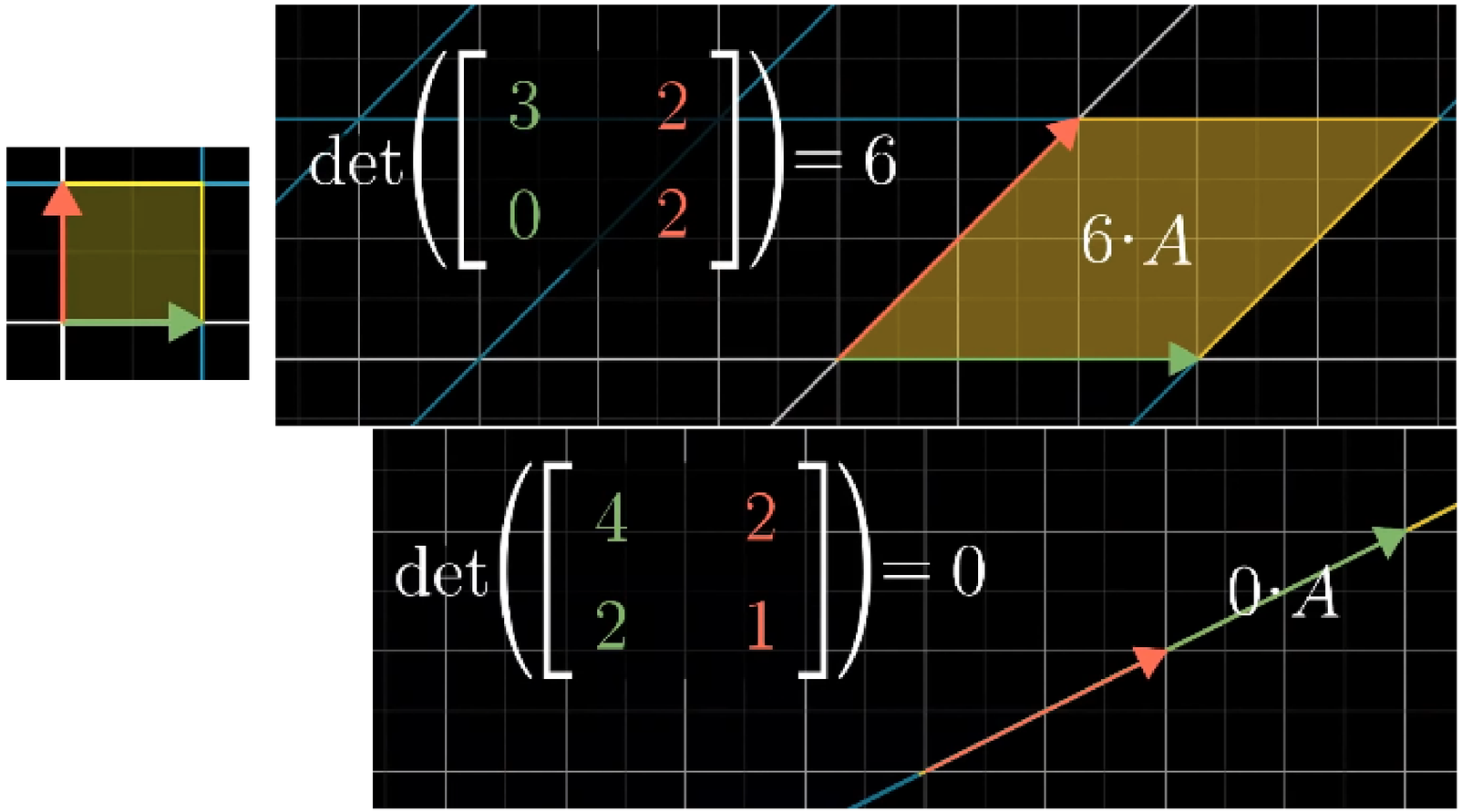
matrix A has **full rank**

Rank(A) = min(# columns, # rows)

⇔ A의 inverse가 존재한다.

Existence of inverse

If Det(A) == 0 ⇔ A의 inverse가 존재하지 않는다.
I.e., A is a singular matrix.



Augmented matrix

$$\begin{bmatrix} 2 & 2 \\ 6 & -3 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \\ 1 \end{bmatrix}$$

coefficient matrix

$$\begin{bmatrix} \mathbf{2} & \mathbf{2} & \mathbf{4} \\ \mathbf{6} & \mathbf{-3} & \mathbf{3} \\ \mathbf{1} & \mathbf{2} & \mathbf{1} \end{bmatrix}$$

augmented matrix

Ranks and solution

$$Ax = b$$

$$x = ?$$

rank(coef) < rank(aug) ⇒ **x** has no solution.
rank(coef) == rank(aug) ⇒ **x** has one or more solution.

- coef has full rank ⇒ There is exactly one solution.
- coef does not have full rank ⇒ There are infinitely many solutions.

Ranks and solution

rank(coef) == rank(aug) ⇒ **x** has one or more solution.

- coef does not have full rank ⇒ There are infinitely many solutions.

$$\begin{aligned} x + y + 2z &= 2 \\ x + y + z &= 3 \\ 2x + 2y + 2z &= 6 \end{aligned} \qquad A = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \qquad (A|B) = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 2 \\ 1 & 1 & 1 & 3 \\ 2 & 2 & 2 & 6 \end{array}\right]$$

rank(coef) < rank(aug) ⇒ **x** has no solution.

$$\begin{aligned} x + y + 2z &= 3 \\ x + y + z &= 1 \\ 2x + 2y + 2z &= 5 \end{aligned} \qquad A = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \qquad (A|B) = \left[\begin{array}{ccc|c} 1 & 1 & 2 & 3 \\ 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 5 \end{array}\right]$$

Ranks and solution

rank(coef) == rank(aug) $\Rightarrow$ **x** has one or more solution.

- coef has full rank $\Rightarrow$ There is exactly one solution.

$$(A|B) = \left[\begin{array}{ccc|c} 1 & 2 & 3 & 0 \\ 3 & 4 & 7 & 2 \\ 6 & 5 & 9 & 11 \end{array}\right]$$

left and right inverse

left inverse $$\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}_n$$

right inverse $$\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$$

The left and right inverses are equal for square matrices.

연습

rank를 구하세요
inverse를 구하세요

$$\begin{bmatrix} 2 & 2 \\ 6 & -3 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

x,y를 구하세요

$$\begin{bmatrix} 2 & 2 \\ 6 & -3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$

Eigenvectors and eigenvalues

Definition of eigenvectors and eigenvalues

For a linear transformation T,
v is an eigenvector of T if T(v) is a scalar multiple of v

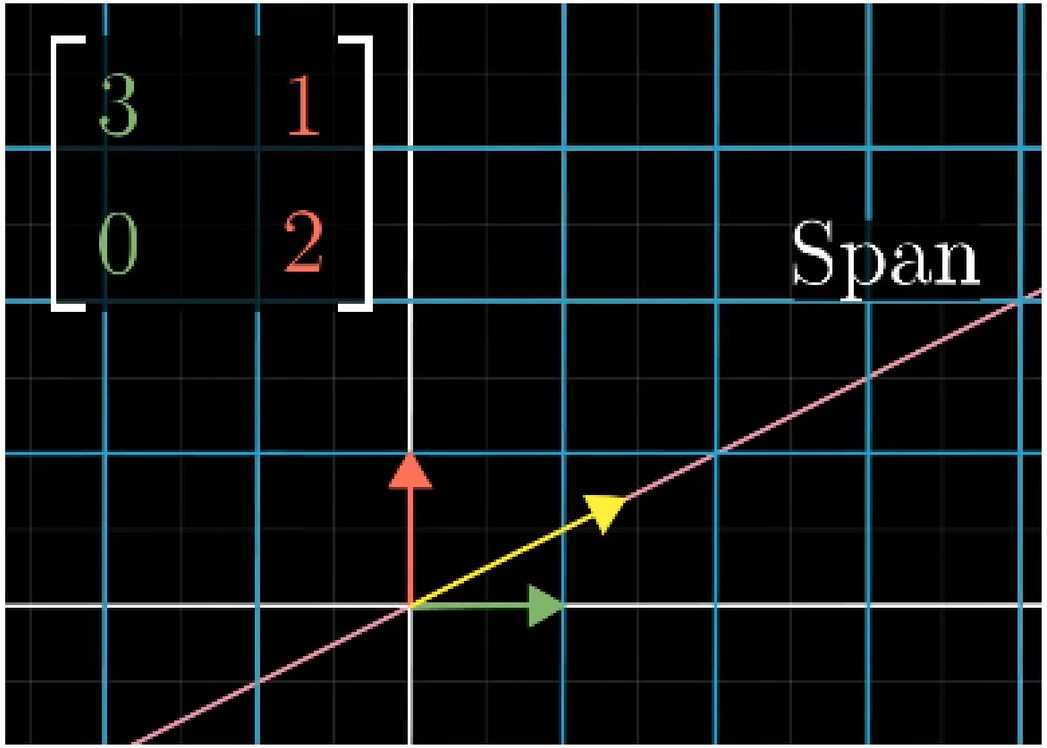
$$T(\mathbf{v}) = \lambda\mathbf{v}$$

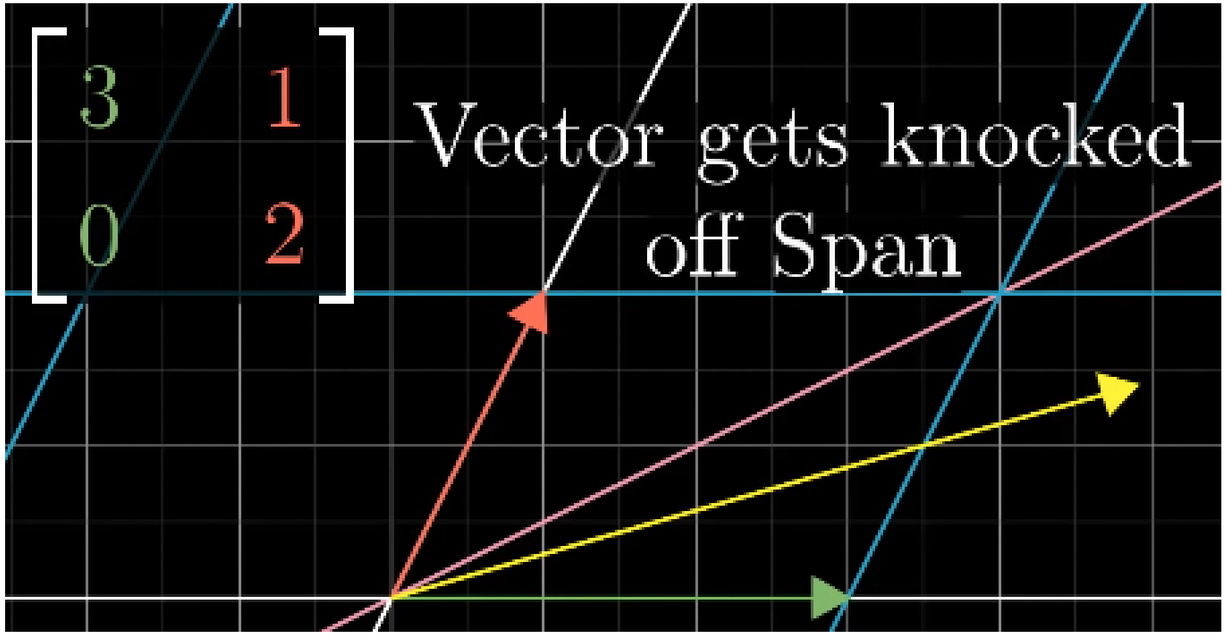
And the scalar multiple is called eigenvalue

$$A\mathbf{u} = \lambda\mathbf{u}$$

Eigenvectors and linear transformations

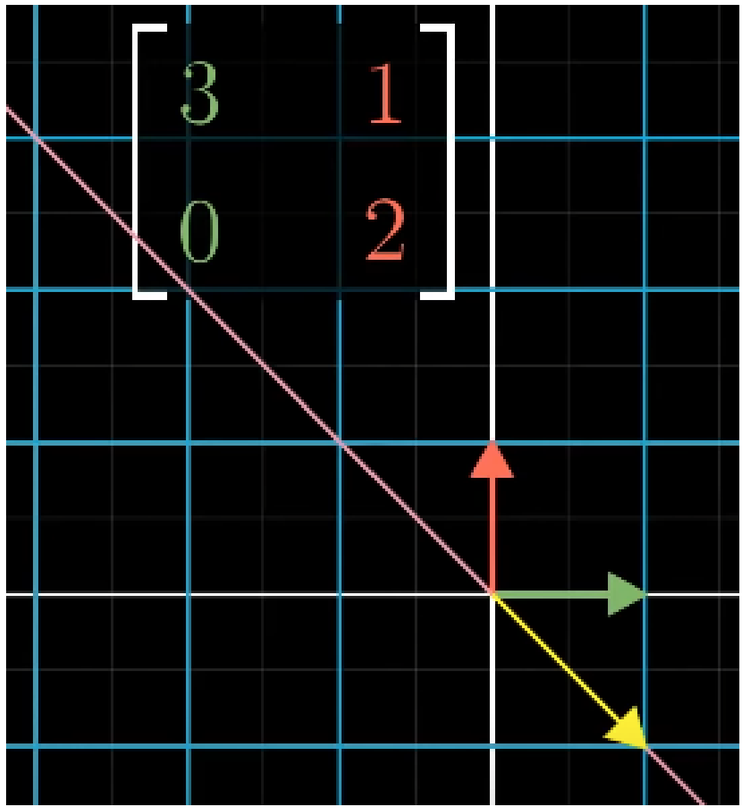
대부분의 벡터는 linear transformation을 통해 자신의 span에서 벗어난다.

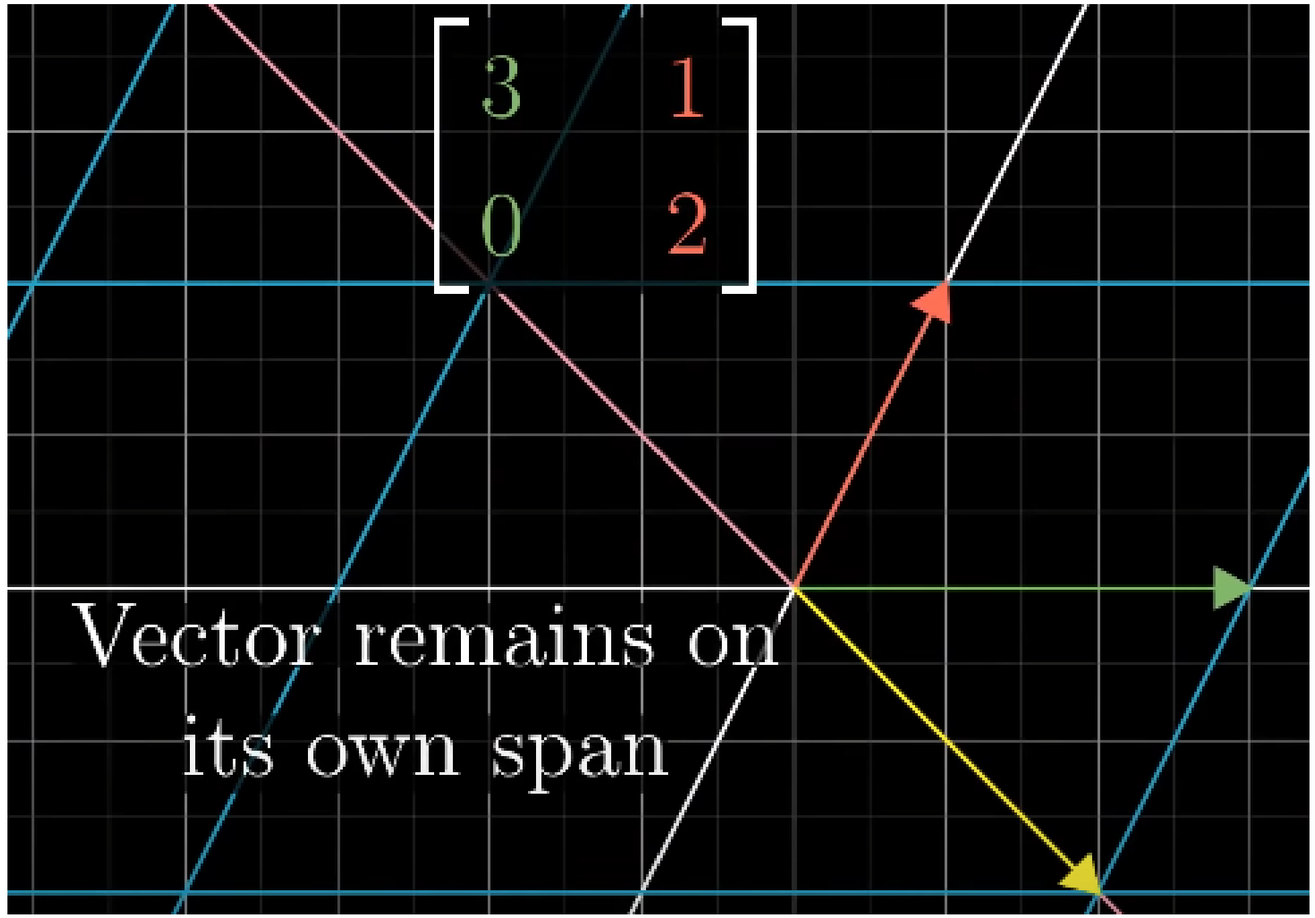




Eigenvectors and linear transformations

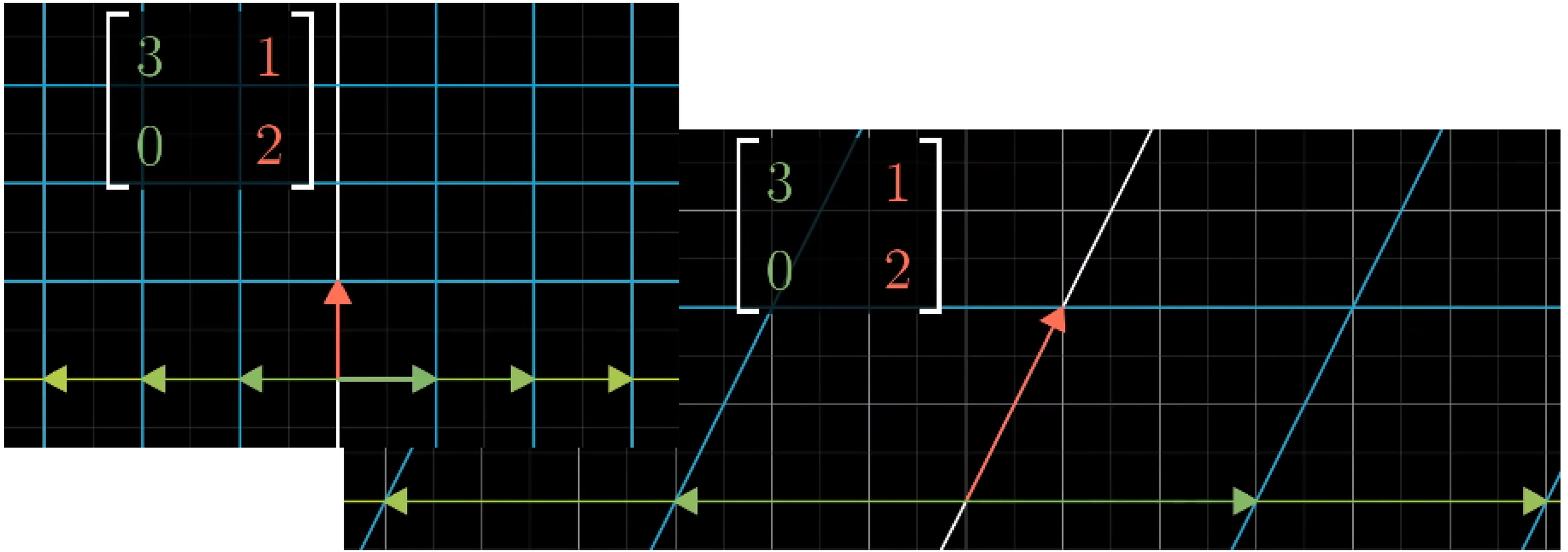
Eigenvector는 span 위에 유지된 채 변형된다.





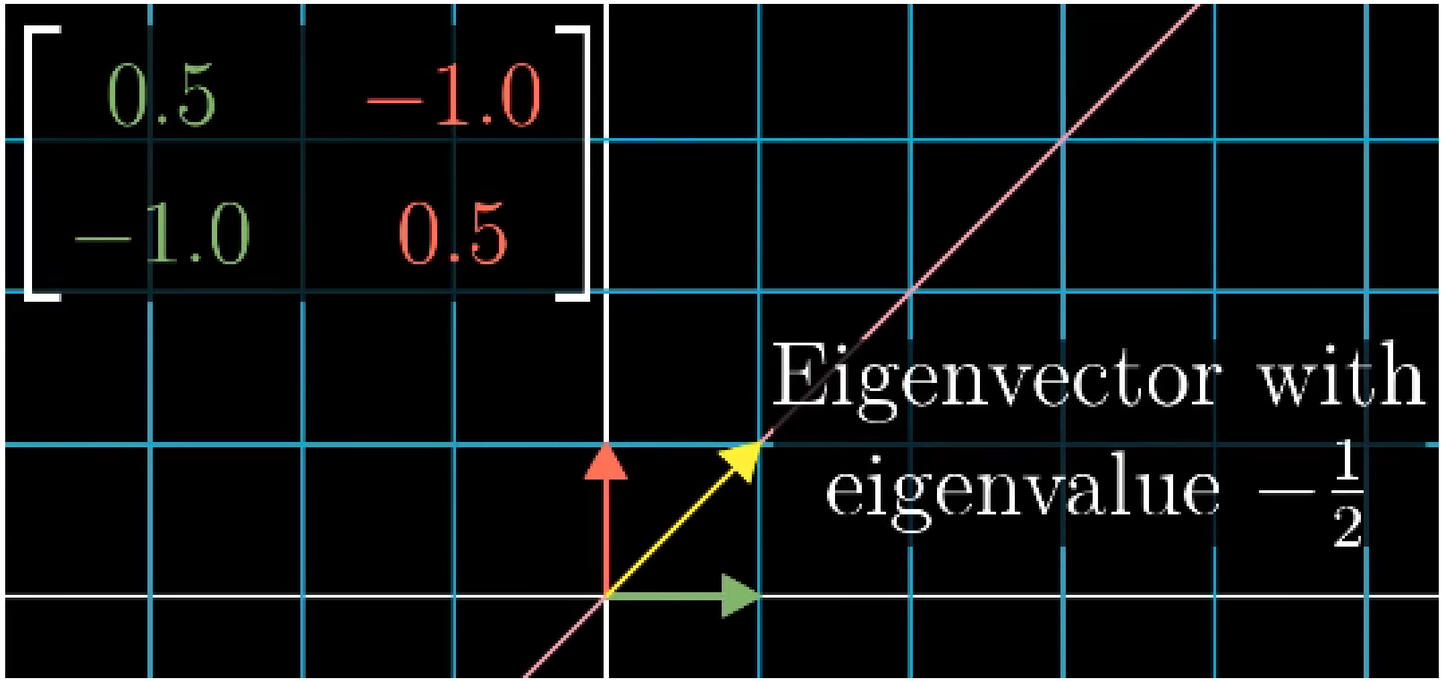
Eigenvalues and linear transformations

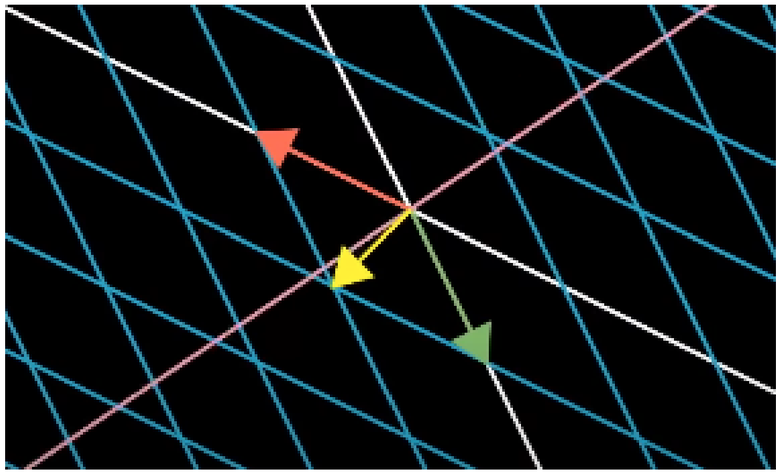
Eigenvalue는 eigenvector가 몇배 늘어날지 나타낸다.



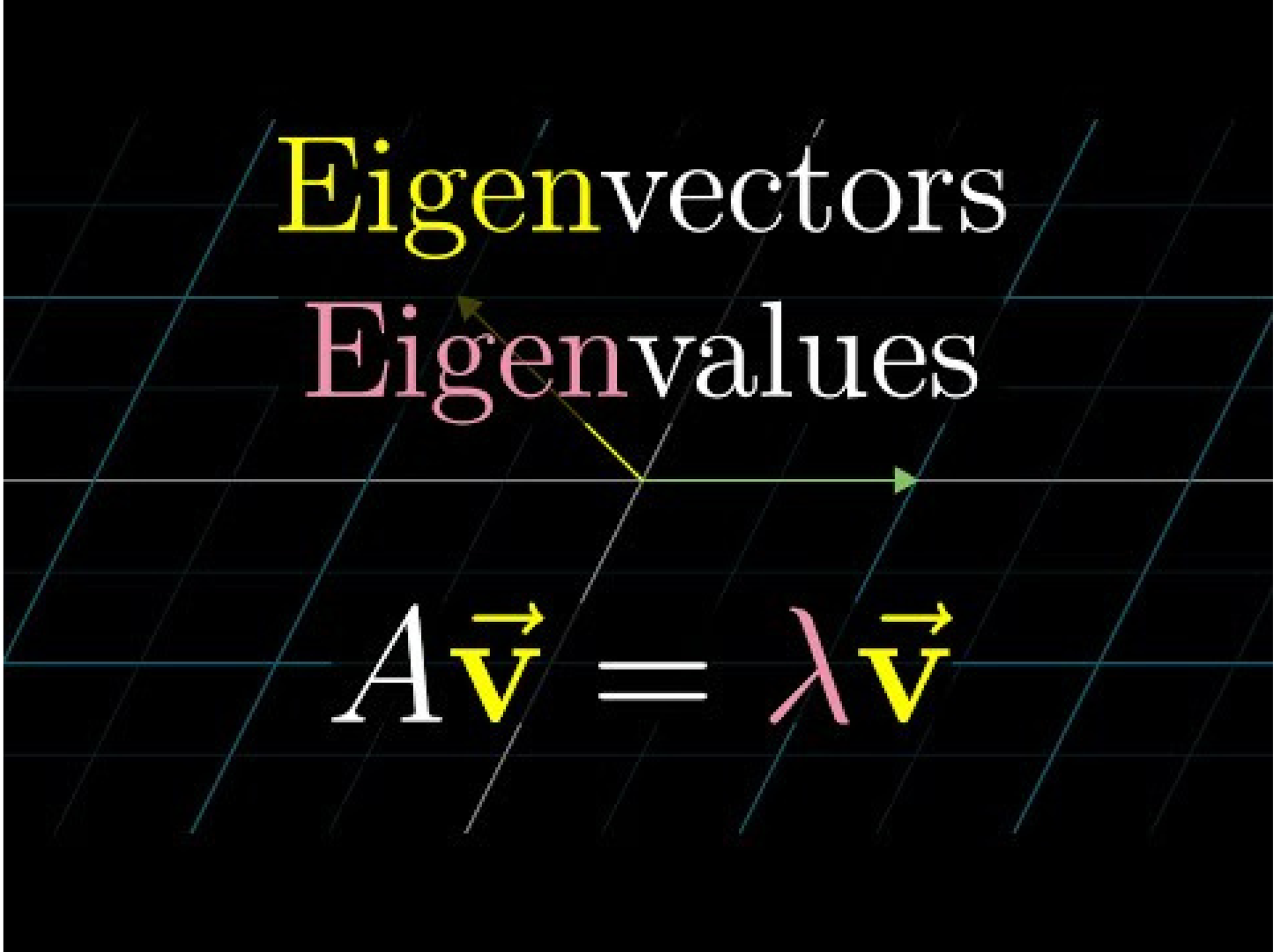
Eigenvalues and linear transformations

Negative eigenvalue에 대응하는 eigenvector는 방향이 뒤집힌다.





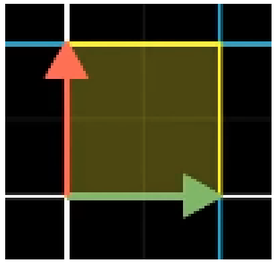
동영상으로 정리


Eigenvectors
Eigenvalues
$A\vec{\mathbf{v}} = \lambda\vec{\mathbf{v}}$

How to compute eigenvectors and eigenvalues

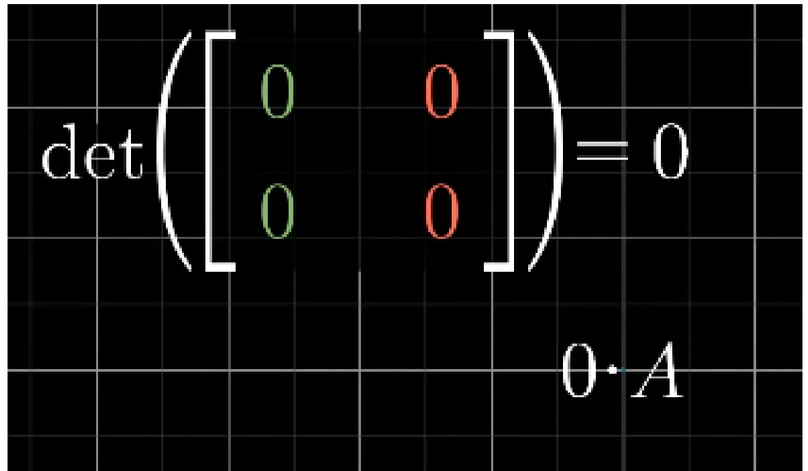
non-zero vector v에도 등식을 만족하는 λ 찾기

$$A\mathbf{v} = \lambda\mathbf{v}$$

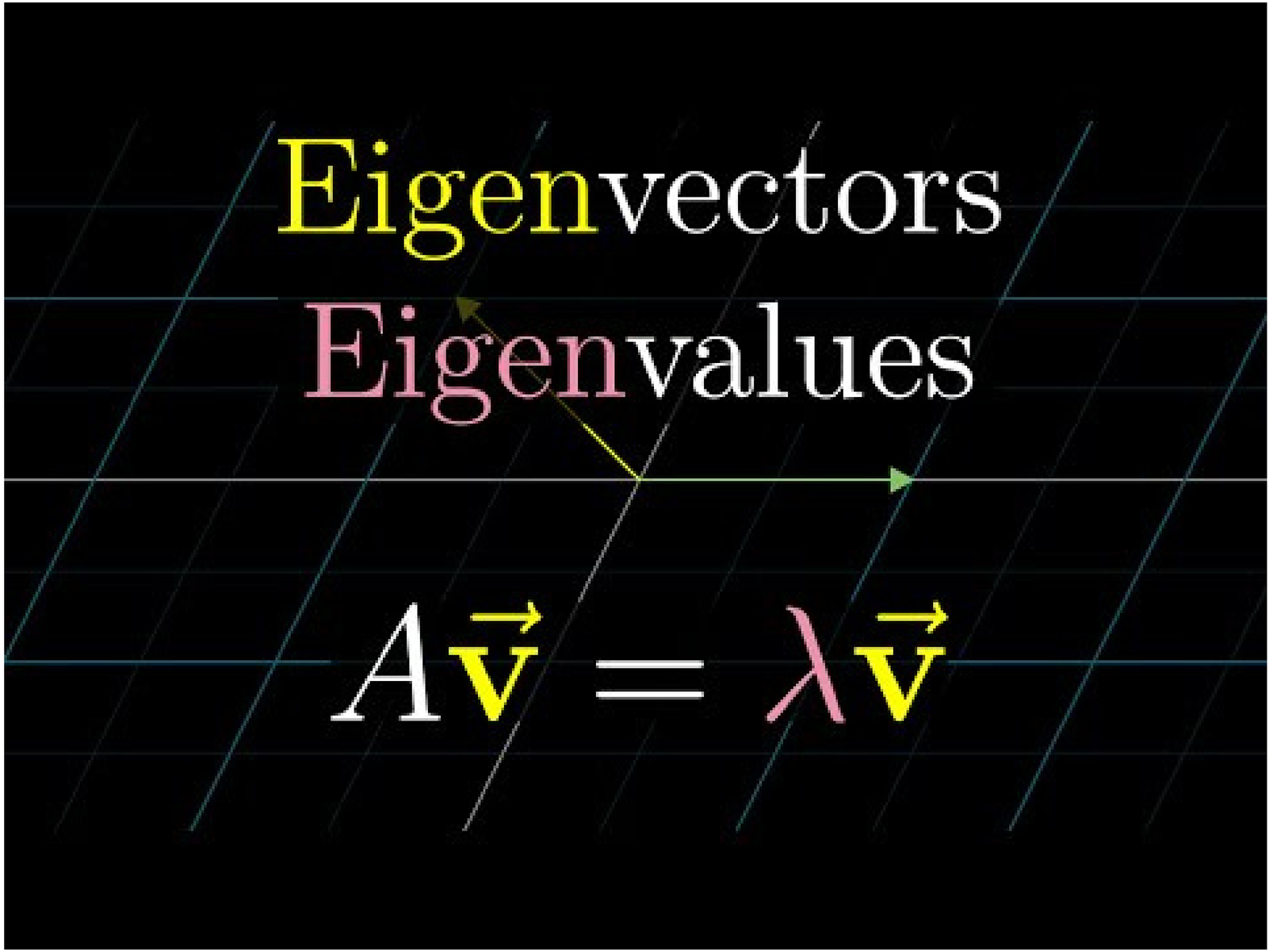


$$(A - \lambda I)\,\mathbf{v} = \mathbf{0}$$

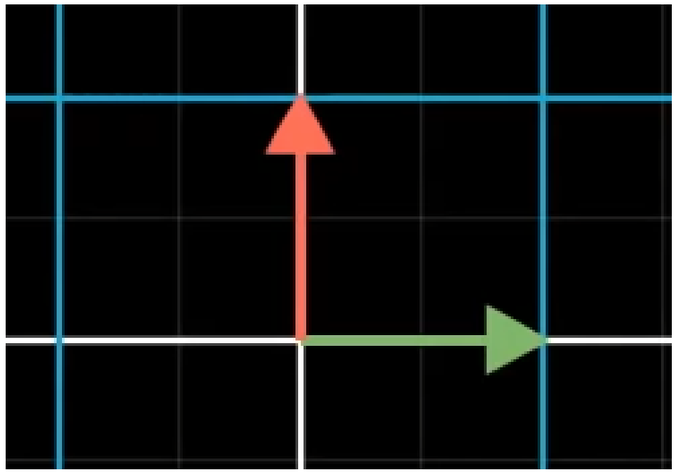
$$|A - \lambda I| = 0$$

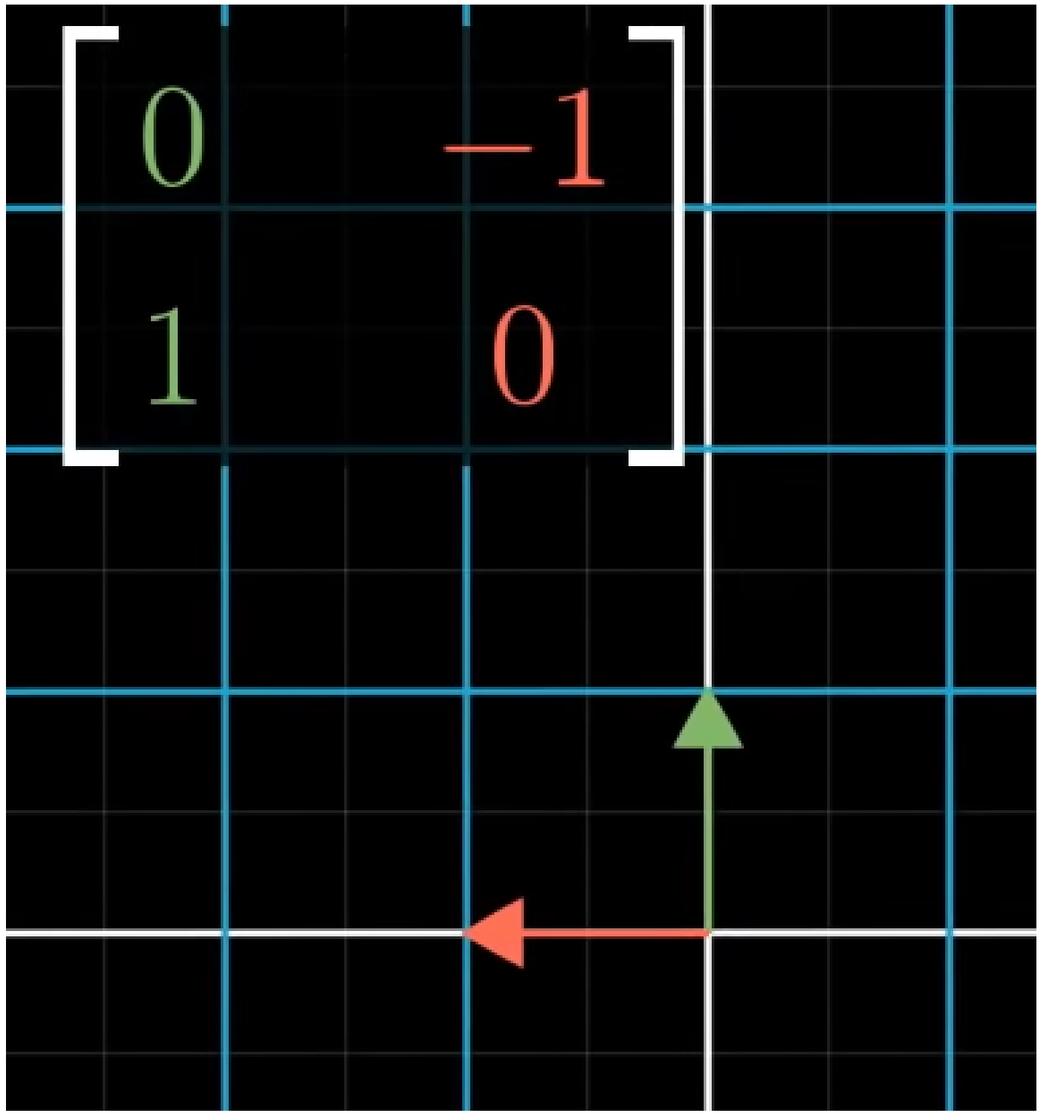


How to compute eigenvectors and eigenvalues



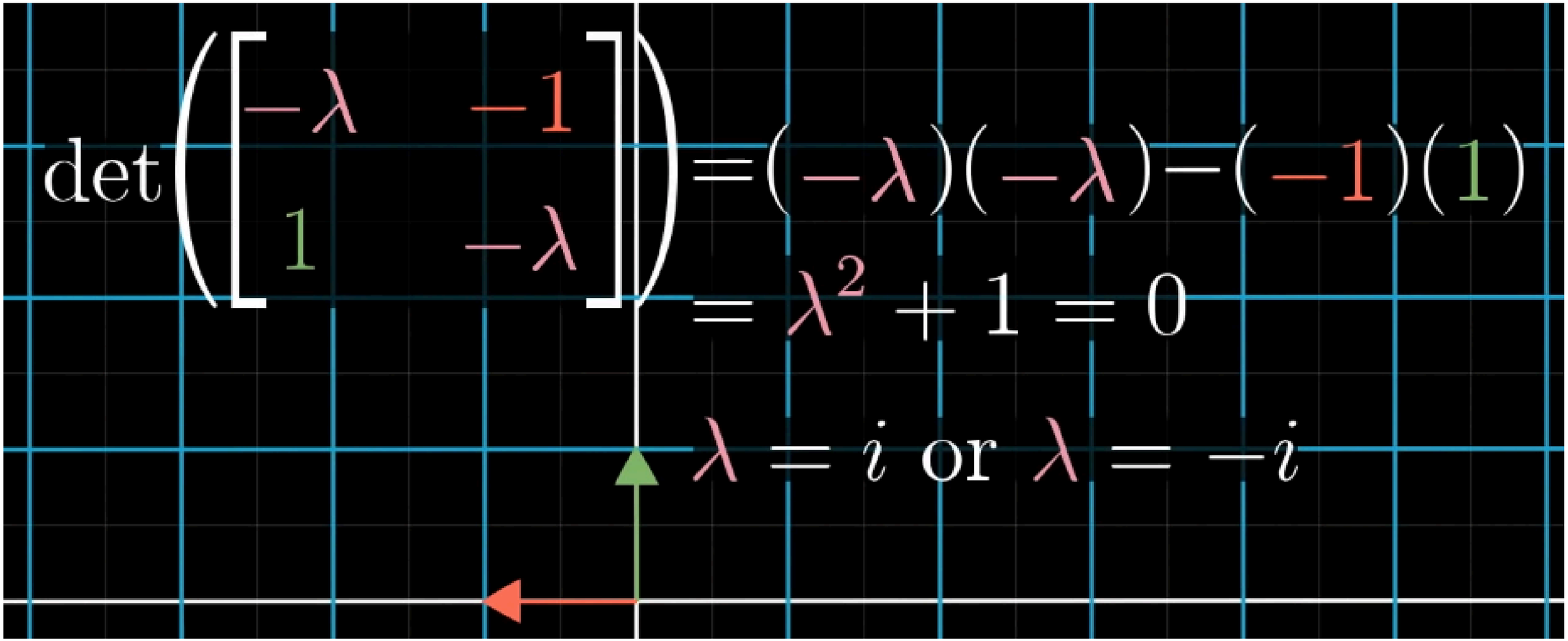
Eigenvector가 없는 linear transformation





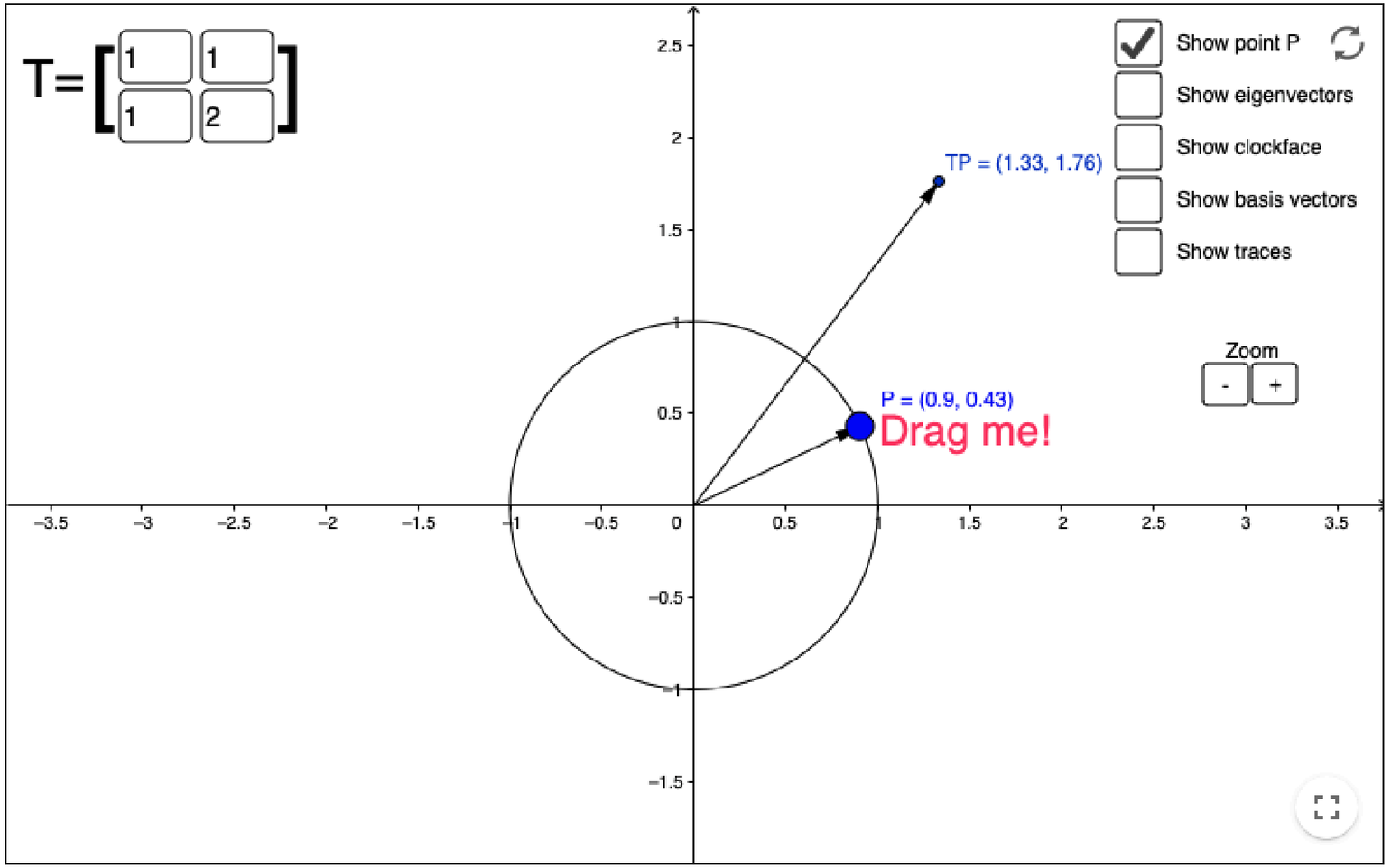
Eigenvector가 없는 linear transformation

real-valued eigenvalue가 존재하지 않으면 eigenvector도 없다.



Eigenvectors and linear transformations

play with this → https://www.geogebra.org/m/mdvN0HTt



Diagonal matrix

Note: not all diagonal matric

$$A = \begin{bmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ 0 & a_{22} & 0 & \cdots & 0 \\ 0 & 0 & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn} \end{bmatrix}$$

```
[15] D = np.diag([3, 4, 5])
     print(D)

     [[3 0 0]
      [0 4 0]
      [0 0 5]]
```

Diagonal matrix

같은 matrix를 여러번 곱하기 쉽다

$$\underbrace{\begin{bmatrix}3 & 0\\ 0 & 2\end{bmatrix}\cdots\cdots\begin{bmatrix}3 & 0\\ 0 & 2\end{bmatrix}\begin{bmatrix}3 & 0\\ 0 & 2\end{bmatrix}\begin{bmatrix}3 & 0\\ 0 & 2\end{bmatrix}}_{100\text{ times}}\begin{bmatrix}x\\ y\end{bmatrix} = \begin{bmatrix}3^{100} & 0\\ 0 & 2^{100}\end{bmatrix}\begin{bmatrix}x\\ y\end{bmatrix}$$

$$\underbrace{\begin{bmatrix}3 & 4\\ 1 & 1\end{bmatrix}\cdots\cdots\begin{bmatrix}3 & 4\\ 1 & 1\end{bmatrix}\begin{bmatrix}3 & 4\\ 1 & 1\end{bmatrix}\begin{bmatrix}3 & 4\\ 1 & 1\end{bmatrix}}_{100\text{ times}}\begin{bmatrix}x\\ y\end{bmatrix}$$

?

Diagonal matrix

같은 matrix를 여러번 곱하기 쉽다

$$\begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}^{n} = \left\{ \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} \right\}^{n}$$

$$= \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}^{n} \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

Diagonal matrix

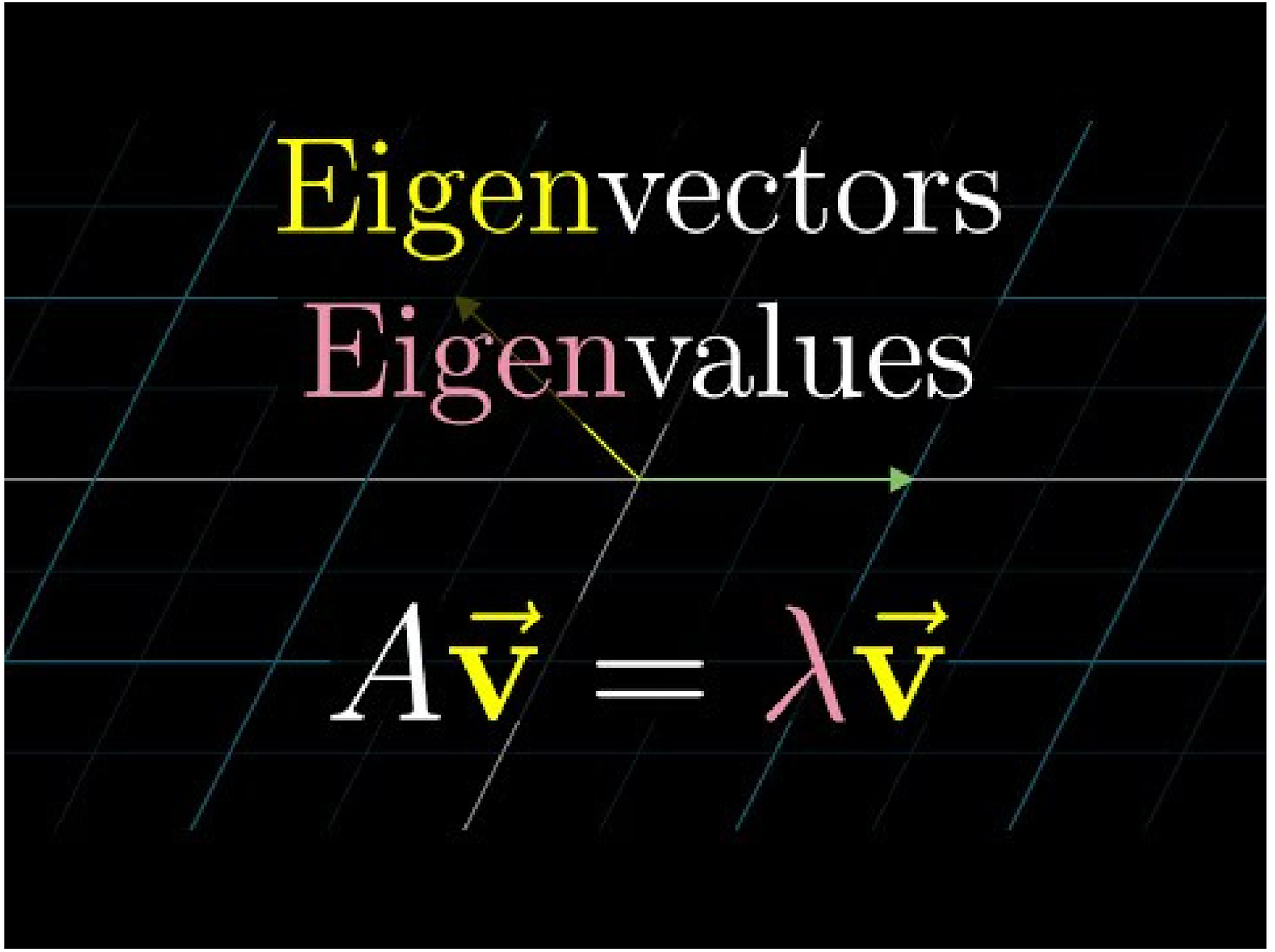
원래좌표계를 eigenbasis로 표현하고 D를 여러번 곱하고 다시 원래좌표계로 돌린다.

$$\begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}\begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1} \begin{bmatrix}x\\y\end{bmatrix}$$

$$\begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1}\begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1}\begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}\begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1} \begin{bmatrix}x\\y\end{bmatrix}$$

$$\begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}x\\y\end{bmatrix} = \begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}\left\{\begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1}\begin{bmatrix}3 & 1\\0 & 2\end{bmatrix}^n \begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}\right\}\begin{bmatrix}1 & -1\\0 & 1\end{bmatrix}^{-1} \begin{bmatrix}x\\y\end{bmatrix}$$

Diagonal matrix



동영상 링크 모음

Linear transformations and matrices: https://youtu.be/kYB8IZa5AuE

The determinant: https://youtu.be/Ip3X9LOh2dk

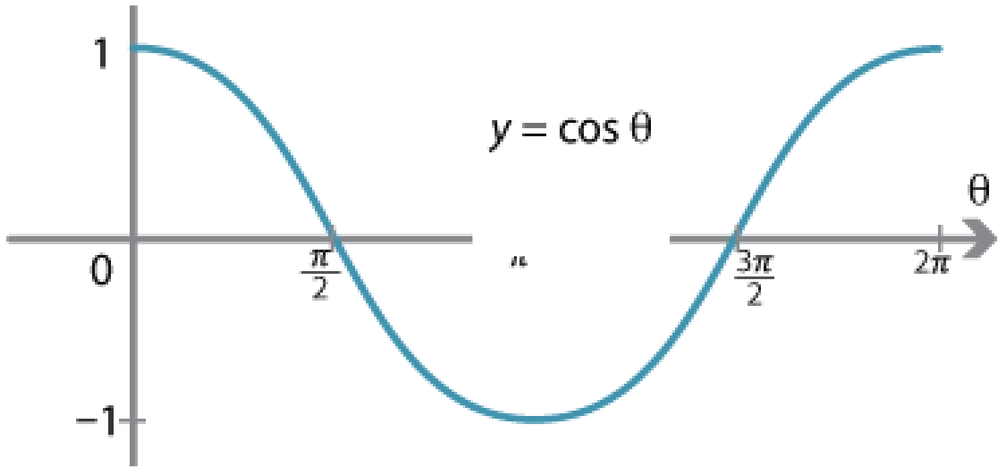
Eigenvectors and eigenvalues: https://youtu.be/PFDu9oVAE-g

orthogonal vectors

A vector x and y are orthogonal to each other if $\boldsymbol{x}^\top \boldsymbol{y} = 0$

If both vectors have nonzero norm, they are at a 90 degree angle to each other.

$$\boldsymbol{x}^\top \boldsymbol{y} = ||\boldsymbol{x}||_2 ||\boldsymbol{y}||_2 \cos\theta$$



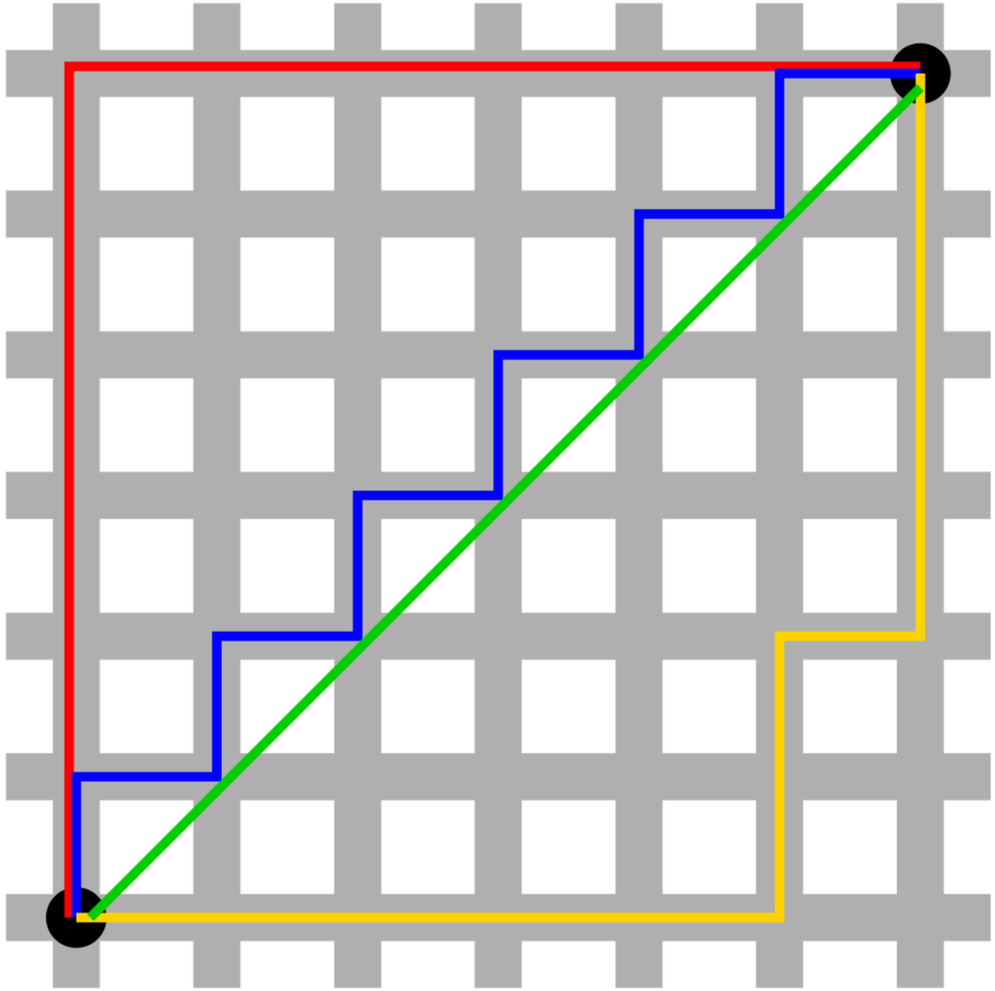
orthonormal matrix

A square matrix whose column vectors

- are orthogonal to each other
- have unit norm
  orthogonal matrix라고도 부른다.

Norm

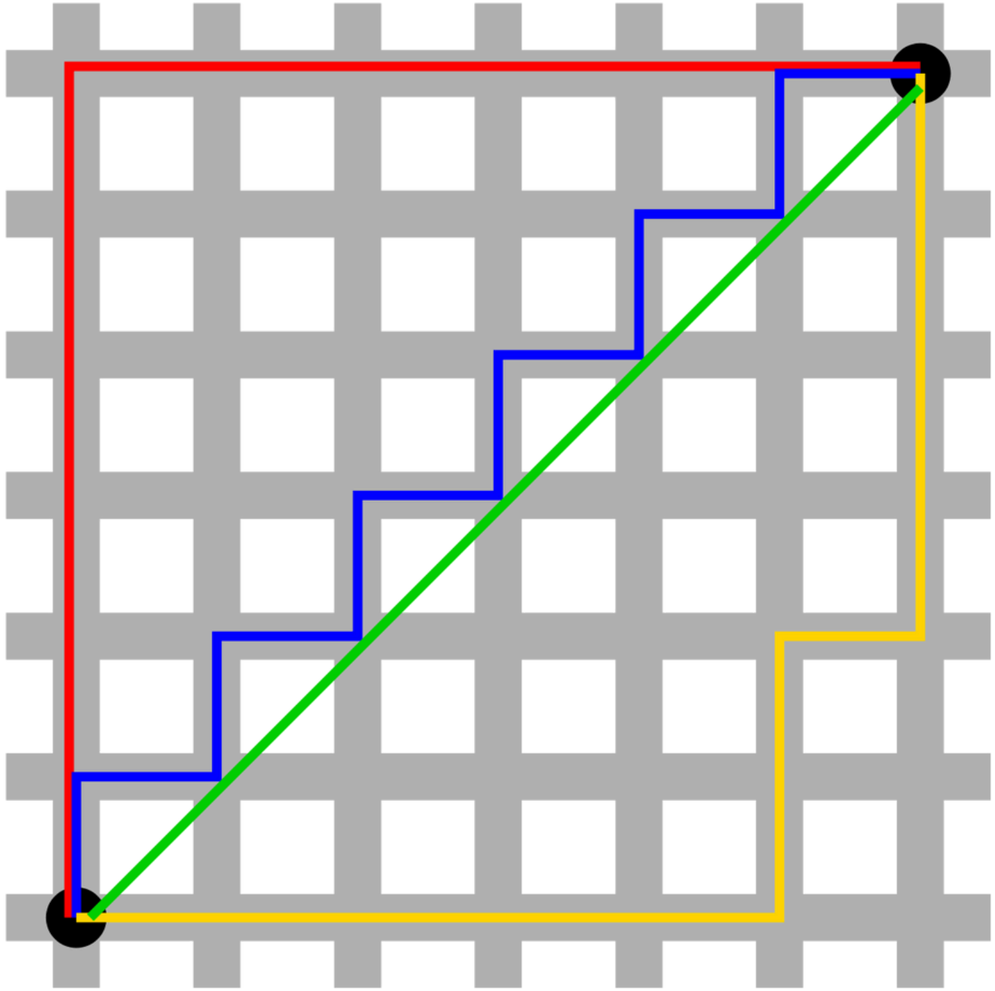
Norm = distance from the origin



L1-norm

i.e., Manhattan norm

$$\|\boldsymbol{x}\|_1 := \sum_{i=1}^{n} |x_i|$$



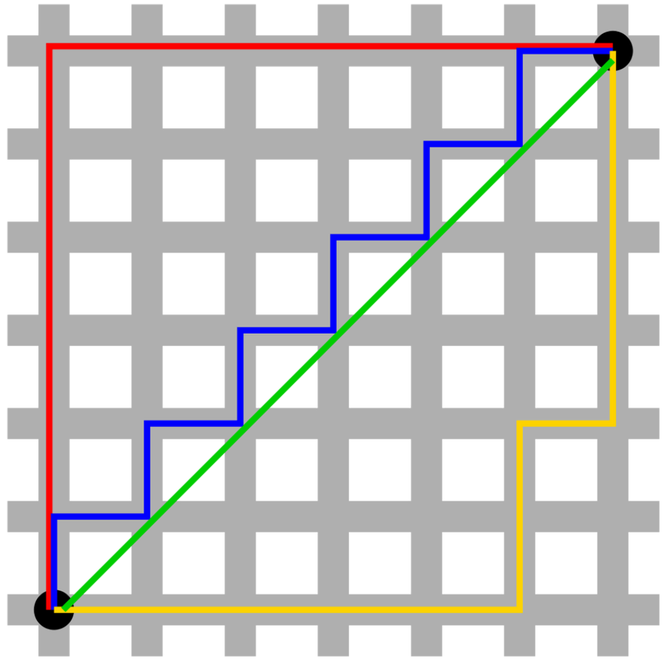
L2-norm

i.e., Euclidean norm

$$\|\boldsymbol{x}\|_2 := \sqrt{x_1^2 + \cdots + x_n^2}$$

```
[13] x = np.array([3, 4])
     print(np.linalg.norm(x))
```

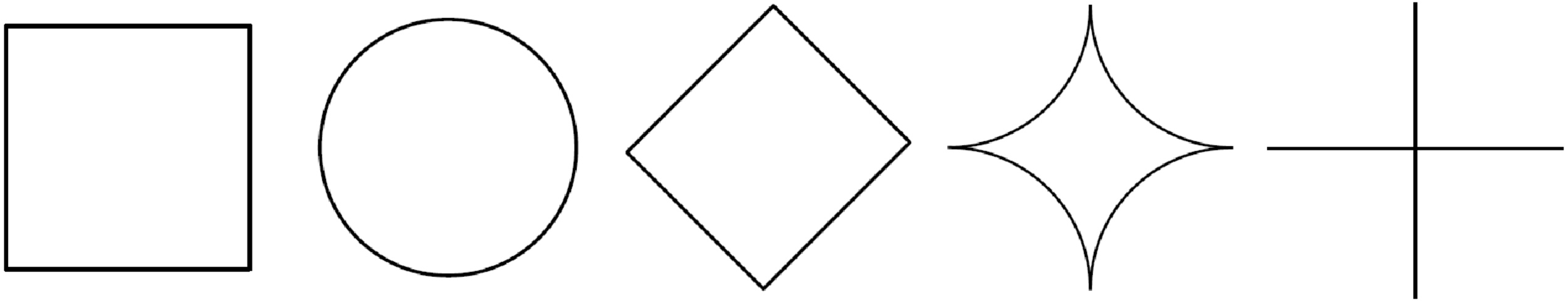
```
5.0
```



Lp-norm

L^pnorm = $||\boldsymbol{x}||_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$

L: Henri Lebesgue

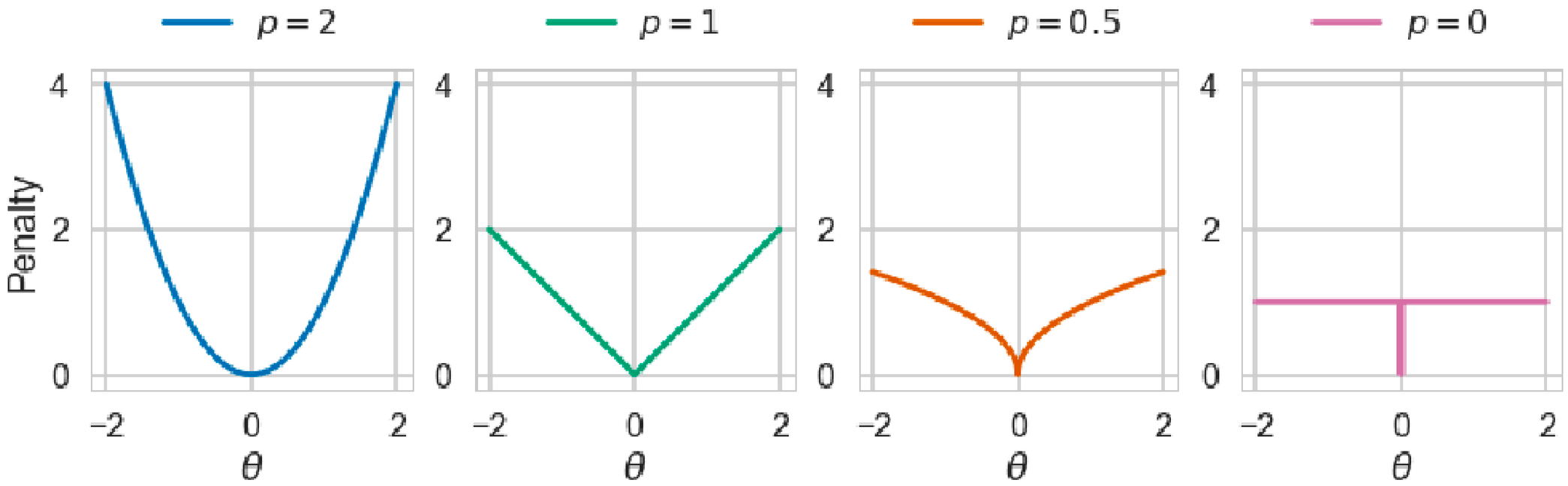


$p = \infty$ $p = 2$ $p = 1$ $0 < p < 1$ $p = 0$

L0-norm

count the number of non-zero elements

$$||\boldsymbol{x}||_p = \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}$$



L∞-norm

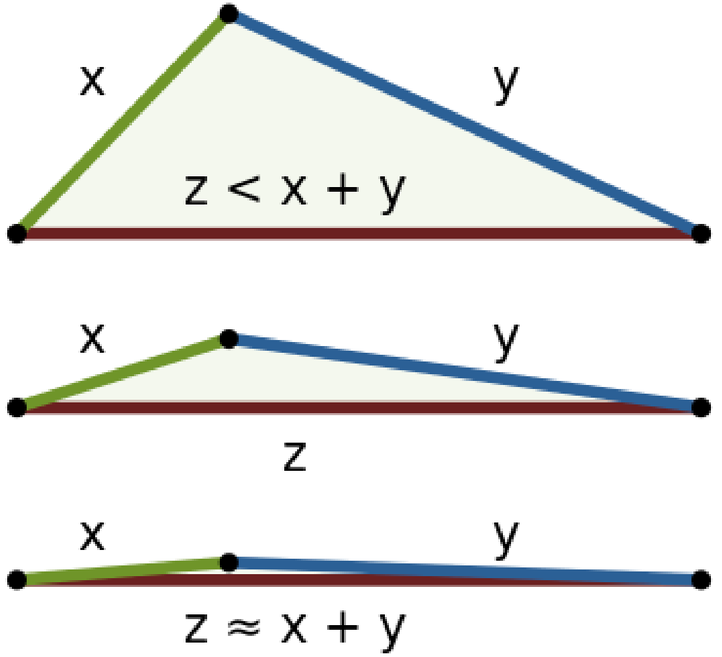
maximum of all elements

$$||\boldsymbol{x}||_p = \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}$$

$$|\mathbf{x}|_\infty = \max_i |x_i|.$$

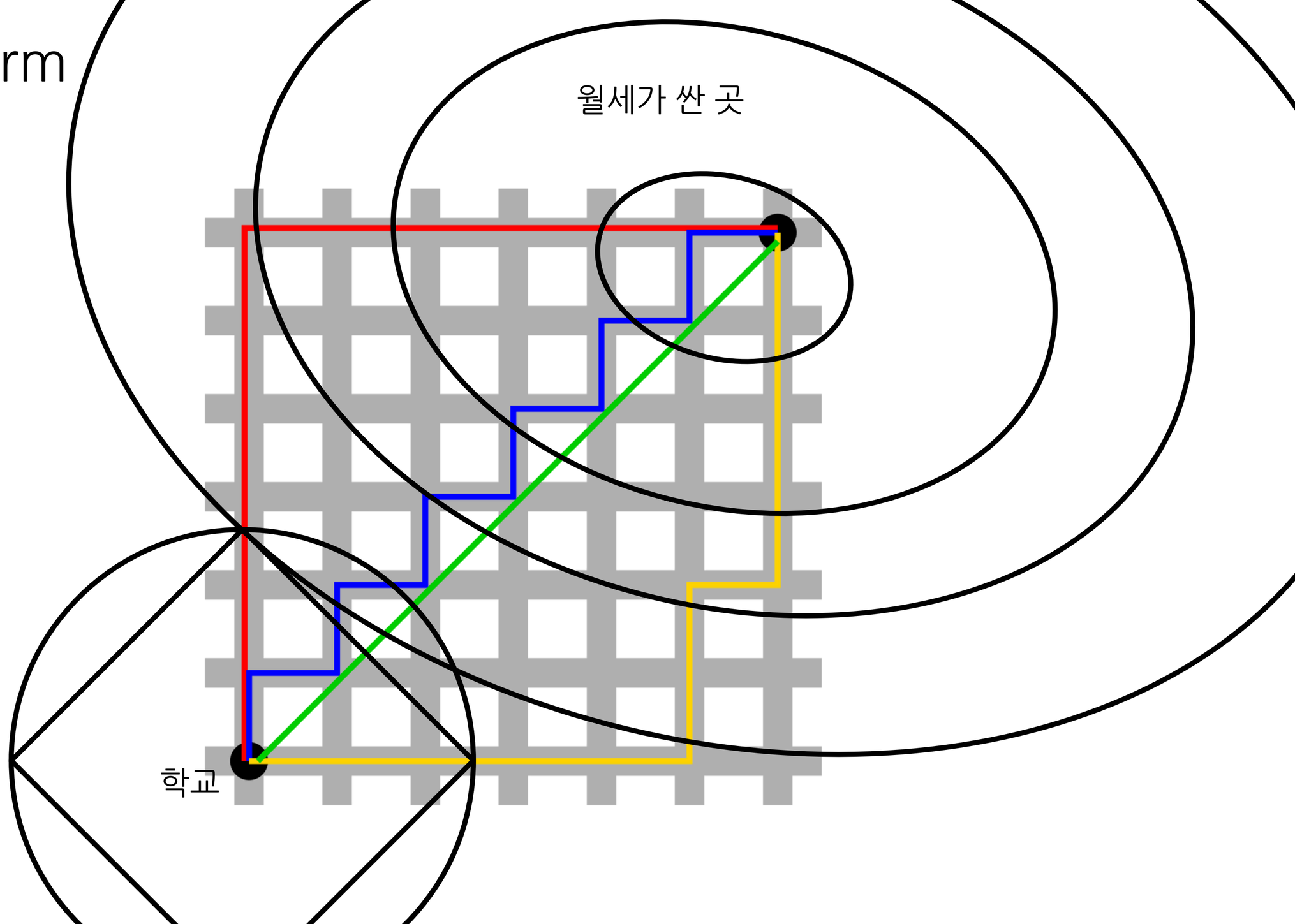
Norm = distance from the origin

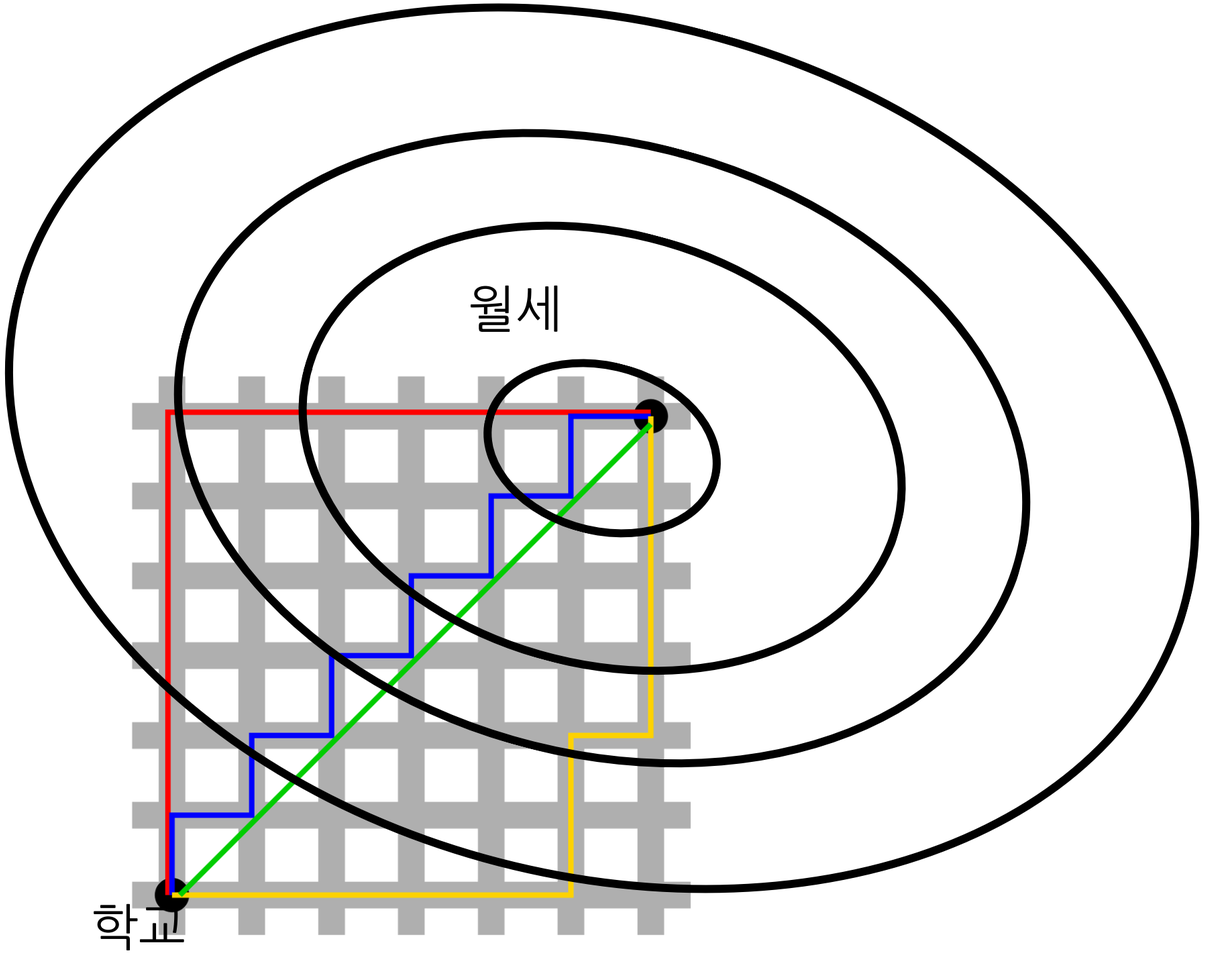
- $f(\boldsymbol{x}) = 0 \Rightarrow \boldsymbol{x} = \mathbf{0}$
- $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$ (the **triangle inequality**)
- $\forall \alpha \in \mathbb{R}, f(\alpha \boldsymbol{x}) = |\alpha| f(\boldsymbol{x})$



L1 and L2 norm

자전거 vs 헬기

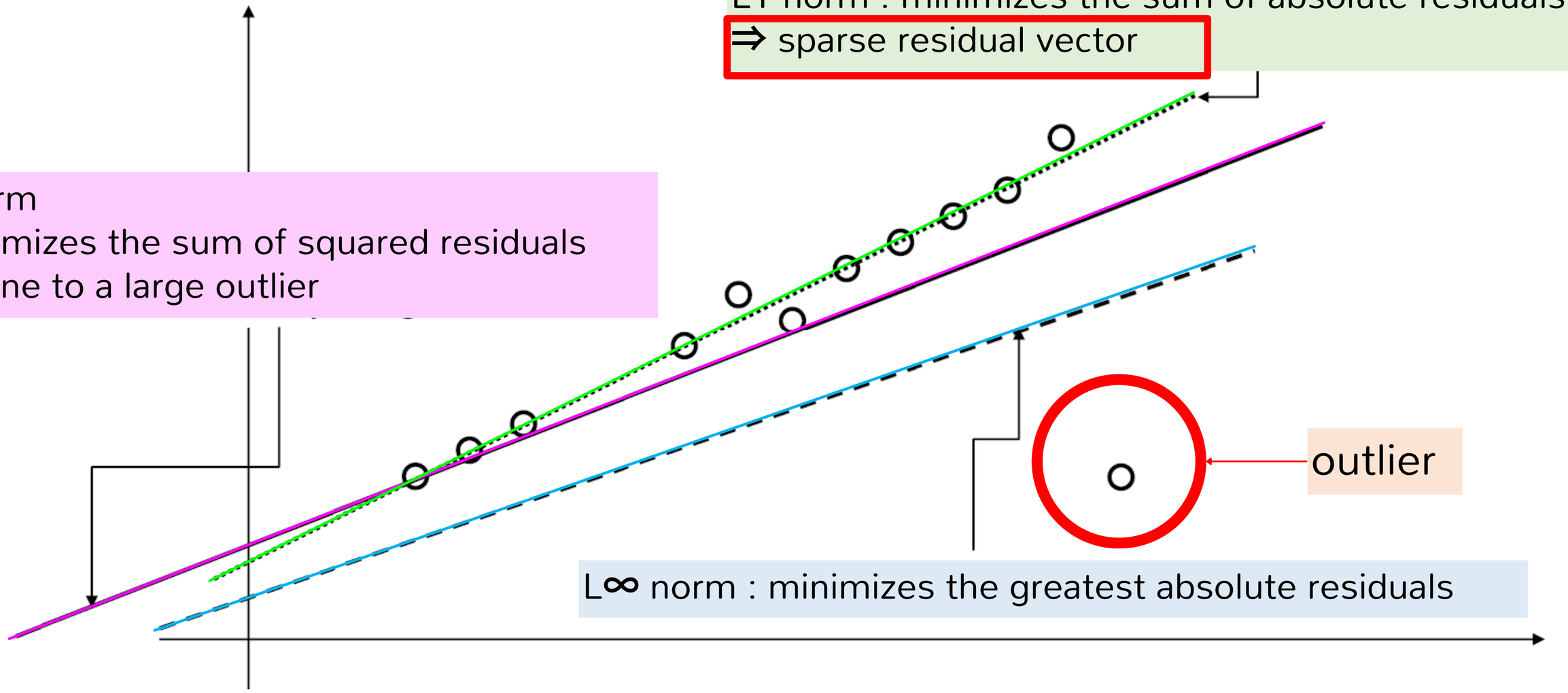



월세
학교

Fitting with different norms

L1 norm : minimizes the sum of absolute residuals
⇒ sparse residual vector

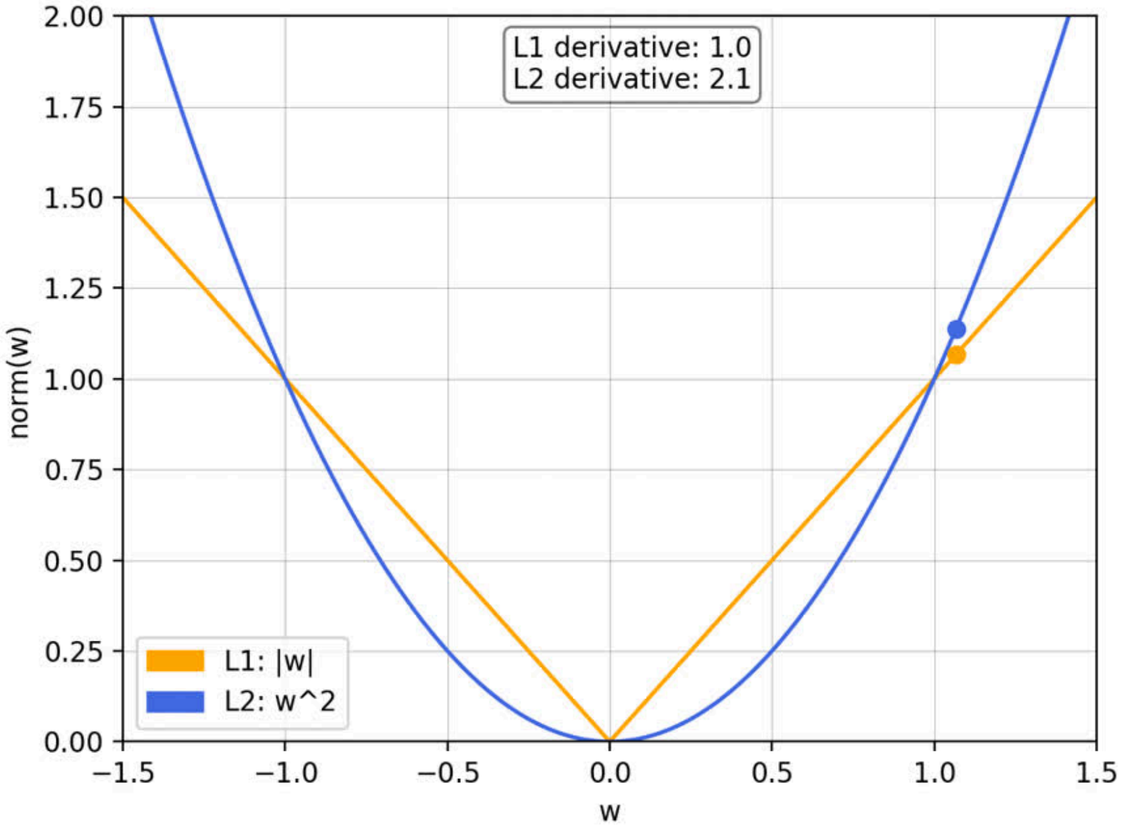
L2 norm
: minimizes the sum of squared residuals
⇒ prone to a large outlier



L∞ norm : minimizes the greatest absolute residuals

L1-norm vs L2-norm

exact 0이냐 0에 가깝냐가 큰 차이인 경우
L1-norm을 사용한다.



The squared L2-Norm

is more convenient to work with mathematically and computationally than the L2-norm

Linear regression

급여 맞추기 문제

모델

- 급여 = 기본급 + 연차 x 근속연수당 승봉
  (raise per year)
- 83000 = b + 8.7 x r
- 88000 = b + 8.1 x r
- …

급여	근속연수
83000	8.7
88000	8.1
48000	0.7
76000	6
…	…



A system of linear equations

Matrix-vector product 표기법은 여러개의 등식을 간단하게 표기할 수 있다.

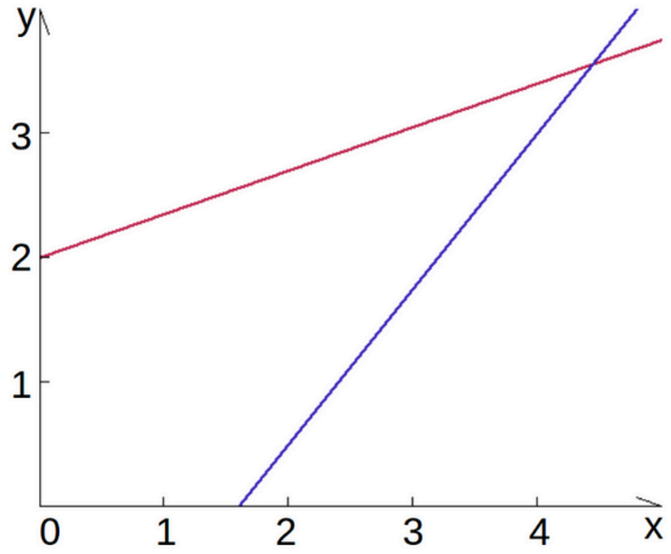
$$\mathbf{Ax} = \mathbf{b}$$

$$\begin{cases} -0.35x + y = 2 \\ 2.5x - 2y = 4 \end{cases}$$

$$\begin{bmatrix} -0.35 & 1 \\ 2.5 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

The same system in matrix form

The same system in form of a plot



A system of linear equations

A and **b** are known
x is not known
find **x**

$$Ax = b$$

$$x = ?$$

$$\boldsymbol{Ax = b}$$

$$\boldsymbol{A^{-1}Ax = A^{-1}b}$$

$$\boldsymbol{I_n x = A^{-1}b}$$

$$\boldsymbol{x = A^{-1}b}$$

Linear regression

Find a linear model that best expresses the data

- 급여 = 기본급 + 연수 x 근속연수당 승봉
  (raise per year)
- s = b + y x r
- 83000 = b x 1 + 8.7 x r
- 88000 = b x 1 + 8.1 x r
- …

$$\begin{bmatrix} 8.7 & 1 \\ 8.1 & 1 \\ \dots & \dots \end{bmatrix} \begin{bmatrix} r \\ b \end{bmatrix} = \begin{bmatrix} 83000 \\ 88000 \\ \dots \end{bmatrix}$$



급여	근속연수
83000	8.7
88000	8.1
48000	0.7
76000	6
69000	6.5
76000	7.5
60000	2.5
83000	10
48000	1.9
63000	4.2

Linear regression

$$\begin{bmatrix} 8.7 & 1 \\ 8.1 & 1 \\ \dots & \dots \end{bmatrix} \begin{bmatrix} r \\ b \end{bmatrix} = \begin{bmatrix} 83000 \\ 88000 \\ \dots \end{bmatrix}$$

Our objective is to minimize squared error

$$\boldsymbol{Ax = s}$$

$$\boldsymbol{A\hat{x} = \hat{s}}$$

$$\text{error} = s - \hat{s}$$

$$\text{squared error} = (s - \hat{s})^2$$



Linear regression

Our objective is to minimize squared error

$$
\begin{aligned}
&\nabla_x (s - \hat{s})^2 = 0 \\
&= \nabla_x (s - Ax)^2 \\
&= \nabla_x (s - Ax)^T (s - Ax) \\
&= \nabla_x (s^T - x^T A^T)(s - Ax) \\
&= \nabla_x (s^T s - x^T A^T s - s^T Ax + x^T A^T Ax) \\
&= \nabla_x (s^T s - 2 s^T Ax + x^T A^T Ax) \\
&= -2 s^T A + 2 x^T A^T A \\
&2 x^T A^T A = 2 s^T A \\
&A^T Ax = A^T s \\
&x = (A^T A)^{-1} A^T s
\end{aligned}
$$

$$
\begin{bmatrix} 8.7 & 1 \\ 8.1 & 1 \\ \dots & \dots \end{bmatrix} \begin{bmatrix} r \\ b \end{bmatrix} = \begin{bmatrix} 83000 \\ 88000 \\ \dots \end{bmatrix}
$$



scalar differentiation

$$y = f(x) = ax$$

$$\begin{aligned}
\frac{\partial y}{\partial x} &= \frac{\partial f}{\partial x} \\
&= \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \\
&= \lim_{h \to 0} \frac{a(x+h) - ax}{h} \\
&= \lim_{h \to 0} \frac{ax + ah - ax}{h} \\
&= \lim_{h \to 0} \frac{ah}{h} \\
&= \lim_{h \to 0} a \\
&= a
\end{aligned}$$

scalar differentiation

$$y = f(x) = x^2$$

$$\begin{aligned}
\frac{\partial y}{\partial x} &= \frac{\partial f}{\partial x} \\
&= \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \\
&= \lim_{h \to 0} \frac{(x+h)^2 - x^2}{h} \\
&= \lim_{h \to 0} \frac{x^2 + 2hx + h^2 - x^2}{h} \\
&= \lim_{h \to 0} \frac{2hx + h^2}{h} \\
&= \lim_{h \to 0} 2x + h \\
&= 2x
\end{aligned}$$

Matrix differentiation

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{A}$$

$$y = ax$$

$$\frac{\partial y}{\partial x} = a$$

Matrix differentiation

$$\mathbf{y} = \mathbf{A}\mathbf{x} \quad \Rightarrow \quad y_i = \sum_{k=1}^{n} a_{ik} x_k$$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{A} \quad \Leftarrow \quad \frac{\partial y_i}{\partial x_j} = a_{ij}$$

Matrix differentiation

$$\alpha = \mathbf{y}^T \mathbf{A} \mathbf{x} \quad \Rightarrow \quad \mathbf{w}^T = \mathbf{y}^T \mathbf{A}$$

$$\alpha = \mathbf{w}^T \mathbf{x}$$

$$\frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{y}^T \mathbf{A} \quad \Leftarrow \quad \frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{w}^T = \mathbf{y}^T \mathbf{A}$$

Matrix differentiation

$$\alpha = \mathbf{y}^T \mathbf{A} \mathbf{x} \Rightarrow \alpha = \alpha^T = \mathbf{x}^T \mathbf{A}^T \mathbf{y}$$

$$\frac{\partial \alpha}{\partial \mathbf{y}} = \mathbf{x}^T \mathbf{A}^T \Leftarrow$$

Matrix differentiation

$$\alpha = \mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} \quad \Rightarrow \quad \alpha = \sum_{j=1}^{n}\sum_{i=1}^{n} a_{ij}x_i x_j$$

$$\frac{\partial \alpha}{\partial x_k} = \sum_{j=1}^{n} a_{kj}x_j + \sum_{i=1}^{n} a_{ik}x_i$$

$$\frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{x}^{\mathsf{T}}\left(\mathbf{A} + \mathbf{A}^{\mathsf{T}}\right) \quad \Leftarrow \quad \frac{\partial \alpha}{\partial \mathbf{x}} = \mathbf{x}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}} + \mathbf{x}^{\mathsf{T}}\mathbf{A} = \mathbf{x}^{\mathsf{T}}\left(\mathbf{A}^{\mathsf{T}} + \mathbf{A}\right)$$

$$= \nabla_x (s^T s - 2s^T Ax + x^T A^T Ax)$$
$$= -2s^T A + 2x^T A^T A$$
$$2x^T A^T A = 2s^T A$$
$$A^T Ax = A^T s$$
$$x = (A^T A)^{-1} A^T s$$

Moore-Penrose pseudoinverse

A의 inverse는 없어도 pseudoinverse는 항상 있다.

$$
\begin{aligned}
Ax &= b \\
x &= A^{-1}b \\
A^{+} &= (A^{T}A)^{-1}A^{T} \\
A^{+}A &= (A^{T}A)^{-1}(A^{T}A) = I \\
x &= A^{+}b = (A^{T}A)^{-1}A^{T}b
\end{aligned}
$$

연습

```
salary_and_year = [[83000, 8.7],
                   [88000, 8.1],
                   [48000, 0.7],
                   [76000, 6],
                   [69000, 6.5],
                   [76000, 7.5],
                   [60000, 2.5],
                   [83000, 10],
                   [48000, 1.9],
                   [63000, 4.2]]
```

5년 일한 사람의 급여는?

$$\boldsymbol{Ax} = \boldsymbol{b}$$

$$\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$$

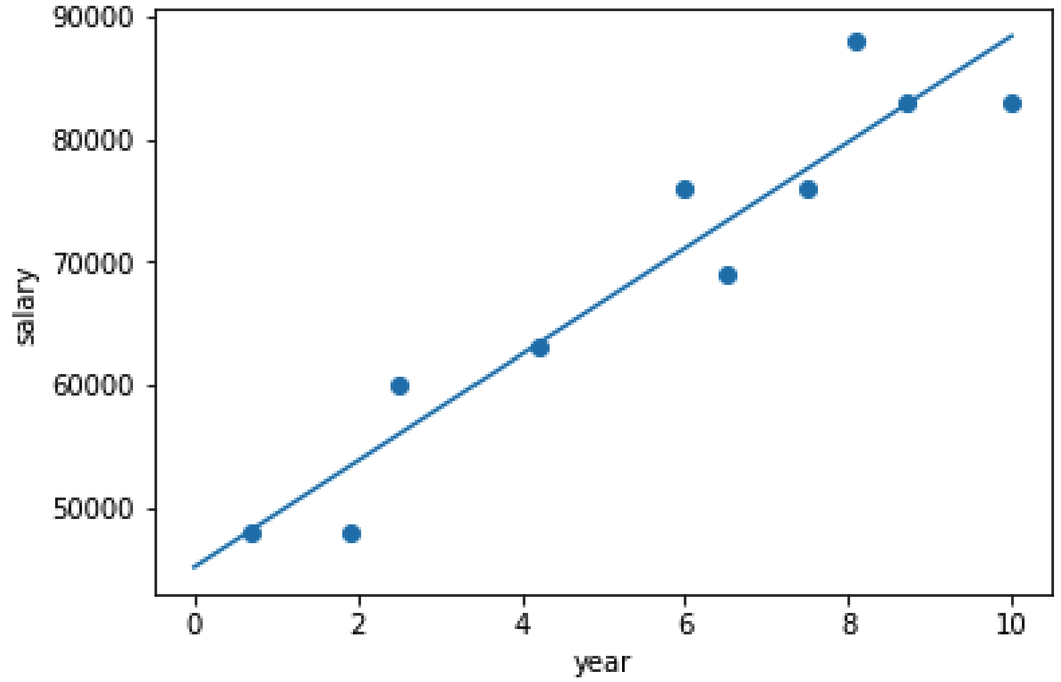
$$x = A^{+}b = (A^{T}A)^{-1}A^{T}b$$



급여	근속연수
83000	8.7
88000	8.1
48000	0.7
76000	6
69000	6.5
76000	7.5
60000	2.5
83000	10
48000	1.9
63000	4.2

example output

```
raise = 4327.65608685401
basis = 45121.84935274899
```




```
salary at 5: [redacted]
student id: 0000000000
name: 어영정
```

더 알아보기

error를 SSE 대신 L1으로 고치면 어떻게 풀 수 있을까?

끝